CricField

Engineering, Architecture, Product & Career Handbook

Reverse-engineered audit of the Data-Insight-Engine repository · 2026-10-13

Evidence labels: ✅ verified · 🟡 partial · 🟠 mocked/hardcoded · 🟣 planned · 🔴 defect · ❔ unverified

Contents

1. 00 EXECUTIVE OVERVIEW
2. AUDIT SUMMARY
3. 01 REPOSITORY AUDIT
4. 02 SYSTEM ARCHITECTURE
5. ARCHITECTURE DIAGRAMS
6. 03 TECHNOLOGY STACK
7. 04 DATABASE ENGINEERING
8. 05 DATA PIPELINES
9. 06 SCOUTING STATISTICS
10. 07 FRONTEND ENGINEERING
11. 08 API ENGINEERING
12. 09 AI ENGINEERING AUDIT
13. 10 TESTING AND DEBUGGING
14. 11 PERFORMANCE SECURITY
15. 12 FEATURE WALKTHROUGHS
16. 20 UI DESIGN AUDIT
17. 21 UX ENGINEERING
18. 22 PRODUCT DESIGN CASE STUDIES
19. 23 DATA VISUALIZATION DESIGN
20. 24 PRODUCT MANAGEMENT
21. 25 BUSINESS GROWTH STRATEGY
22. 26 SOLO BUILDER ASSESSMENT
23. 27 DESIGN PORTFOLIO READINESS
24. 13 SKILL MATRIX
25. 14 CAREER ROLE MAPPING
26. 28 COMPLETE CAREER ROLE MATRIX
27. 15 INTERVIEW QUESTION BANK
28. 29 INTERVIEW AND DESIGN CHALLENGES
29. 16 INDEPENDENT CHALLENGES
30. 17 LEARNING ROADMAP
31. 18 RESUME EVIDENCE
32. 19 TECHNICAL GLOSSARY

00 — Executive Overview

CricField / Data-Insight-Engine — Engineering, Architecture, Product & Career Handbook Audit date: 2026-10-12/13 · Scope: the repository at `Data-Insight-Engine/` including uncommitted work · Method: source reading, read-only database queries, test and typecheck runs, live API benchmarks and browser inspection on test ports. No application code, database, configuration or asset was modified.

What CricField is

An IPL-first cricket analytics product built as a pnpm monorepo:

- a **React 19 + Vite** single-page app with ≈50 pages;
- an **Express 5** API with **123 endpoints**;
- two **SQLite** warehouses — `sqlite.db` (1,243 IPL matches, 295,732 deliveries) and `global.db` (3,309 matches across IPL, SMAT, BBL, SA20, CPL and ILT20; 771,965 deliveries);
- a TypeScript **ETL** layer.

It serves fans (team HQ, player pages, scorecards, natural-language stats) and aims at franchise analysts (cross-league Scout Lab, weakness scans, simulations, auction valuation).

The five things to know

1. **The strongest engineering is the data work.** The global engine downloads Cricsheet archives with hashing, loads incrementally, preserves unknown fields, resolves identity by registry id and gates on 118 validation checks (0 failures). Its 23 tests pass.
2. **The analytics are statistics, not machine learning.** They are league-relative indices, credibility shrinkage, Wilson and normal intervals, z-tests with Bonferroni correction, Monte Carlo with Platt calibration, kNN pricing and MILP. The "AI Analyst" is deterministic. The only LLM use is an optional, ungrounded Gemini call, and its no-key fallback shows **invented statistics**.
3. **Correctness gaps exist in shipped UI and the tests show it.**
   - Scout Lab's default bowling ranking is inverted, unqualified players are listed, and roles always display "Unclassified".
   - 4 of the 15 Step-3 regression tests now fail, including 20 player pages that show empty recent matches.
   - The API and web typechecks fail (2 and 11 errors).
4. **Security and scale need work before any public launch.**
   - `POST /api/ipl/sync-csv` reads arbitrary server paths, with no auth and no rate limits.
   - Synchronous SQLite serialises requests: 10 concurrent Advanced Stats queries took 24 s.
   - The JavaScript bundle is 4.1 MB.
5. **Process evidence is real but thin in git.** Plans, patches, before/after reports and approval gates are documented in `docs/global-data-engine/`. But only 19 commits exist; one checkpoint added 1.8 M lines, and 274 paths are uncommitted.

Implementation status at a glance

Area	Status
Global data engine + scouting maths	✅ verified (tests, validation, worked examples)
IPL warehouse	🟡 works, not reproducible from the repo; known residuals documented
Decision modules (Ask, Weakness, Pressure, Sims, Squad DNA, War Room, Playing XI)	✅ run deterministically; 🟡 little validation; War Room backtest weak
Scout Lab UI/API	🟡 uncommitted; three correctness defects
Fan surfaces (Home, Dugout, Beyond XI, Pulse, Shop)	🟡 mostly presentational; Pulse adds random noise
AI / LLM	🔴 optional Gemini; fabricated fallback
Tests	🟡 41 data/logic tests; no API/UI/E2E tests; no CI
Deployment	Replit autoscale config; no Vercel/CI/containers; data files not packaged

What this means for your career

CricField is strong, directly relevant evidence for **sports data analyst, data analyst, analytics engineer, data engineer and technical product analyst** roles. That holds **once you can reproduce the core SQL, metrics, statistics and ETL design yourself** (Chapters 15–17).

It is weak evidence for ML/AI engineering, UI/UX design and user research until you build the missing artefacts (Chapters 27–28).

Use the evidence-backed resume bullets in Chapter 18 and avoid the five claims listed there as unsafe.

How to read the handbook

Start with AUDIT-SUMMARY.md, then:

- Chapters **04** → **05** → **06** cover data and statistics.
- Chapters **02, 08, 12** cover how requests flow.
- Chapters **13–18 and 26–29** cover career preparation.

The navigation index is README.md.

Audit Summary — Major Findings, Risks and Missing Knowledge

A. Verified strengths

#	Strength	Evidence
1	Reproducible, validated six-league warehouse (incremental, hashed, lineage, 118 checks, 0 fail)	`scripts/src/global/*`; `reports/validation-2026-10-10.md`
2	Correct, tested cricket metric definitions	`metrics.ts`, `metrics.test.ts` (pass)
3	Transparent scouting maths with uncertainty (index, shrinkage, Wilson/normal CI, tiers)	`scout-service.ts`; worked examples reproduced
4	Identity by registry id; ambiguous names never merged	`resolvePlayer("Yuvraj Singh")` → 3 candidates, no player
5	Deterministic NL querying with a validated query boundary	`nl-parser.ts`, `stats-engine.ts`; live 78 ms answer matching independent recount
6	Honest documentation of residual issues and before/after evidence	`docs/global-data-engine/step3/*`
7	Leakage-aware War Room backtest that reports its own weakness	`auction-war-room.ts`
8	Good empty/demo/error states in Decision modules	`module-shell.tsx`

B. Defects and risks (ranked)

Rank	Finding	Severity	Chapter
1	`POST /api/ipl/sync-csv` reads arbitrary server file paths (unauthenticated)	High	8, 11
2	Gemini fallback shows invented statistics labelled as model output	High (trust)	9
3	Scout bowling default order inverted; `minBalls` not applied; role "Unclassified"; CI labels/units wrong	High (correctness)	6, 23
4	4 / 15 Step-3 regression tests fail (20 empty player pages; unexplained stat diffs)	High	10
5	Typecheck failures (API 2, web 11) block the root build script	Medium	10
6	Synchronous SQLite: 10 concurrent heavy queries → 24 s; `no-store` on all responses	Medium–High (availability)	11
7	`global.db` + raw JSON gitignored; deploys lack data; source-run crash (`__dirname`)	Medium	2
8	IPL warehouse not reproducible; ingest still has the float over/ball bug and random delivery types	Medium	5
9	Scraped, unlicensed sources (commentary, live scores, photos, logos); affiliation-implying copy	High (legal) for any commercial use	25
10	Accessibility: zoom disabled, ~900 low-contrast class usages, 1,558 tiny-text usages	Medium	20
11	`/api/ipl/overview` mixes hardcoded literals with live counts	Low–Medium	8
12	Shared global auction state; 1 s polling	Medium	8, 11
13	`drizzle-kit push` on every Replit merge against the production DB	Medium	5
14	Weak git evidence (19 commits; 274 uncommitted paths)	Career risk	1

C. Validation gaps (claims you cannot yet make)

Cross-league equivalence · interval coverage · out-of-sample validity of weakness findings · holdout calibration of the win model · predictive value of War Room signals (ρ 0.17, n 9) · any usability or engagement outcome · accessibility compliance · production scalability.

D. Missing knowledge to build (yours, not the code's)

1. SQL for Q1–Q9 without help.
2. Cricket metric rules applied on paper.
3. Shrinkage and Wilson-interval derivations.
4. ETL design principles (idempotency, lineage).
5. React Query data flow.
6. Express + Zod endpoint design and testing.
7. Reading `EXPLAIN QUERY PLAN`.
8. Basic web security (path traversal, injection).
9. Accessibility fundamentals.
10. User-research methods.
11. Model validation (time splits, calibration).

E. Recommended next 10 actions

1. Remove the `/sync-csv` route.
2. Fix Scout sort, `minBalls` and role display, plus the CI labels; add API tests for each.
3. Remove the Gemini fallback numbers.
4. Explain or fix the 4 failing regression tests.
5. Fix the typecheck errors.
6. Commit the uncommitted work in logical commits.
7. Make historical API responses cacheable, and precompute Advanced Stats.

8. Fix the accessibility P0s.
9. Complete Challenges C1–C5 and C10 without AI.
10. Run five user interviews for Scout Lab.

01 — Repository Audit (Phase A)

> **Audit date:** 2026-10-12. **Auditor stance:** everything below was checked against files on disk, the two SQLite databases (opened read-only) and commands that were actually run. Where something could not be verified it is labelled **Unverified**.
>
> **Evidence labels used throughout the handbook**
>
> | Label | Meaning |
> |---|---|
> | ✅ **Verified** | Code exists *and* was exercised (test, query, build, or live request) during this audit. |
> | 🟡 **Partial** | Code exists and partly works, or works only for some inputs. |
> | 🟠 **Mocked / hardcoded** | The UI shows values that do not come from the data, or come from a frozen snapshot. |
> | 🟣 **Planned** | Described in docs or comments, not implemented. |
> | 🔴 **Defect / debt** | Known bug, failing test, or risky construction. |
> | ❔ **Unverified** | Plausible from reading code, not executed in this audit. |

---

1. Where the project lives

Item	Value
Outer folder	`C:\Users\KIIT0001\Downloads\Data-Insight-Engine\` (contains a 137 MB `Data-Insight-Engine.zip`, a second `sqlite.db` of 28 KB, and `scratch/`)
Actual repository	`Data-Insight-Engine/Data-Insight-Engine/` (git root)
Package manager	**pnpm workspaces** (`pnpm-workspace.yaml`, `pnpm-lock.yaml`)
Runtime	**Node.js 24.10** (verified with `node --version`). `node:sqlite` is used, which requires Node ≥ 22.5.
Original host	**Replit** (`.replit`, `replit.nix`, `replit.md`, commits titled "Published your App")
Vercel	**No Vercel configuration exists in the repository** (no `vercel.json`, no `.vercel/`). Any claim that CricField is deployed on Vercel is **Unverified** from the repo.

Git history

- **19 commits**, dated 2026-05-06 → 2026-10-09 (`git log`).
  - 12 commits in May 2026 (Replit Agent era: "Initial commit", "Add IPL analytics engine…", "Published your App" ×3).
  - 7 commits on 2026-10-09, including `9289895 checkpoint: pre-video-engine-architecture-audit`, which alone added **2,041 files / 1.81 M lines** — i.e., months of work were committed in one checkpoint, so **git history cannot show how most features evolved**.
- **274 paths are uncommitted** right now (`git status --short`), including the whole Scout Lab API (`routes/scout.ts`, `services/scout/`), the Scout pages, the global data engine (`scripts/src/global`, `scripts/src/scout`, `scripts/src/step3*`), `docs/global-data-engine/`, and a modified `sqlite.db`.
- Author on all October commits: `Srihari1806`.

> **Interview implication.** You cannot point an interviewer at a commit history that shows your iteration. The best evidence of process is the `docs/global-data-engine/` folder (plans, before/after reports, patches, regression results). Committing the current work in logical chunks would materially improve your evidence.

---

2. Size and layout

Measure	Value	How measured
Tracked files	2,270	`git ls-files | wc -l`
Hand-written source (TS/TSX/MJS/PY in `src` folders)	≈ **97,900 lines**	`wc -l` over `artifacts/*/src`, `scripts/src`, `lib/*/src`
Largest files	`pages/stats.tsx` 4,411 · `pages/player.tsx` 3,241 · `services/stats-aggregator.ts` 3,028 · `pages/shop.tsx` 1,803 · `pages/intelligence.tsx` 1,763	`wc -l`
API endpoints	**123** route handlers in 9 route files	grep of `router.get/post`
Frontend routes	≈ **75** `<Route>` entries in `App.tsx` (≈ 50 distinct pages + redirects)	`artifacts/ipl-analytics/src/App.tsx`
IPL warehouse	`sqlite.db` **119 MB** + 27 MB WAL, 25 tables	read-only dump
Global warehouse	`lib/cricket-data/global/warehouse/global.db` **79 MB**, 37 tables, 4 views	read-only dump
Raw IPL match JSON	`lib/cricket-data/raw/` **1,247 files, 112 MB** (gitignored)	`ls`, `du`
Raw IPL CSV	`ipl_csv/` 31 MB	`du`
Static site data	`artifacts/ipl-analytics/public/data/` 62 MB (largest: `player-recent-matches.json` 34 MB, now split into shards)	`ls -S`

Measure	Value	How measured
Built frontend	`dist/public` 151 MB (mostly images); **one JS bundle of 4.11 MB (864 KB gzip)**, CSS 403 KB	measured with zlib

Directory map (what each top-level folder is for)

```
Data-Insight-Engine/                    ← git root, pnpm workspace
├─ artifacts/
│  ├─ ipl-analytics/        React 19 + Vite 7 SPA (the website)            ✅ main product
│  ├─ api-server/           Express 5 API on Node 24 (123 endpoints)       ✅ main product
│  ├─ image-ingestion/      Python photo pipeline (api.py, run_pipeline.py) 🟡 tooling
│  └─ mockup-sandbox/       Replit design-preview sandbox (Vite)           ⚪ not part of product
├─ lib/
│  ├─ db/                   Drizzle ORM schema + node:sqlite proxy client  ✅
│  ├─ api-spec/             OpenAPI 3.1 spec (19 paths) + Orval config     🟡 covers ~15% of API
│  ├─ api-zod/              Zod schemas generated from the spec            🟡
│  ├─ api-client-react/     React-Query hooks generated from the spec      🔴 not imported by the web app
│  └─ cricket-data/
│     ├─ raw/               1,247 Cricsheet IPL JSON files (gitignored)
│     ├─ aggregated/        derived JSON
│     └─ global/            Global engine: manifests (tracked), reports (tracked),
│                           raw archives + global.db (gitignored)
├─ scripts/                 ETL + audits. src/ is the real pipeline; the ~100 loose
│  │                        *.mjs/*.py files at scripts/ root are one-off photo
│  │                        scraping/cropping utilities.
│  └─ src/{global,scout,step3,step3_5}/   global engine, scouting, fix verification
├─ docs/
│  ├─ global-data-engine/   High-quality engineering docs (plan, reports, patches)
│  └─ architecture-handbook/   ← this handbook
├─ ipl_csv/                 Cricsheet IPL CSVs (source of sqlite.db deliveries)
├─ sqlite.db (+ -wal, -shm, .pre-delivery-fix.bak)   IPL warehouse
├─ schema.sql, migrations.sql, erd.md, database_inventory.md
├─ *.md at root (PROJECT_OVERVIEW, TECH_STACK, MASTER_ARCHITECTURE_HANDBOOK …)
│                           earlier, shorter docs from June–July 2026; several are now stale
├─ ~60 stadium/logo images at root (should live under public/ or assets/)
└─ scratch/                 ~90 ad-hoc debug scripts (tracked in part)
```

---

3. Verified technology stack

Only technologies **imported by running code** are listed. Versions come from `package.json` + the pnpm catalog.

Layer	Technology	Evidence
Language	**TypeScript ~5.9** (strict typecheck runs), some **Python 3** for photo tooling	`package.json`, `tsconfig.base.json`
Frontend framework	**React 19.1**, **Vite 7**, **wouter 3** (routing), **TanStack React Query 5** (server state)	`ipl-analytics/package.json`, `App.tsx`
Styling	**Tailwind CSS 4** (`@tailwindcss/vite`), shadcn/ui-style components on **Radix UI** primitives, `class-variance-authority`, `tailwind-merge`	`components/ui/*`, `index.css`
Motion / 3D	**framer-motion 12**, **three.js 0.170** (cricket stadium replay)	`components/cricket3d/*`
Charts	**Recharts 2.15**, d3-scale-chromatic, hand-written SVG	`components/visualizations/*`
Client state	**zustand 5** (`lib/store`), React context (`lib/team-context.tsx`)	
Backend	**Express 5**, **Zod 3** validation, **pino / pino-http** logging, `cors`	`api-server/src/app.ts`
Optimisation	**javascript-lp-solver** (mixed-integer LP for Playing XI)	`services/decision/playing-xi.ts`
Database	**SQLite** via **`node:sqlite (DatabaseSync)`**; **Drizzle ORM** in `sqlite-proxy` mode for the legacy stats path	`lib/db/src/index.ts`, `services/decision/db.ts`, `services/scout/db.ts`
ETL	TypeScript run with **tsx**; `fflate` for zip reading; Node's built-in test runner (`node --test`)	`scripts/package.json`
Build	**esbuild** (API bundle → `dist/index.mjs`), Vite (web → `dist/public`)	`api-server/build.mjs`, `vite.config.ts`
External data at runtime	Cricbuzz HTML/JSON scraping, Google News RSS, ESPNcricinfo RSS	`services/cricbuzz.ts`, `services/news.ts`
LLM	**Google Gemini 1.5 Flash REST API — optional, called from the browser** with a user-supplied key; plus a server-side clip router that also calls Gemini if `GEMINI_API_KEY` is set	`lib/gemini-client.ts`, `routes/clip-router.ts`
Analytics/ads	Google Analytics 4 + AdSense via `VITE_*` env vars, consent banner	`.env.example`, `components/consent-banner.tsx`

Present but not used by the product: `pg` (PostgreSQL driver; `lib/db` replaces the Pool with a throwing Proxy — evidence of a Postgres→SQLite migration), `better-sqlite3` (declared, `node:sqlite` is used instead), `puppeteer`/`cheerio` (dev-only scraping), `@workspace/api-client-react` (generated but not imported), `mockup-sandbox`.

Not present (do not claim): Next.js, FastAPI, DuckDB, Pandas, PostgreSQL in production, Docker, CI/CD pipelines (no `.github/workflows`), authentication, Redis, any ML training framework, any vector database or RAG.

---

4. Major implemented features and their status

Feature	Route(s)	Data source	Status
Homepage	`/home`	static JSON (`/data/*.json`) + `/api/ipl/news` + live scores	✅ data-driven, with editorial content
Advanced Stats (filterable batting/bowling tables, matchups, scenarios, drilldowns)	`/stats`	`/api/stats/query`, `/drilldown`, `/matchup`, `/scenario` → `stats-aggregator.ts` (SQL path + JSON-replay path)	✅ works; 🔴 legacy endpoint takes 3–5 s warm (measured in Step 3 docs)
Player profiles	`/players/:slug`	static JSON + per-player shards + `/api/intel/player/:key`	✅ (fixed in Step 3: previously crashed by loading a 33 MB file)
Team / Venue profiles	`/teams/:slug`, `/venues/:slug`	static JSON + `/api/ipl/team-analytics`, `/api/intel/*`	✅ ❔ not individually re-tested
Match Detail	`/match/:id`	`/api/ipl/match/:id/scorecard`, `/commentary`, `/api/intel/match/:id`	✅; 🟠 `OversView` with random numbers exists but is **not rendered** (dead code)
My Dugout (franchise HQ)	`/dugout`, `/dugout/:team`	`/api/dugout/franchise`, `/api/dugout/squad` + photo registry	✅
Scout Lab (global talent)	`/scout`, `/scout/player/:id`, `/scout/compare`, `/scout/roles`	`/api/scout/*` → `global.db`	🟡 **uncommitted**; API typecheck fails on `scout.ts:123` (`unique_name` not on `Rating`)
AI Analyst hub + 11 modules (Ask, Weakness, Pressure, Insight Feed, Strategy Sim, Alt Universe, Opposition Map, Decision Replay, Squad DNA, Auction War Room, Player Pool)	`/ai-analyst/*`	`/api/ask`, `/api/weakness`, `/api/pressure`, `/api/war-room/*`, … → `services/decision/*`	✅ deterministic statistics/simulation; 🟠 snapshot "demo" fallbacks shown when API is unreachable
"Gemini LLM" tab on Ask page	`/ai-analyst/ask?tab=gemini`	browser → Gemini API with **user's own key**	🟠 **without a key it returns hardcoded text containing invented statistics, labelled "gemini-1.5-flash (Calibrated Quant Engine)"** (`lib/gemini-client.ts:105-140`)
Auction simulator (game)	`/auction`	`/api/auction/*` in-memory state machine, random AI bidders	✅ game logic; not analytics
Scores / Pulse (live)	`/scores`, `/pulse`	Cricbuzz scraping (`services/cricbuzz.ts`)	❔ depends on third-party site; 🟠 Pulse "emotion" bars include `Math.random()` (`pulse.tsx:137,168`)
Fan Arena / Beyond XI	`/arena`	mostly static content	🟠 largely presentational
Shop	`/shop`	static	🟠 presentational, no commerce backend
Intelligence hub, Feed ("2027 Intel"), Insights, Visualizations, Portal	various	static JSON + content modules (`content-engine-100.ts`, `strategy-2027-data.ts`)	🟡 mix of computed and editorial content
Photo pipeline dashboard	`/photo-pipeline`, `/admin/photo-pipeline`	registry JSON	🟡 internal tool, **publicly routed, no auth**
Global data engine (CLI)	—	Cricsheet → `global.db`	✅ 23/23 tests pass

---

5. Known defects and incomplete work (verified)

#	Defect	Evidence	Severity
D1	**Step 3 regression suite fails 4 of 15 tests** today (economy baseline drift after the Step 3.5 `sqlite.db` change on 2026-10-11; 20 player pages now "MISSING (empty page)")	`npx tsx --test src/step3/tests/identity.test.ts src/step3/tests/step3-regression.test.ts` → pass 11, fail 4	🔴 release blocker until explained
D2	API typecheck fails: `routes/scout.ts(123)` uses `unique_name` not present on `Rating`	`tsc --noEmit` in `artifacts/api-server` → exit 2	🔴
D3	Web typecheck: 11 errors, all in `components/cricket3d/*`	`tsc --noEmit` in `artifacts/ipl-analytics` → exit 2	🟡 pre-existing
D4	Ingest script writes **random** `delivery_type` / `scoring_area` (`scripts/src/update-ipl-data-v3.ts:678-681`). The **current** DB values were later overwritten from ESPN commentary (`scratch/backfill-delivery-columns.mjs`; values now e.g. "Short of a length", 95,511 NULLs). Re-running the ingest would silently re-introduce random data.	code + `GROUP BY delivery_type` on `sqlite.db`	🔴 latent data-integrity risk
D5	`deliveries.pressure_state` is `'Standard'` on all 295,732 rows, yet the SQL filter tests for `'Clutch'/'High'/'Playoff'` (`stats-aggregator.ts:1760-1768`). The code routes pressure filters to the JSON path, so the SQL branch is dead.	query + code	🟡
D6	Economy residual: 13 bowlers on 17 deliveries (net −30 runs) cannot be exactly reconciled	`docs/global-data-engine/step3/economy-residuals.md`	🟡 documented, not fully resolved
D7	Gemini fallback fabricates statistics and labels them as a model	`lib/gemini-client.ts`	🔴 trust / honesty risk
D8	Single 4.1 MB JS bundle, no route-level code splitting (`React.lazy` count = 0)	measured	🟡 performance
D9	`CORS` fully open (`app.use(cors())`), no auth, no rate limiting, admin photo dashboard public	`app.ts`	🟡 security

#	Defect	Evidence	Severity
D10	`global.db` is gitignored and the API server expects it at runtime; a clean deploy has no Scout Lab data unless the warehouse is built or shipped	`.gitignore:61-63`, `services/scout/db.ts`	🔴 deployment gap
D11	`artifacts/ipl-analytics/.env` is **tracked in git**. It holds only public `VITE_` identifiers (AdSense client, GA4 id, port), not secrets — but the pattern is risky.	`git ls-files`	🟡
D12	Live scores depend on scraping cricbuzz.com with a spoofed browser User-Agent	`services/cricbuzz.ts:3-9,81`	🔴 legal / reliability risk
D13	Stale earlier docs at repo root contradict current code (e.g., they predate Scout Lab and the decision services)	root `*.md` dated June–July	🟡
D14	`players.country` / `players.role` largely empty or wrong; `dismissals.fielder_id` all NULL; `matches.match_type` = "T20" everywhere; `seasons` champions NULL	project memory + queries	🟡 known, worked around in code

6. Tests and checks run during this audit

Command (run from the folder shown)	Result
`scripts/`: `npx tsx --test src/global/tests/*.test.ts src/scout/tests/*.test.ts`	✅ **23 / 23 pass**
`scripts/`: delivery-sequence test, run against a **copy** of `sqlite.db`	✅ 3 / 3 pass
`scripts/`: `npx tsx --test src/step3/tests/identity.test.ts src/step3/tests/step3-regression.test.ts`	🔴 **11 pass / 4 fail**
`artifacts/api-server`: `npx tsc -p tsconfig.json --noEmit`	🔴 2 errors
`artifacts/ipl-analytics`: `npx tsc -p tsconfig.json --noEmit`	🔴 11 errors (cricket3d)
`scripts`: `npx tsc -p tsconfig.json --noEmit`	✅ 0 errors
Frontend unit / component / E2E tests	**None exist**
API contract tests	Only `scripts/src/scout/tests/api.test.ts` (checks the service, not HTTP)

`sqlite.db` modification time was checked before and after: unchanged (`Oct 11 00:15`).

7. Estimated handbook scope and execution order

The handbook has 30 chapters plus 6 companion files. Because the most error-prone material is the data and statistics, the order is **data → statistics → API → architecture → UI → career**:

1. 01 Repository audit (this file)
2. 04 Database engineering, 05 Data pipelines
3. 06 Scouting & statistics
4. 08 API engineering
5. 02 System architecture + `ARCHITECTURE-DIAGRAMS.md`, 03 Technology stack
6. 07 Frontend, 09 AI audit, 10 Testing, 11 Performance & security
7. 12 Feature walkthroughs
8. 20–23 UI, UX, product case studies, data-visualisation
9. 24–27 Product management, business, solo-builder, portfolio readiness
10. 13, 14, 28 Skills and role matrices; 15 & 29 interview banks; 16 challenges; 17 roadmap; 18 resume evidence; 19 glossary
11. `00-EXECUTIVE-OVERVIEW.md`, `AUDIT-SUMMARY.md`, `README.md`, CSV files, PDF

02 — System Architecture

> Diagrams for every section live in ARCHITECTURE-DIAGRAMS.md. This chapter explains the *decisions* behind them and traces one real request end to end.

---

1. Beginner explanation

CricField is a **website** (React, runs in your browser) that talks to a **server** (Express, runs on Node.js). The server reads cricket data from two **database files** (SQLite) and some **JSON files**, does the maths, and sends back JSON. The website turns that JSON into tables and charts. Some pages skip the server entirely and download pre-computed JSON files directly.

---

2. Architectural style

Property	What CricField does	Consequence
Monolith	One Node process serves all 123 endpoints, runs simulations, scrapes live sites and (in production) serves the built SPA.	Simple to run and deploy; one slow query slows everyone.
Modular service layer	`routes/*` are thin; logic lives in `services/*`; decision modules share a data layer (`services/decision/db.ts`, `player-profiles.ts`, `ball-context.ts`).	Good separation — the newer code is easy to test in isolation (this audit ran `ratePlayers` and `auctionBoard` directly).
Embedded database	SQLite via Node's built-in synchronous driver.	No DB server; but every query blocks the event loop.
Read-mostly	Only two code paths write at runtime (insight click counter; and the unsafe CSV reader).	Easy to cache; could be served from a CDN if responses were cacheable.
Pre-compute where possible	Static JSON (`public/data`), start-up caches (column store, impact model, auction board), aggregates tables.	Fast reads (2–14 ms) after an 11.9 s warm-up.
Two generations of code	*Legacy* (`stats-aggregator.ts`, `ipl.ts`: no validation, JSON fallbacks, hardcoded literals) and *modern* (decision + scout: Zod, typed services, documented maths).	Inconsistent quality; the legacy path is also the slowest.
Client-side LLM	Gemini is called from the browser with a user-supplied key.	No server cost, but no control, no grounding, key exposure.

---

3. Layers

3.1 Front end (`artifacts/ipl-analytics`)

- React 19 + Vite 7, routing with **wouter** (a 2 KB router; `<Switch>`/`<Route>` in `App.tsx`).
- Server state with **TanStack React Query** (caching, loading/error states, `staleTime` per query).
- Client state: React context for the chosen franchise (`lib/team-context.tsx`, also writes CSS variables `--team-accent` etc. so the whole UI re-themes per team), zustand for Advanced Stats filters (`lib/store.ts`).
- Styling: Tailwind CSS 4 utility classes + shadcn-style components on Radix primitives (`components/ui/*`).
- Visuals: Recharts, hand-built SVG, three.js 3D stadium replay, framer-motion.

3.2 Back end (`artifacts/api-server`)

- Express 5; middleware: pino-http, cors, json, no-store headers.
- Route groups: legacy IPL (`ipl.ts`), live (`live.ts`), auction game (`auction.ts`), decision Phase 1–2 (`decision.ts`), decision Phase 3–4 + intel + dugout (`decision-intel.ts`), Scout Lab (`scout.ts`), clip router, CSV sync, health.
- Build: esbuild bundles everything (except native modules) into one ESM file with a banner that recreates `__dirname`/`require`.

3.3 Data

- `sqlite.db` (IPL), `global.db` (6 leagues), `lib/cricket-data/raw/*.json` (raw match files, used for stage, scorecards, DLS targets and the JSON-replay stats engine), `artifacts/api-server/data/*.json` and `artifacts/ipl-analytics/public/data/*.json` (curated/derived files).

3.4 Analytics

- Deterministic statistics everywhere (Chapter 6). Two small fitted components: Platt calibration and kNN pricing.

3.5 Assets and media

- ≈540 images in `public/` (player photos, stadiums, logos), ~60 more at the repo root, a photo registry (`public/data/player-photo-registry.json`) with a validation script and an internal dashboard (`/photo-pipeline`). Photos were scraped from official sites (licensing risk, Chapter 25).

3.6 External integrations

Cricbuzz (scraped), Google News & ESPN RSS, Gemini (optional), GA4 + AdSense (env-gated, consent banner).

---

4. Request trace: “show me the best batters across six leagues”

User action: opens `/scout`, leaves *All 6 Leagues*, *All Phases*, *Rankable (120+)*, sort *Shrunk Strike Rate*.

Step	Where	What happens
1	`App.tsx`	wouter matches `/scout` → `ScoutHomePage`.
2	`pages/scout-home.tsx:88-118`	React state holds filters; `useQuery` builds key `["scout-rankings", discipline, comps, phase, role, minBalls, page, limit, sortBy, sortOrder, search]`, `staleTime` 5 min.
3	`lib/scout-api.ts:240`	`fetchScoutRankings()` builds `URLSearchParams` → `GET /api/scout/rankings?discipline=batting&minBalls=120&page=1&limit=25&sortBy=shrunk&sortOrder=desc`.
4	Vite dev proxy (`vite.config.ts server.proxy`)	forwards `/api` to `localhost:${VITE_BACKEND_PORT}` (in production the API serves the SPA itself, same origin).
5	`app.ts`	pino logs, CORS, `no-store`.
6	`routes/scout.ts:78-104`	`requireGlobalWarehouse()`; Zod `rankingsQuerySchema` coerces numbers, defaults, enums.
7	`scout-service.ts:487`	`ratePlayers()` → for each of 6 competitions `competitionRatings()` runs ~6 SQL statements (aggregates, variance, coverage, roles, names, teams).
8	`scout-service.ts:395-482`	computes index, shrinkage, intervals, percentiles per competition.
9	`routes/scout.ts:117-187`	text search, sort by `metrics.strike_rate.shrunk` desc, paginate 25, attach strength/weakness sentences. **No `minBalls` filter here.**
10	Response	≈36 KB JSON, measured 35–58 ms.
11	`scout-home.tsx`	renders the leaderboard: `PlayerScoutRow`, `RolePill`, CI text, tier badges. `RolePill` receives an object → shows “Unclassified” (defect).
12	React Query	caches for 5 min; changing any filter creates a new key → new request; going back to an old filter is instant.

What breaks if a piece is removed: no `global.db` → 503 `global_warehouse_unavailable` and the page shows its error state; no Zod → `minBalls=abc` would reach SQL as `NaN`; no React Query key per filter → stale tables after changing filters.

---

5. System boundaries and trust

Boundary	Today	Risk
Browser ↔ API	public, no auth, open CORS	abuse of heavy endpoints, shared auction state, file-read route
API ↔ SQLite	same process, file access	a single process; multi-instance autoscale gives each instance its own DB copy
API ↔ third parties	server-side scraping with spoofed UA	ToS / legal and reliability
Browser ↔ Gemini	direct, user key	key exposure, ungrounded output
Repo ↔ data	warehouses and raw data partly gitignored	clean deploys lack data

---

6. Deployment architecture (as configured)

- **Replit** (`.replit`): `deploymentTarget = "autoscale"`, ports 8080→80 etc., post-merge hook `pnpm install && drizzle-kit push`. Commit messages show three “Published your App” events in May 2026.
- **Production process:** `node dist/index.mjs` serves `/api/*` and, if `ipl-analytics/dist/public` exists, the SPA with an index.html fallback (`app.ts:43-58`).
- **Local:** `START.bat`, `run.sh`, `run.ps1` start API (:5000) and Vite (:3000) together; Vite proxies `/api`.
- **Not present:** Vercel config, Docker, CI, IaC, environment-specific config files, secrets manager. Deploying anywhere else would require shipping `sqlite.db` (+WAL), the raw JSON folder and `global.db` with the build.

Known run-path defect: the documented source-run (`pnpm exec tsx src/index.ts`, used by `.claude/launch.json`) crashes with `ReferenceError: __dirname is not defined in ES module scope` at `app.ts:46` (uncommitted change). The esbuild bundle works because its banner defines `__dirname`.

---

7. Dependencies that matter

Dependency	Why it matters architecturally
`node:sqlite`	experimental API in Node (prints a warning); synchronous; no extra native build
Drizzle ORM (`sqlite-proxy`)	used only by the legacy path, wrapped around `node:sqlite`; leftovers from a Postgres setup (`pg` dependency, Pool proxy that throws)
Zod	the contract layer for modern routes
javascript-lp-solver	MILP for the Playing XI
React Query	the whole front end’s data layer
three.js	~1/3 of the custom visual code; its components are where the 11 web type errors are

---

8. Architecture interview questions

1. Why a monolith? *(One developer, read-mostly data, simplest deploy; split when a component needs independent scaling — e.g., move simulations to a worker.)*

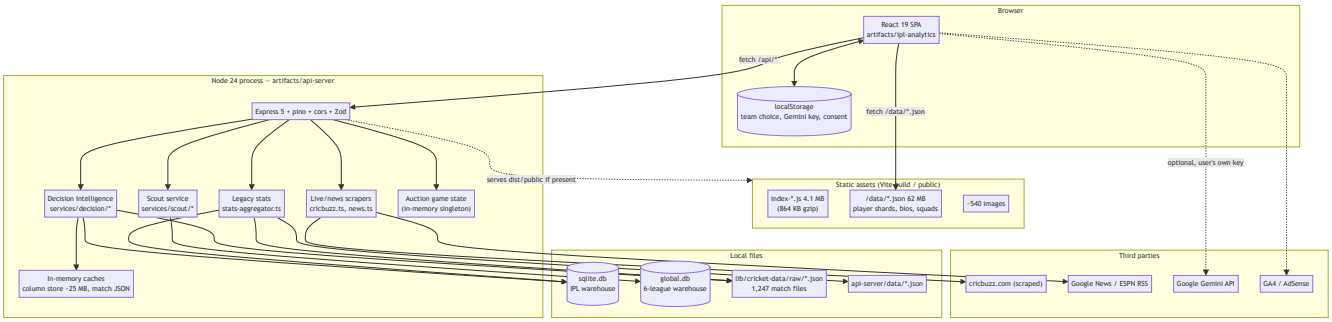
2. What is the single biggest architectural bottleneck? *(Synchronous SQLite on the event loop + no response caching.)*
3. Why are there three ways of computing a batting average? *(History: JSON replay, SQL path, decision column store — consolidation is overdue.)*
4. How would you make Scout Lab safe to host for 10,000 users/day? *(Chapter 11.)*
5. Where are the trust boundaries, and which one is currently broken? *(`/api/ipl/sync-csv`.)*

Architecture Diagrams

All ten diagrams are Mermaid (render on GitHub, VS Code with a Mermaid extension, or mermaid.live). Each is followed by the files it describes, what moves across it, and where its boundaries are. Diagrams describe the code **as it is on 2026-10-12**, including the uncommitted Scout Lab work.

---

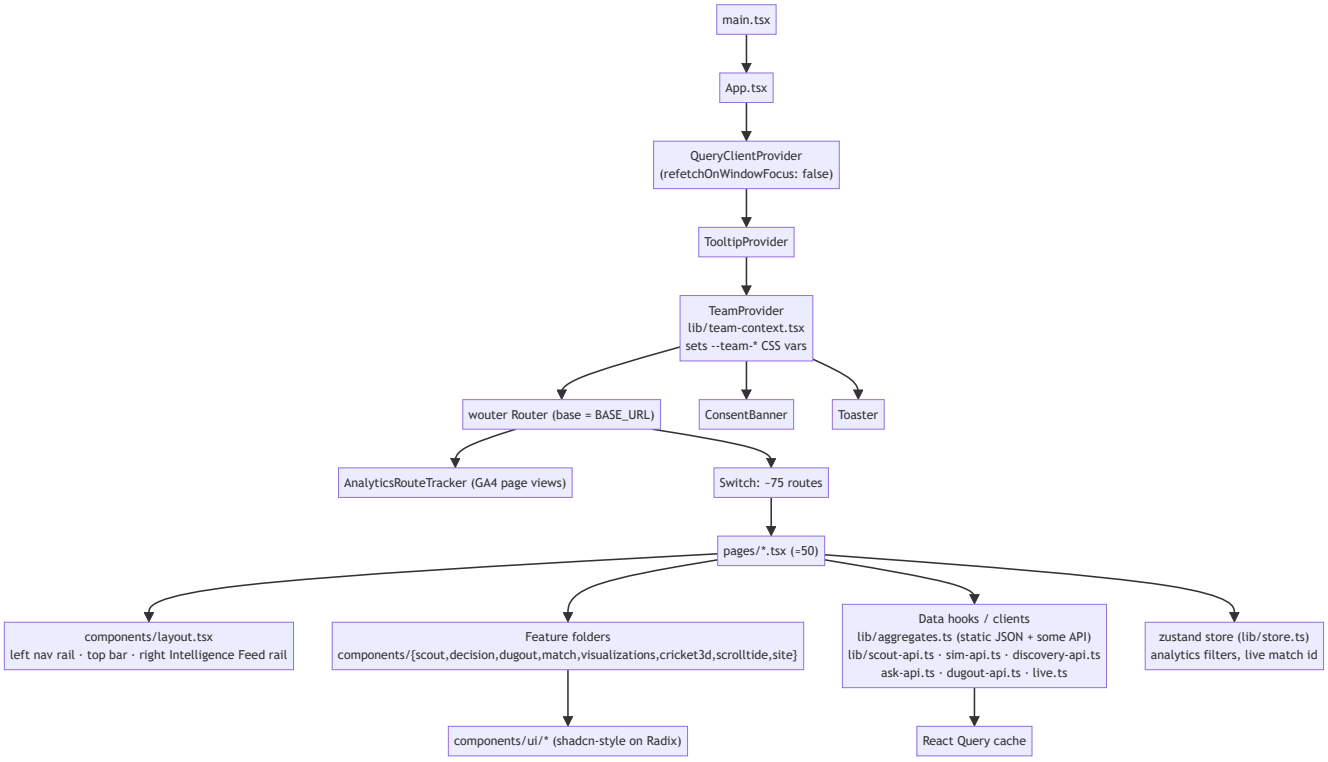
1. High-level system architecture



Files: `artifacts/ipl-analytics/src/App.tsx`, `artifacts/api-server/src/{index,app}.ts`, `routes/index.ts`. **Movement:** the browser pulls both *static JSON* (precomputed at build time) and *API JSON* (computed per request). Only the API touches SQLite. **Boundaries:** one Node process does everything (HTTP, SQL, simulation, scraping). No queue, no worker, no separate DB server, no auth layer. The Gemini call is made **from the browser**, outside the server boundary.

---

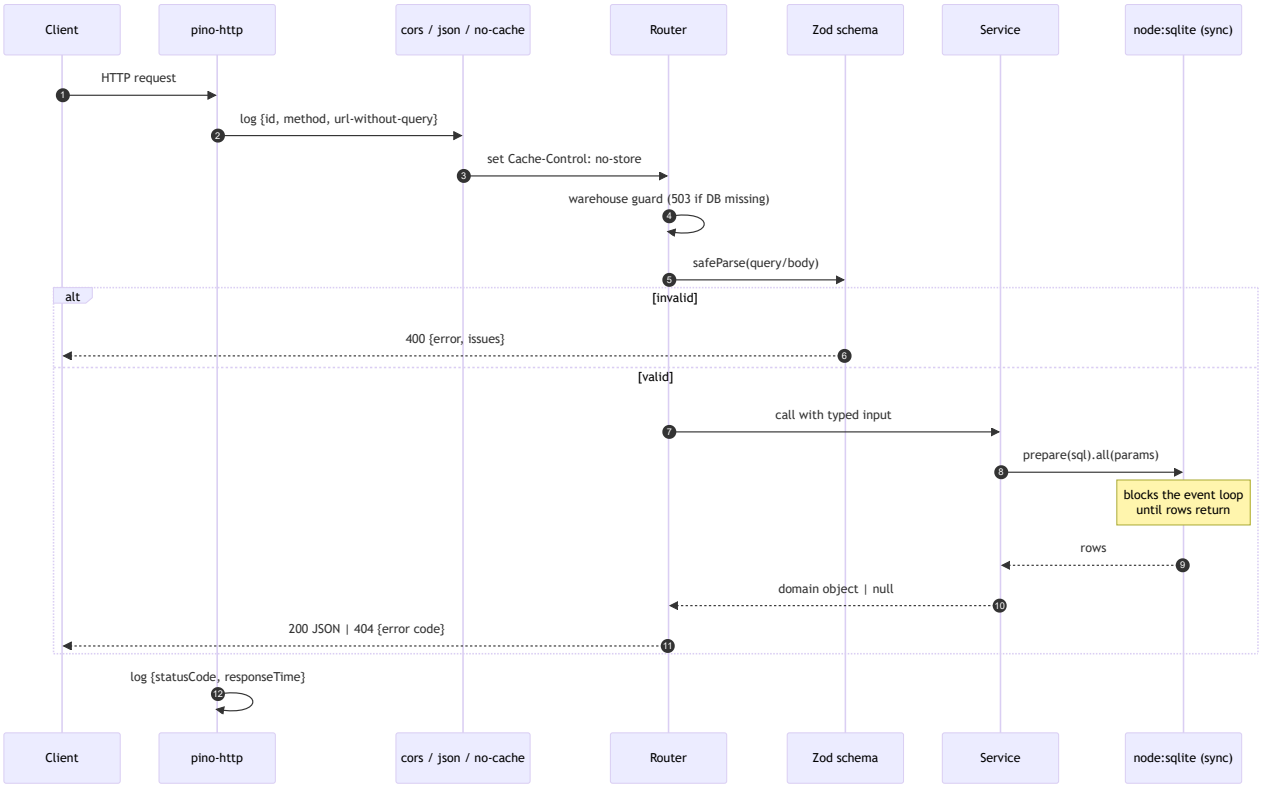
2. Front-end component architecture



Files: `App.tsx`, `components/layout.tsx`, `lib/*.ts`. **Data movement:** pages call hooks; hooks call `fetch` and cache in React Query (keys like `["agg","top-batters"]`, `staleTime` 1 h for static JSON). Global UI state (chosen franchise) lives in React context + localStorage; filter state on Advanced Stats in zustand. **Boundaries:** there is **no code splitting** — every page is imported statically into `App.tsx`, so the whole site ships in one 4.1 MB bundle.

---

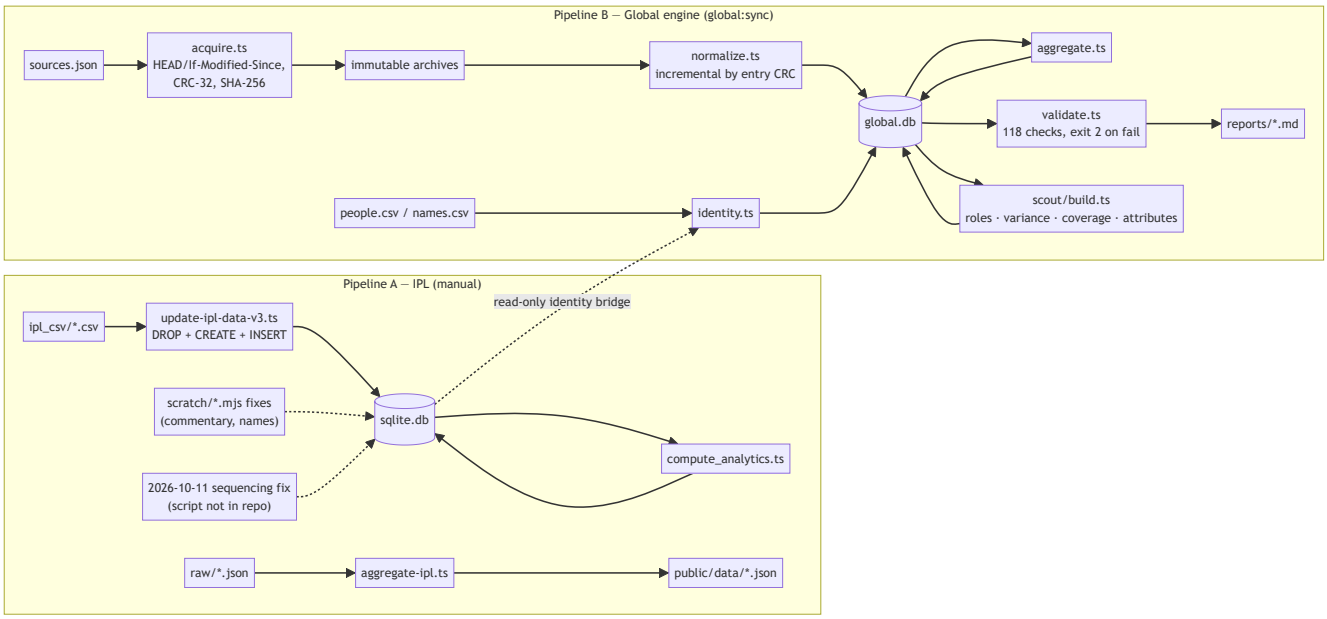
3. Back-end request lifecycle



Files: `src/app.ts`, `routes/scout.ts` (exemplary), `routes/ipl.ts` (no Zod; `500 {error: err.message}`). **Boundary to remember:** because the SQLite driver is synchronous, "concurrent" requests are processed one after another; measured 10 parallel Advanced Stats queries → 24 s.

---

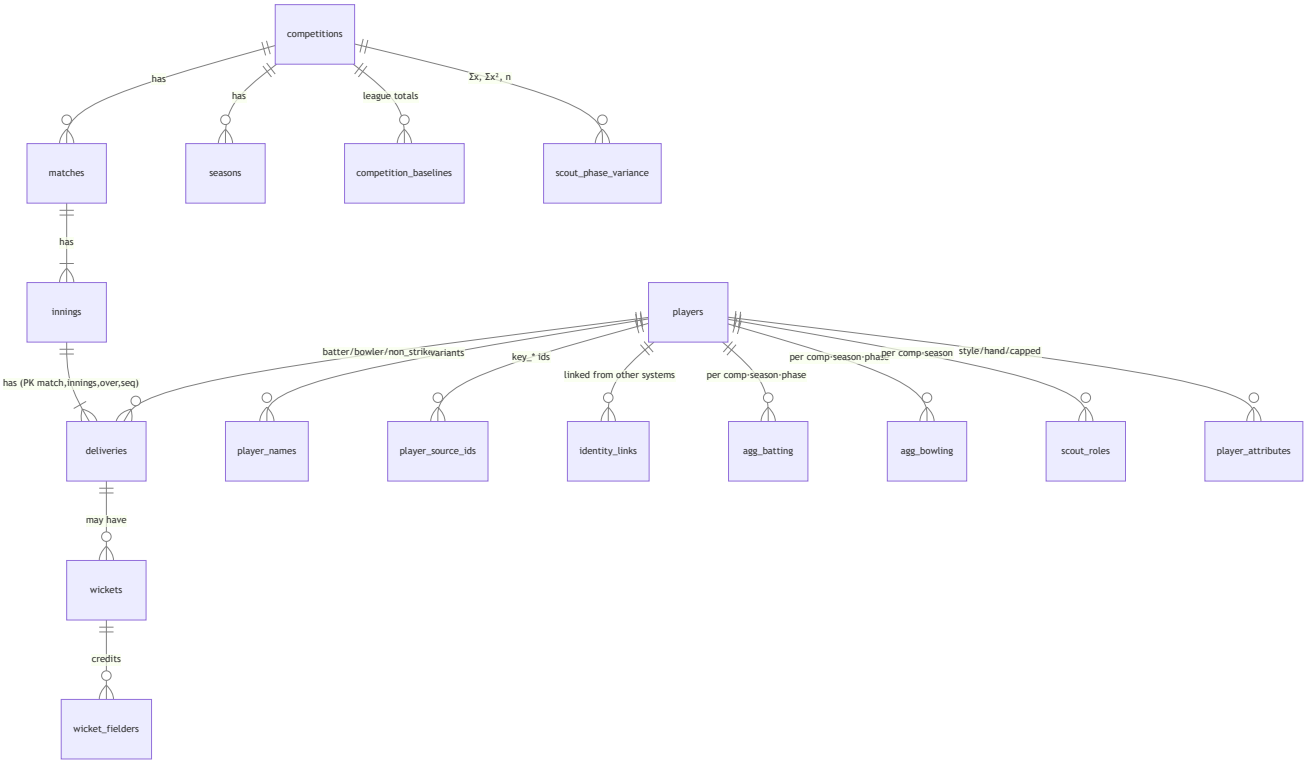
4. Data ingestion and ETL pipeline



Files: `scripts/src/update-ipl-data-v3.ts`, `compute_analytics.ts`, `aggregate-ipl.ts`, `scripts/src/global/*.ts`, `scripts/src/scout/*.ts`. **Boundary:** Pipeline B never writes to `sqlite.db`. Pipeline A is not reproducible from the repo (Chapter 5 §8).

---

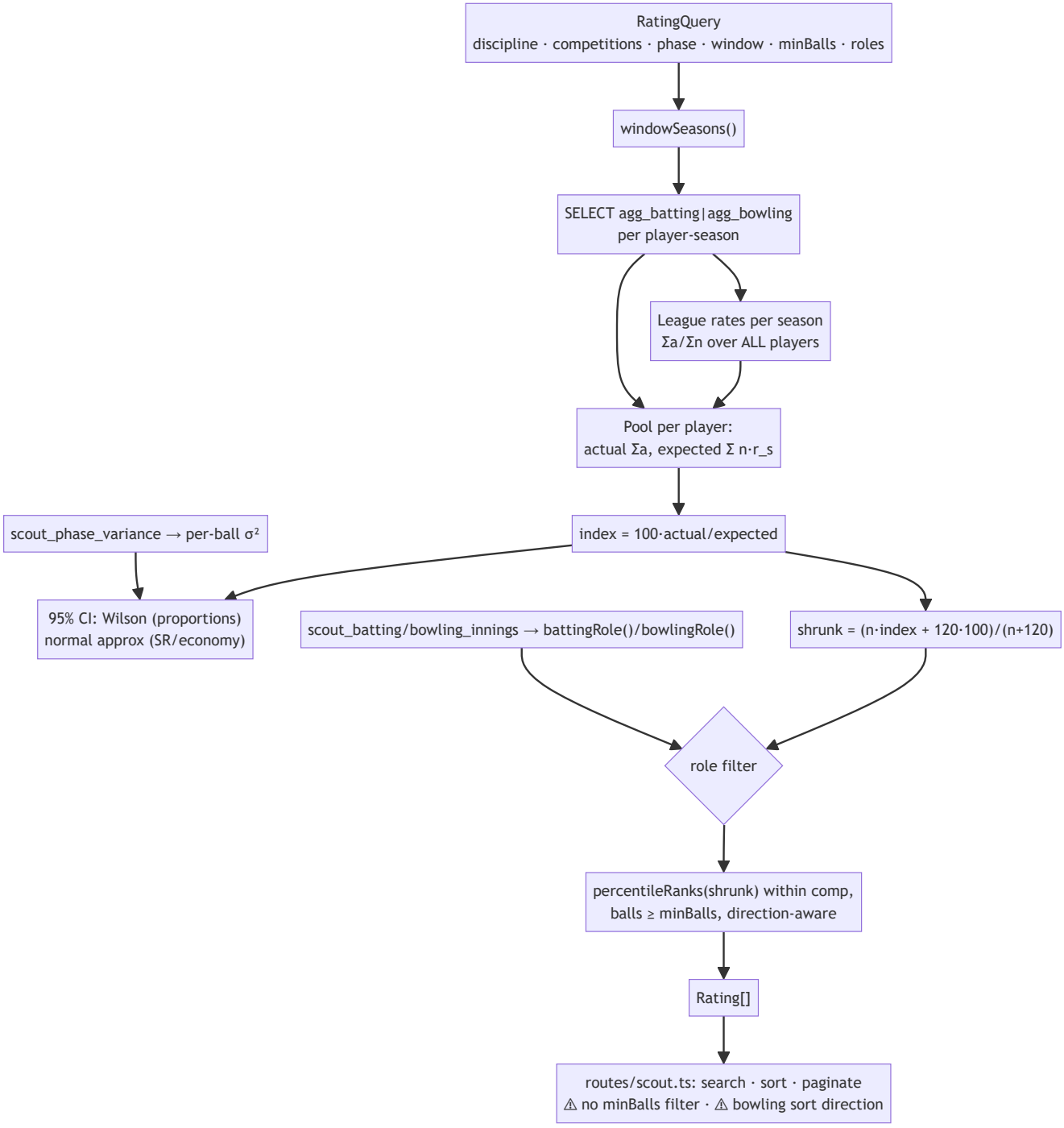
5. Database relationships



This is the **global** warehouse (the IPL warehouse ERD is in Chapter 4 §3.3). Every aggregate has the same grain as its league baseline, which is what makes `index = 100 × actual / expected` a simple join.

---

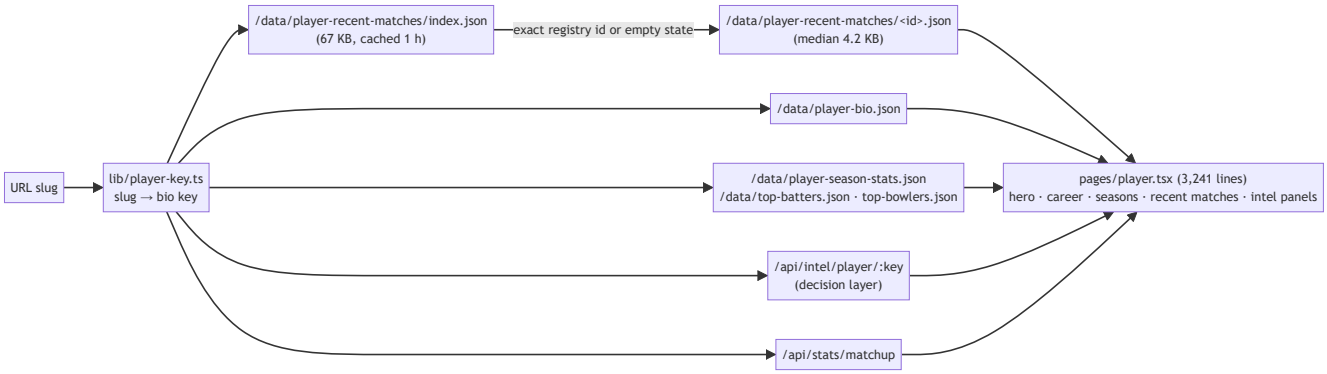
6. Scouting engine processing



Files: `services/scout/scout-service.ts:269-489`, `routes/scout.ts:78-188`.

---

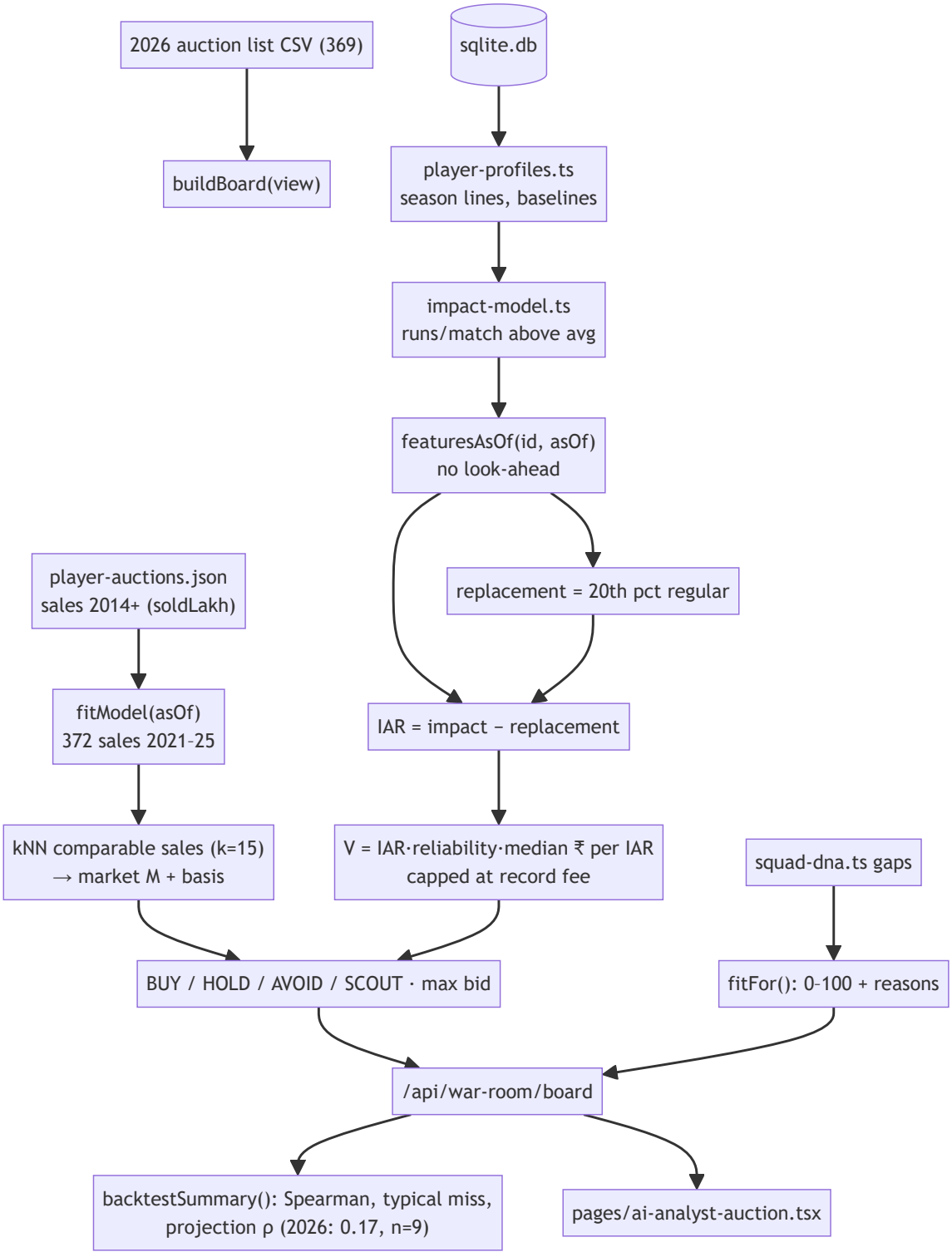
7. Player profile data flow (`/players/:slug`)



Why it matters: before Step 3 the page loaded one 33 MB JSON for every player and crashed; now it loads a 67 KB index + one shard. The identity rule is exact-only, so an unverified bio shows an empty state instead of another player's matches.

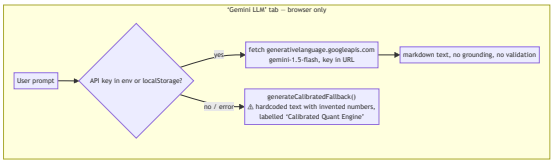
---

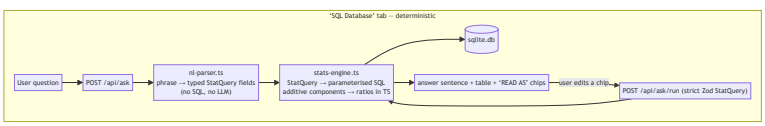
8. War Room data flow



---

9. AI Analyst request flow

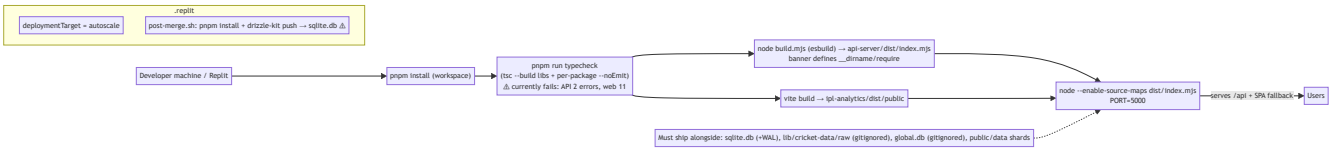




Files: `routes/decision.ts:138-171`, `services/decision/nl-parser.ts`, `stats-engine.ts`, `ipl-analytics/src/lib/gemini-client.ts`.

---

10. Deployment and build process



Facts: root `build` = typecheck then `pnpm -r build`, so a failing typecheck blocks a clean build. The API bundle was last built 2026-10-12 15:50. There is **no CI** (no `.github/workflows`), **no container**, **no Vercel config**. Autoscale hosting with a local SQLite file means each instance has its own copy; writes (click counter, auction state) are not shared between instances.

03 — Technology Stack Deep Dive

> **Rule for this chapter:** a technology is listed only if code that runs imports it. "Declared but unused" and "absent" technologies are listed separately in §9 so you never claim them by accident.
>
> Each entry: **What** · **Problem it solves** · **Where in CricField** · **Why here** · **Alternatives** · **Pros / limits** · **Skills to use it independently** · **Interview questions**.

---

1. Languages

TypeScript (~5.9) — ✅ verified

- **What:** JavaScript with static types, compiled away at build time.
- **Problem:** catches shape mismatches (e.g., passing an object where a string is expected) before runtime.
- **Where:** all of `artifacts/*/src`, `lib/*`, `scripts/src`. `tsconfig.base.json` (strict settings), `pnpm run typecheck`.
- **Why here:** one language for browser, server and ETL; types shared via `lib/api-zod`.
- **Alternatives:** plain JS (faster to write, more runtime bugs), Python for ETL.
- **Limits observed:** types only help if they reflect reality. The Scout client declares `role: string` (`lib/scout-api.ts:23`) while the API sends `{label, inputs}` → the bug compiled cleanly. Also, esbuild/Vite do **not** type-check, so the current type errors (API 2, web 11) do not stop dev builds.
- **Skills:** generics, union/discriminated union types (`altSchema` in `decision-intel.ts:163-171`), `unknown` vs `any` (the codebase uses `any` heavily in `stats-aggregator.ts`), `tsc --noEmit`.
- **Interview:** *"How does `z.infer` connect runtime validation to compile-time types?" "Why didn't TypeScript catch the role bug?"*

JavaScript (ES modules) — ✅

`.mjs` scripts in `scripts/` (photo scraping, registry generation, `generate-player-recent-matches.mjs`). `"type": "module"` in every package.

Python 3 — 🟡 tooling only

- **Where:** `artifacts/image-ingestion/` (requests, BeautifulSoup, Pillow, FastAPI, pandas — an image-collection tool with its own small API) and ~30 scripts using **OpenCV** (`cv2.grabCut`) + NumPy + Pillow to cut out and crop player photos (`scripts/test_perfect_cutouts.py`, `crop_rcb_2025_to_chest.py`).
- **Not used for:** the cricket data pipeline or analytics. **Do not claim "Python/Pandas data pipeline" for the cricket warehouse.**

SQL (SQLite dialect) — ✅

Hand-written SQL in `services/decision/*`, `services/scout/*`, `scripts/src/global/*`; Drizzle-generated SQL in the legacy path. Features used: CTEs (`stats-aggregator.ts` `overall_stats`, `match_scores`), `CASE` aggregation, `GROUP BY … HAVING`, window-free ranking done in TS, `WITHOUT ROWID` tables, composite primary keys, views (`v_batting`, …), `EXPLAIN QUERY PLAN` (used in this audit).

---

2. Front end

React 19.1 — ✅

- **What:** UI library; components re-render from state.
- **Where:** every page in `artifacts/ipl-analytics/src/pages/*`.
- **Why:** largest ecosystem for dashboards; pairs with React Query and Radix.
- **Alternatives:** Svelte, Vue, server-rendered Next.js.
- **Limits observed:** very large components (`stats.tsx` 4,411 lines, `player.tsx` 3,241) mix data shaping, layout and visuals — hard to test or review; 29 `useMemo` calls in `player.tsx` alone.
- **Skills:** props/state, effects, memoisation, controlled inputs, lists with keys, error boundaries (none exist in the app).
- **Interview:** *"How does a React component retrieve data in CricField?"* → via a React Query hook; walk through `useQuery` in `scout-home.tsx:89-118`.

Vite 7 — ✅

Dev server with hot reload and a `/api` proxy (`vite.config.ts`), production build to `dist/public`. Alternatives: webpack, Next.js. Limit observed: no manual chunks / lazy routes → one 4.1 MB bundle.

wouter 3 — ✅

Tiny router: `<Switch>`, `<Route path="/players/:slug">`, `<Redirect>`, `useRoute`, `useLocation`. Alternatives: React Router (bigger, more features: loaders, nested routes). Limit: no built-in code splitting.

TanStack React Query 5 — ✅

- **Problem:** fetching, caching, deduplicating, retrying and exposing `isLoading/isError` for server data.
- **Where:** `lib/aggregates.ts` (≈20 hooks over static JSON, `staleTime` 1 h), `scout-home.tsx`, decision pages.
- **Pattern to know:** the **query key** contains every filter, so each filter combination is a separate cache entry.
- **Interview:** *"What is `staleTime` vs `gcTime`?" "What happens when the API fails?"* → `isError` → page shows `ErrorBanner` or, for Phase 2 pages when the API is unreachable, a clearly labelled **DemoBanner** with a saved real snapshot (`ai-analyst-weakness.tsx:35-56`).

Tailwind CSS 4 + shadcn-style components on Radix UI — ✅

- **Where:** `index.css` (`@import "tailwindcss"`, HSL CSS variables), `components/ui/*` (~50 primitives: dialog, tabs, select, tooltip…).
- **Why:** fast iteration, consistent primitives, accessible behaviour from Radix (focus traps, ARIA on dialogs/menus).
- **Limits observed:** `bg-primary/text-primary` render nothing (Tailwind v4 needs an `@theme` mapping that is missing), so many pages hard-code hex colours (`bg-[#0B1020]`) and arbitrary sizes (`text-[7px]`, `text-[10px]`) — 69 text elements under 10 px on `/stats` at 375 px width.

framer-motion 12 — ✅

Animations (layout rail width, hero transitions). Known quirk: animations stall when the tab is hidden, distorting measurements (project memory note).

three.js 0.170 — ✅

`components/cricket3d/*` (stadium, ball trajectory, crowd, audio) driven by `lib/cricket3d/deliveryAdapter.ts`. All 11 web type errors are here.

Recharts 2.15 + custom SVG + d3-scale-chromatic — ✅

Charts in decision modules (`components/decision/*`), visualisations (`components/visualizations/*`: sankey, knowledge graph, spatial analytics).

zustand 5 — ✅

Small global store for Advanced Stats filters and live-match id (`lib/store.ts`).

Others imported by the web app

`@tanstack/react-table` (data tables), `papaparse`/`react-csv` (CSV export), `lucide-react`/`react-icons`, `sonner` (toasts), `embla-carousel`, `cmdk`.

---

3. Back end

Node.js 24 — ✅

Runtime for API and ETL. Uses built-in `node:sqlite`, `node:test`, `fetch`. Single-threaded event loop — the reason synchronous SQL serialises requests.

Express 5 — ✅

- **Where:** `artifacts/api-server/src/app.ts`, `routes/*.ts`.
- **Why:** minimal, ubiquitous. Express 5 forwards rejected promises to error handlers.
- **Alternatives:** Fastify (faster, schema-first), Hono, NestJS (structure), FastAPI (Python).
- **Limit:** no global error-handling middleware is registered; each route has its own try/catch with inconsistent responses.
- **Interview:** *"What is middleware? Name the four in `app.ts` in order."*

Zod 3 — ✅

Runtime validation with type inference. Exemplary schemas: `rankingsQuerySchema` (`routes/scout.ts:64-76`), `statQuerySchema` (`routes/decision.ts:102-127`, `.strict()` rejects unknown keys), `strategySchema`. Uses `z.coerce.number()` to turn query strings into numbers. Not used by legacy `ipl.ts`.

pino / pino-http — ✅

Structured JSON logs; serializers strip query strings from logged URLs. No log shipping or monitoring.

javascript-lp-solver 0.4.24 — ✅

Solves the Playing XI mixed-integer linear programme (`services/decision/playing-xi.ts`). Alternatives: GLPK, OR-Tools, PuLP. Limit: small problems only; fine for 25-player squads.

cors — ✅ (open to all origins)

---

4. Data layer

SQLite via `node:sqlite` (`DatabaseSync`) — ✅

- **What:** embedded relational database; one file; the driver runs in-process.
- **Why here:** no server to run on Replit; ~300k–770k rows fit easily; read-mostly workload; easy to copy.
- **Alternatives:** PostgreSQL (concurrency, many writers, async drivers), DuckDB (columnar analytics, much faster scans), better-sqlite3 (mature sync driver; declared but not used).

- **Limits observed:** synchronous → blocks the event loop; WAL file must travel with the DB; `node:sqlite` is flagged experimental.
- **Interview:** *"Why did you choose SQLite?"* (one dev, embedded, read-mostly, fast enough at this scale; I know the ceiling is concurrency).

Drizzle ORM 0.45 (sqlite-proxy) — 🟡

Schema in `lib/db/src/schema/index.ts`; queries via `db.execute(sql\`...\`)` in the legacy aggregator. drizzle-kit push `runs on every Replit merge` (scripts/post-merge.sh`) — risky against a hand-patched warehouse.

fflate — ✅ (ETL only)

Reads zip archives in memory for the global engine (`scripts/src/global/zip.ts`).

---

5. Build, tooling, testing

Tool	Role	Evidence
pnpm workspaces + catalog	monorepo dependency management; shared versions in `pnpm-workspace.yaml` catalog; `minimumReleaseAge: 1440` (refuses packages < 1 day old — a supply-chain safeguard)	`pnpm-workspace.yaml`
esbuild	bundles the API to one ESM file, externalises native modules, adds `__dirname` banner	`api-server/build.mjs`
tsx	runs TypeScript directly for scripts and tests	`scripts/package.json`
node:test + `node:assert/strict`	test runner (no Jest/Vitest)	`scripts/src/**/tests/*.test.ts`
OpenAPI 3.1 + Orval	generates Zod and React Query client from `openapi.yaml`	`lib/api-spec/`, only 19 paths; client unused
Prettier	declared at root	no config enforcement seen
Git	version control	19 commits, much uncommitted

---

6. AI / LLM

Item	Status
Google Gemini 1.5 Flash REST (browser)	🟡 optional, user key, no grounding (`lib/gemini-client.ts`)
Gemini in clip router (server)	🟡 optional via `GEMINI_API_KEY`, keyword fallback (`routes/clip-router.ts:126-186`)
Natural-language → typed query	✅ deterministic phrase parser, **not** an LLM (`nl-parser.ts`)
ML training / models	❌ none

Details: Chapter 9.

---

7. DevOps and hosting

Item	Status
Replit (autoscale)	✅ configured (`.replit`)
Vercel	❌ no config in repo
CI/CD	❌ none
Containers	❌ none
Environment variables	`PORT`, `BASE_PATH`, `VITE_BACKEND_PORT`, `VITE_GA_MEASUREMENT_ID`, `VITE_ADSENSE_CLIENT`, `VITE_GEMINI_API_KEY`, `GEMINI_API_KEY`, `REPL_ID`, `NODE_ENV`
Monitoring / error tracking	❌ none (pino to stdout only)

---

8. Security-relevant technology

Concern	Technology used	Gap
Input validation	Zod (modern routes)	legacy routes unvalidated
SQL injection	prepared statements / Drizzle parameters	one interpolated, enum-guarded fragment
Secrets	`.env` (only public `VITE_` ids); `.env.example` warns that `VITE_` vars ship to the browser	`.env` is tracked in git; Gemini key handled in browser
AuthN/AuthZ	none	admin page and write endpoints public
Dependency hygiene	`minimumReleaseAge`, `onlyBuiltDependencies` allow-list	no `pnpm audit` in a pipeline

---

9. Declared-but-unused and absent technologies (do not claim)

Technology	Status
PostgreSQL / `pg`	declared in `lib/db`; replaced by a throwing Proxy — **not used**

Technology	Status
better-sqlite3	declared; **not used** (node:sqlite instead)
`@workspace/api-client-react`	generated; **not imported** by the site
puppeteer, cheerio	dev scraping only
mockup-sandbox	Replit design sandbox, not product
Next.js, FastAPI (for the product), DuckDB, Pandas (for analytics), Redis, Docker, Kubernetes, GitHub Actions, Jest/Vitest/Playwright, LangChain, vector DBs, RAG, fine-tuning, scikit-learn, PyTorch	**absent**

---

10. Skills needed to work on this stack independently

Foundational: JavaScript/TypeScript, HTML/CSS, SQL (joins, GROUP BY, CASE), HTTP/REST, Git, debugging with logs and devtools. Framework: React hooks, React Query, Express routing/middleware, Zod schemas, Tailwind utilities. Data: SQLite specifics (WAL, indexes, EXPLAIN QUERY PLAN), ETL design (idempotency, validation), cricket scoring rules. Statistics: rates vs counts, binomial proportions, Wilson interval, shrinkage, percentiles, multiple testing, calibration, backtesting. Ops: environment variables, build pipelines, bundling, performance measurement.

04 — Database Engineering

> **Read this chapter in three passes.** Beginner: §1–§2. Engineer: §3–§7. Interview: §8–§10. All counts were taken on 2026-10-12 by opening the databases **read-only** (`new DatabaseSync(path, { readOnly: true })`). Query timings were measured on a *copy* of `sqlite.db`, never the production file.

---

1. Beginner view: what a "warehouse" is here

CricField stores cricket in two **SQLite** files. SQLite is a full SQL database that lives inside one file and is read by a library *inside* the application process — there is no separate database server to run.

Database	File	What it holds	Who reads it
IPL warehouse	`sqlite.db` (repo root, 119 MB + 27 MB WAL)	1,243 IPL matches 2008–2026, 295,732 deliveries, plus precomputed player/team tables	The Express API (Advanced Stats, player pages, every AI Analyst module, Dugout, War Room)
Global warehouse	`lib/cricket-data/global/warehouse/global.db` (79 MB, gitignored)	3,309 matches across IPL, SMAT, BBL, SA20, CPL, ILT20; 771,965 deliveries; identity tables; scouting tables	The global ETL CLI and the Scout Lab API (`/api/scout/*`)

A **delivery** (one ball) is the atomic fact. Everything else — a batter's strike rate, a team's powerplay run rate, an auction valuation — is an aggregation of deliveries.

---

2. Why two databases instead of one

Reason	Evidence
Risk isolation. The global engine was built later and was required never to change the live IPL site. It opens `sqlite.db` only read-only.	`docs/global-data-engine/README.md` "Boundaries"; `scripts/src/global/identity.ts` reads the IPL DB read-only
Different schema philosophy. `sqlite.db` uses surrogate integer IDs and denormalised flags (`is_wide`, `phase`), built by a script that *drops and recreates every table*. `global.db` keeps Cricsheet's structure (0-based overs, per-type extras, `WITHOUT ROWID` composite keys) and is updated incrementally.	`scripts/src/update-ipl-data-v3.ts:19-45` vs `scripts/src/global/schema.sql`
Rebuildability. `global.db` is gitignored and rebuilt from hashed raw archives in ≈30 s. `sqlite.db` is tracked in git (119 MB binary) and has hand-applied fixes.	`.gitignore:61-63`; `docs/global-data-engine/README.md`

Trade-off you must be able to state in an interview: two warehouses mean the same IPL ball exists twice, and the two copies can disagree (they did — see §7 economy residuals and the `is_bye` gap). The engine compensates with a *cross-check* (`scripts/src/global/validate.ts:202-306` compares player-season totals between the warehouses).

---

3. IPL warehouse (`sqlite.db`) — schema

25 tables. Row counts are live. **Bold** = core fact/dimension tables used by running code.

Table	Rows	Purpose	Status
matches	1,243	One row per match: season, date, venue, teams, toss, winner	✅ `match_type` = "T20" for every row (it is the *format*, not the stage); playoff stage lives only in raw JSON
innings	2,514	Innings totals; `innings_number` 3–6 are **super overs** (16+16+1+1 rows)	✅
deliveries	295,732	One row per ball	✅ core; see column notes below
dismissals	14,705	One row per wicket	✅; `fielder_id` NULL on **all** rows
players	1,141	Player dimension, `cricsheet_id` UNIQUE	🟡 `country/role` empty or "Unknown/Player" on 793 rows; 177 synthetic `json:N` ids (squad-only players)
teams	19	Franchises incl. renamed ones	✅
venues	60	Ground names (several variants of the same ground)	🟡
seasons	20	Season ids incl. Cricsheet labels `2007/08`, `2009/10`, `2020/21` and a synthetic `All`	🟡 champions/runner-up NULL
delivery_commentary	293,652	Per-ball length, line, variation, shot, contact, zone, `speed_kph` (7,173 non-null) parsed from ESPN commentary	✅ enrichment
fact_batting_innings	18,842	Per-innings batting line	🟡 built at ingest
fact_bowling_spells	14,734	Per-innings bowling line	🔴 credits "retired out" to bowlers (6 cases)
player_career_stats / season / team / team_season / phase	804 / 3,337 / 1,631 / — / 2,193	Precomputed aggregates	🟡 built by `compute_analytics.ts`
player_matchups	31,370	Batter × bowler head-to-head	✅

Table	Rows	Purpose	Status
current_squad, player_team_history	804 each	"Latest team" derived from last appearance	🟡 heuristic
team_intelligence, venue_intelligence	19 / 60	Team/venue summaries	🟡
auction_roi, partnerships, dim_scenario, dim_team_match_bridge	**0**	Empty — designed but never populated	🟣
insight_clicks	1	Click counter for `/api/ipl/insights/go/:slug` (the only table the API writes)	✅

3.1 `deliveries` — the most important table

```
-- sqlite.db (abridged from the live schema)
CREATE TABLE deliveries (
  delivery_id    INTEGER PRIMARY KEY AUTOINCREMENT,   -- surrogate; chronological only WITHIN an innings
  match_id       TEXT    REFERENCES matches(match_id) ON DELETE CASCADE,
  innings_id     INTEGER REFERENCES innings(innings_id) ON DELETE CASCADE,
  over_num       INTEGER NOT NULL,                    -- 0-based (over 0 = first over)
  ball_num       INTEGER NOT NULL,                    -- 1..N position in the over (fixed 2026-10-11)
  batter_id, bowler_id, non_striker_id INTEGER REFERENCES players(player_id),
  batter_runs    INTEGER NOT NULL DEFAULT 0,
  extra_runs     INTEGER NOT NULL DEFAULT 0,
  total_runs     INTEGER NOT NULL DEFAULT 0,
  is_wide, is_noball            INTEGER NOT NULL DEFAULT 0,   -- populated
  is_bye, is_legbye, is_penalty INTEGER NOT NULL DEFAULT 0,   -- NEVER populated (0 on all rows)
  is_boundary, is_dot_ball, is_death_over, is_powerplay INTEGER,
  delivery_type, scoring_area TEXT,       -- commentary-derived today (see §7)
  pressure_state TEXT DEFAULT 'Standard', -- 'Standard' on all 295,732 rows (placeholder)
  current_rr, required_rr REAL,           -- placeholders (0)
  phase TEXT, wickets_left, wicket_seq, wickets_remaining, required_run_rate,
  delivery_seq INTEGER                    -- added 2026-10-11 with the sequencing fix
);
```

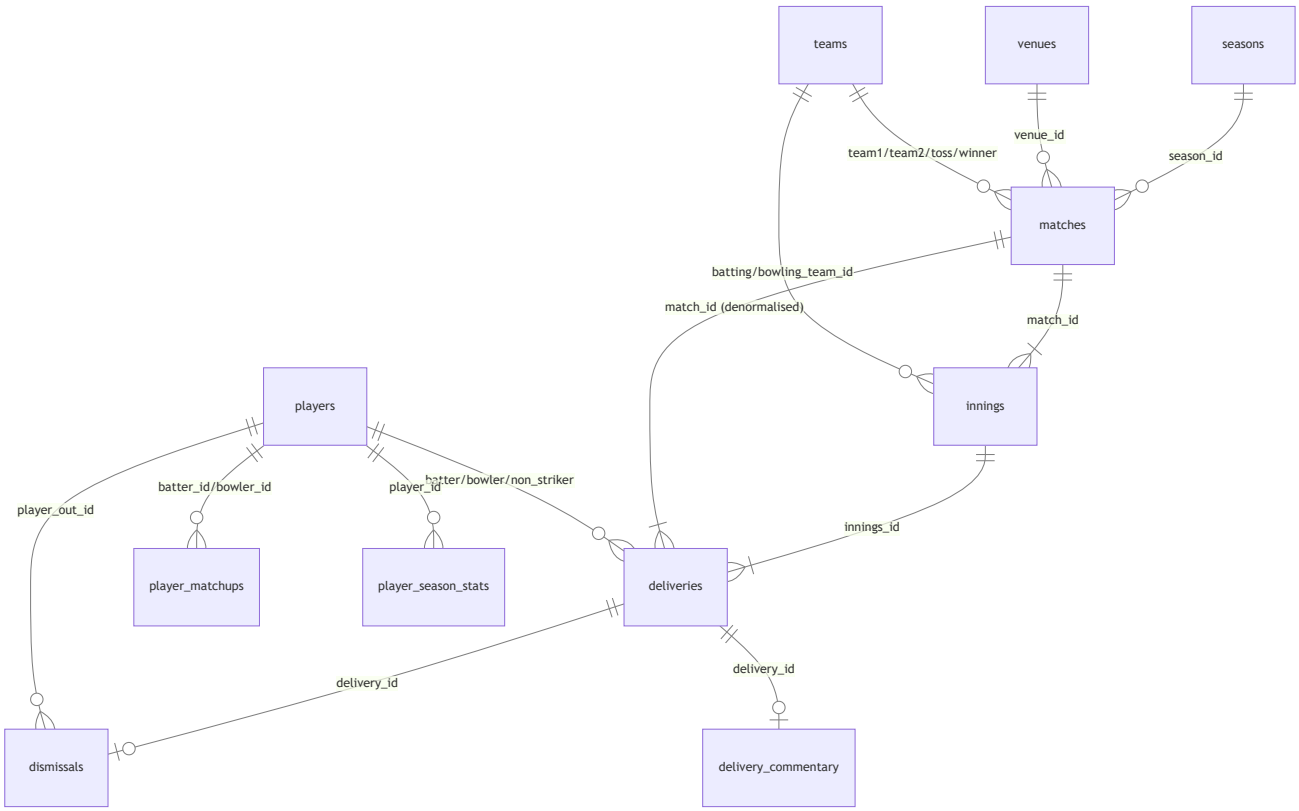
Verified facts (queries run on a copy):

Check	Result
`SUM(is_wide)` / `SUM(is_noball)`	9,876 / 1,226
`SUM(is_bye)`, `SUM(is_legbye)`, `SUM(is_penalty)`	**0, 0, 0** — the ingest never sets them (`update-ipl-data-v3.ts:580` insert has no such columns)
`SUM(extra_runs)`	20,209
`pressure_state` distinct values	only `'Standard'`
`delivery_seq IS NULL`	0

3.2 Indexes on `sqlite.db`

Index	Columns	Used by
`idx_deliveries_batter` / `_bowler`	`batter_id` / `bowler_id`	player career, matchups
`idx_deliveries_match_id`, `_match_innings`, `_innings`, `_filter_lookup (match_id, over_num, ball_num)`	match scans	scorecards, season filters
`idx_matches_season`, `_venue_id`, `_team1_id`, `_team2_id`, `_teams`		season/venue/team filters
`idx_dismissals_delivery`, `idx_dismissals_player_out` **and** `dismissals_player_out_idx`		dismissals — note the **duplicate** index on `player_out_id` (wasted space, slower writes)
`idx_delcomm_*`	commentary `match_id`, `length`, `zone`	Advanced Stats delivery filters

3.3 Relationships (ERD)



Foreign keys are declared but enforcement depends on the connection. `lib/db/src/index.ts` runs `PRAGMA foreign_keys = ON`; the decision layer opens read-only so enforcement is moot; the ingest turns them OFF while dropping tables (`update-ipl-data-v3.ts:20`).

---

4. Global warehouse (`global.db`) — schema

37 tables + 4 views, schema version 1, rules version `roles-v1` (`meta` table). Commented DDL: `scripts/src/global/schema.sql`, `scripts/src/scout/scout-schema.sql`.

4.1 Layers

```
Source registry   competitions(6) · source_files(6) · ingest_runs · meta · schema_drift(0)
Raw-faithful      matches(3,309) · innings(6,648) · deliveries(771,965) · wickets(39,972)
                  wicket_fielders(29,663) · powerplays · replacements · reviews · match_players(73,743)
Identity          players(18,623) · player_names(29,254) · player_source_ids(28,528)
                  identity_links(964) · identity_review_queue(177) · team_aliases(4)
                  teams(92) · venues(185) · venue_names(185)
Aggregates        agg_batting(32,192) · agg_bowling(25,757) · agg_fielding · agg_appearances
                  competition_baselines(260) · seasons(65)
Scouting          scout_batting_innings(51,567) · scout_bowling_innings(39,201) · scout_roles(11,759)
                  scout_phase_variance(260) · scout_coverage(65)
Enrichment        player_attributes(1,960) · attribute_link_issues(285)
Views             v_batting · v_bowling · v_competition_baselines · v_player_competitions
```

4.2 Design choices worth explaining

Choice	Where	Why it matters
`deliveries` PK = `(match_id, innings_no, over_no, seq) WITHOUT ROWID`	`schema.sql`	`seq` is the **true bowling order inside the over**, so a 10-ball over cannot collide or wrap (the bug that hit `sqlite.db`). `WITHOUT ROWID` stores rows clustered by that key.
Per-type extras columns `wides, noballs, byes, legbyes, penalty` + `non_boundary`	`deliveries`	Runs conceded can be computed exactly; the IPL warehouse cannot.
`extra TEXT` columns + `schema_drift` table	every raw table	Unknown JSON keys are preserved, never dropped silently.
`in_latest_source` flag	`matches`	A match missing from the newest archive is flagged, not deleted (lineage).
Aggregates keyed `(player_key, competition_id, season_label, phase)`	`agg_batting/agg_bowling`	One row per player-competition-season-phase; any window is a SUM. Phase rows sum to the `all` row (asserted in tests).
`scout_phase_variance` holds `n, Σx, Σx²`	scouting	Lets the API compute per-ball variance for any season window without re-scanning deliveries.

4.3 Global indexes

Index	Note
`ix_matches_competition_season (competition_id, season_label)`, `ix_matches_start_date`	
`ix_deliveries_batter`, `ix_deliveries_bowler`	
`ix_sbi_player / ix_sbo_player (player_key, competition_id, season_label)`	scouting innings
`ix_player_names_name`, `ix_player_source_ids_lookup (source, source_id)`	identity lookups
Gap: `agg_batting/agg_bowling` have no index leading with `competition_id`. The rankings query `WHERE competition_id = ? AND phase = ? AND season_label IN (…)` relies on the PK's *skip-scan* (`SEARCH … USING PRIMARY KEY (ANY(player_key) AND competition_id=? …)` — see Q7 plan). Fast today (≤ 32 k rows) but it would not scale to 100× the data.	

---

5. Data dictionary (core columns)

5.1 Cricket terms → columns

Concept	IPL warehouse expression	Global warehouse expression
Ball faced (batter)	`is_wide = 0`	`wides = 0`
Legal ball (bowler)	`is_wide = 0 AND is_noball = 0`	`wides = 0 AND noballs = 0`
Runs off the bat	`batter_runs`	`runs_batter`
Runs conceded by bowler	`RUNS_CONCEDED_SQL = batter_runs + (is_wide ? extra_runs : 0) + (is_noball ? 1 : 0)` (`services/decision/db.ts:88`)	`runs_batter + wides + noballs`
Boundary	`batter_runs IN (4,6)` (no `non_boundary` flag)	`runs_batter IN (4,6) AND non_boundary = 0`
Phase	`over_num` 0–5 PP, 6–14 middle, 15–19 death	same, plus 5-ball sets for The Hundred
Super over	`innings.innings_number > 2`	`innings.super_over = 1`
Bowler wicket	`dismissals.kind NOT IN ('run out','retired hurt','retired out','obstructing the field', …)`	metric definition in `scripts/src/global/metrics.ts`

5.2 Columns you must **not** trust

Column	Why
`deliveries.is_bye / is_legbye / is_penalty`	Always 0
`deliveries.pressure_state / current_rr / required_rr`	Placeholders; real match state is rebuilt in `services/decision/ball-context.ts`
`matches.match_type`	Always "T20"
`seasons.champion_team_id`	NULL
`dismissals.fielder_id`	NULL
`players.country / role`	Mostly empty; app merges `squads-by-season.json`, `player-bio.json` (`services/decision/player-profiles.ts`)
`fact_bowling_spells.wickets`	Credits "retired out"; use `dismissals`
`delivery_id` order across matches	Ingest order, not date order — sort by `matches.date`

---

6. Representative SQL (all executed; timings on a copy)

> SQL is shown in a readable form. Production code builds equivalent queries with **parameters** (`?`) — see §9 on SQL injection.

Q1 — Player career batting (IPL, super overs excluded)

```
SELECT p.name,
       COUNT(DISTINCT d.match_id)                          AS matches,
       SUM(d.batter_runs)                                  AS runs,
       SUM(CASE WHEN d.is_wide = 0 THEN 1 ELSE 0 END)      AS balls,      -- wides are not balls faced
       ROUND(100.0 * SUM(d.batter_runs)
             / SUM(CASE WHEN d.is_wide = 0 THEN 1 ELSE 0 END), 2) AS sr,
       SUM(d.batter_runs = 4) AS fours, SUM(d.batter_runs = 6) AS sixes
FROM deliveries d
JOIN innings i ON i.innings_id = d.innings_id
JOIN players p ON p.player_id  = d.batter_id
WHERE p.name = 'V Kohli' AND i.innings_number <= 2          -- exclude super overs
GROUP BY p.player_id;
```

- **Result:** V Kohli — 275 matches, 9,336 runs, 6,926 balls, SR 134.8, 845 fours, 317 sixes. **81 ms.**
- **Plan:** `SCAN p | SEARCH d USING INDEX idx_deliveries_batter | SEARCH i USING INTEGER PRIMARY KEY | TEMP B-TREE FOR count(DISTINCT)` — the players scan is cheap (1,141 rows) but an index on `players(name)` would remove it.

- **Edge cases:** `COUNT(DISTINCT match_id)` counts matches *batted in*, not matches played (a player who did not bat is missed — use `agg_appearances`/squads for appearances). Dismissals must come from `dismissals` (Q1b: 231 outs, 10 ms), excluding `retired hurt`.

Q2 — Season strike-rate leaders (2025, min 200 balls)

```
SELECT p.name, SUM(d.batter_runs) runs, SUM(d.is_wide = 0) balls,
       ROUND(100.0 * SUM(d.batter_runs) / SUM(d.is_wide = 0), 1) sr
FROM deliveries d
JOIN matches m ON m.match_id = d.match_id
JOIN innings i ON i.innings_id = d.innings_id
JOIN players p ON p.player_id = d.batter_id
WHERE m.season_id = '2025' AND i.innings_number <= 2
GROUP BY d.batter_id HAVING balls >= 200
ORDER BY sr DESC LIMIT 5;
```

Result: N Pooran 196.3 (524/267), Abhishek Sharma 193.4, Priyansh Arya 182.3, PD Salt 176.0, SS Iyer 175.1. **71 ms**; plan starts from `idx_matches_season` then `idx_deliveries_match_id`. **Season gotcha:** 2008, 2010, 2020 are stored as `2007/08`, `2009/10`, `2020/21`; `normalizeSeason()` in `services/decision/db.ts:60-70` maps them.

Q3 — Bowling phase analysis: death-overs economy since 2022 (min 100 legal balls)

```
SELECT p.name,
       SUM(d.batter_runs + CASE WHEN d.is_wide=1 THEN d.extra_runs ELSE 0 END
                         + CASE WHEN d.is_noball=1 THEN 1 ELSE 0 END)          AS runs,
       SUM(d.is_wide = 0 AND d.is_noball = 0)                                  AS legal,
       ROUND(6.0 * SUM(/* same expression */ …) / SUM(d.is_wide=0 AND d.is_noball=0), 2) AS econ
FROM deliveries d JOIN matches m ON m.match_id=d.match_id
JOIN innings i ON i.innings_id=d.innings_id JOIN players p ON p.player_id=d.bowler_id
WHERE d.over_num >= 15 AND i.innings_number <= 2
  AND CAST(substr(m.season_id,1,4) AS INT) >= 2022
GROUP BY d.bowler_id HAVING legal >= 100 ORDER BY econ LIMIT 5;
```

- **Result:** SP Narine 6.06 (218 runs / 216 legal), JJ Bumrah 7.28, PWH de Silva 8.11, Noor Ahmad 8.26, L Ngidi 8.53. This matches the Ask page live output and an independent recount in `global.db` (218 runs, 216 balls).
- **871 ms. Plan:** `SCAN d USING INDEX idx_deliveries_bowler` — the whole table is walked in bowler order because no index covers `over_num` and the season filter is on `matches`. **Optimisation exercise:** a composite index `(over_num, bowler_id)` or reading from a pre-aggregated phase table (as `global.db` does) — see Challenge 11.
- **Why the CASE expression:** byes/leg-byes are not the bowler's fault, but `is_bye` is never set, so the code charges only what it can *prove*: wides (all their runs) + 1 per no-ball + bat runs. This mis-states 17 deliveries (multi-run no-balls, a 5-penalty wide; net −30 runs) — documented in `docs/global-data-engine/step3/economy-residuals.md`.

Q4 — Venue analysis: average first-innings score (min 20 matches)

```
SELECT v.name, COUNT(*) n, ROUND(AVG(i.runs),1) avg1
FROM innings i JOIN matches m ON m.match_id=i.match_id JOIN venues v ON v.venue_id=m.venue_id
WHERE i.innings_number = 1 GROUP BY v.venue_id HAVING n >= 20 ORDER BY avg1 DESC LIMIT 5;
```

Result: Uppal (Hyderabad) 194.7, Jaipur 194.5, Arun Jaitley 193.6, Chinnaswamy 190.4, Eden Gardens 190.3. **1 ms.** ⚠️ Venue names are not normalised in SQL ("Wankhede Stadium" and "Wankhede Stadium, Mumbai" are separate rows); the app rolls variants up in TypeScript (`resolveVenue` alias map, `stats-engine.ts`). Rain-shortened innings are averaged with full ones — a known bias.

Q5 — Team comparison: head-to-head CSK v MI

```
SELECT w.short_name winner, COUNT(*) n
FROM matches m
JOIN teams a ON a.team_id IN (m.team1_id, m.team2_id) AND a.short_name='CSK'
JOIN teams b ON b.team_id IN (m.team1_id, m.team2_id) AND b.short_name='MI'
LEFT JOIN teams w ON w.team_id = m.winner_id
GROUP BY m.winner_id;
```

Result: MI 21, CSK 20. **1 ms.** Franchise renames (Delhi Daredevils→Capitals, Kings XI→Punjab Kings, RCB Bangalore→Bengaluru) are separate `teams` rows; queries must group through the alias map (`stats-aggregator.ts:214 getTeamAliases`).

Q6 — Player comparison: batter v bowler (Kohli v Bumrah)

Result: 164 runs off 110 balls, 5 dismissals. **4 ms** via `idx_deliveries_bowler`. Production uses the precomputed `player_matchups` table or `stats-engine.ts`.

Q7 — Global T20 scouting: death-overs strike rate, IPL + BBL, latest seasons

```
SELECT p.unique_name, c.code, SUM(a.runs) runs, SUM(a.balls_faced) balls,
       ROUND(100.0*SUM(a.runs)/SUM(a.balls_faced),1) sr
FROM agg_batting a JOIN competitions c USING (competition_id) JOIN players p USING (player_key)
```

```
WHERE c.code IN ('ipl','bbl') AND a.phase='death' AND a.season_label IN ('2026','2025/26')
GROUP BY a.player_key, c.code HAVING balls >= 60 ORDER BY sr DESC LIMIT 5;
```

Result: Tilak Varma 256.7 (154/60), D Ferreira 221.7, Nithish Kumar Reddy 215.9, TH David 214.1, SS Iyer 212.7 — **6 ms** because it reads pre-aggregated rows. **This is raw strike rate; Scout Lab ranks on league-adjusted, shrunk indices (Chapter 6), which is why its order differs.**

Q8 — One player across leagues (AD Russell, by Cricsheet id `bbd41817`)

League	Runs	Balls	SR	Outs
BBL	454	291	156.0	19
CPL	2,117	1,257	168.4	79
ILT20	675	405	166.7	23
IPL	2,651	1,522	174.2	94
SA20	30	23	130.4	3

1 ms. The join is on the registry id, so no name matching is involved.

Q9 — Performance trend: league run rate by phase (IPL 2024–2026)

`SELECT season_label, phase, ROUND(6.0*runs_total/legal_balls,2) FROM competition_baselines WHERE competition_id=1 …` → 2024 all 9.56, death 11.21, PP 9.47; 2025 all 9.63 … **1 ms**, PK search.

---

7. Data quality: fixed vs unresolved

Issue	Status	Evidence
Ball sequencing wrap — `ball_num` restarted after the 9th delivery of an over ("19.10" parsed as float 19.1 → ball 1)	✅ **Fixed 2026-10-11** in `sqlite.db` (`delivery_seq` added). Backup `sqlite.db.pre-delivery-fix.bak` shows `1,2,…,9,1,1`; current DB `1…11` for match 1473469 over 19.	`update-ipl-data-v3.ts:659-661` root cause; `scripts/src/step3/tests/delivery-sequence.test.ts` (3/3 pass on a copy). ⚠️ The script that applied the fix to production is **not in the repo** (lineage gap) and the ingest script still contains the float-parsing bug.
Byes/leg-byes charged to bowlers (7,142 runs across history)	✅ Fixed in code via `RUNS_CONCEDED_SQL`	`services/decision/db.ts:77-89`
Residual: 17 deliveries / 13 bowlers / net −30 runs	🟡 Unresolved; needs per-type extras columns (`scripts/src/step3_5/migrate-delivery-extras.ts` is written and refuses to touch production without `--allow-production`; **not applied**)	`economy-residuals.md`
Random `delivery_type`/`scoring_area` at ingest	🟡 Current values are commentary-derived (`scratch/backfill-delivery-columns.mjs`); 95,511 / 100,451 NULLs. Re-running the ingest would re-introduce random values.	`update-ipl-data-v3.ts:678-681`
`pressure_state` placeholder	🟡 Worked around (`ball-context.ts`)	
Duplicate player names	✅ Handled by registry id in `global.db` (11 collision groups, e.g. two "Yuvraj Singh") and by exact-only rules on player pages (Step 3)	`docs/global-data-engine/identity.md`; live check: `resolvePlayer("Yuvraj Singh")` returns 3 candidates and **no** player
177 squad-only players with synthetic ids	🟡 In `identity_review_queue`; 59 have exact-name candidates awaiting human review	`identity.md`
Empty designed tables (`partnerships`, `auction_roi`, …)	🟣	row counts

---

8. Transactions, concurrency and integrity

- **Ingest** (`update-ipl-data-v3.ts`): one `BEGIN … COMMIT` per match file with `ROLLBACK` on error (`:735-741`), so a bad file never half-loads. But the script first **drops every table**, so a failure mid-run leaves a partial warehouse.
- **Global ETL**: incremental — only new or changed zip entries (by CRC) are re-normalised; whole steps run inside transactions; exit code 2 if validation fails.
- **Runtime**: `node:sqlite`'s `DatabaseSync` is **synchronous**. Every query blocks Node's single event loop. Measured: one filtered `/api/stats/query` ≈ 2.3 s warm; ten concurrent ≈ **24 s** wall time (they queue). This is the system's main scalability limit (Chapter 11).
- **WAL mode** (`PRAGMA journal_mode = WAL`) lets readers proceed while a writer appends; the 27 MB `-wal` file next to `sqlite.db` is un-checkpointed write-ahead log. Copying `sqlite.db` without its `-wal` gives a stale database — a classic backup mistake.

---

9. SQL injection posture

Code path	Parameterised?
Decision layer (`services/decision/*`)	✅ `db.prepare(sql).all(...params)`
Scout service	✅ placeholders built with `inList()` (`"?"` per value). One string-interpolated fragment: `` `AND ${PHASE_SQL} = '${phase}'` `` (`scout-service.ts:390`) — safe only because `phase` is Zod-validated to an enum in `routes/scout.ts`

Code path	Parameterised?
Legacy stats (`stats-aggregator.ts`)	✅ Drizzle `sql` template tag binds `${…}` as parameters (e.g. `LIKE LOWER(${…})` at `:1772`)
Ingest scripts	🟡 `db.exec(\`UPDATE innings SET runs = ${totalRuns} …\`)` (`update-ipl-data-v3.ts:715`) — values are computed numbers, not user input, so not exploitable, but it is a bad pattern to copy

---

10. Interview-level questions this chapter prepares you for

1. Why is `COUNT(*)` of deliveries not "balls faced"? (Wides.) Why not "balls bowled"? (Wides and no-balls.)
2. Why does Q3 take 871 ms when Q7 takes 6 ms? (Raw fact scan vs pre-aggregated rows; index coverage.)
3. What breaks if you copy `sqlite.db` without the WAL file?
4. Why store per-type extras instead of one `extra_runs` column?
5. Why is `delivery_id` not safe for ordering across matches?
6. When would you move from SQLite to PostgreSQL/DuckDB? (Concurrent writers, multi-instance deploys, analytical scans over 10^8 rows, need for async drivers.)

05 — Data Engineering and Cricket Data Modelling

> **Scope.** Where every number on the site comes from, how it is transformed, how it is checked, and which known problems remain. Two pipelines exist and they have very different maturity:

Pipeline	Maturity	One-line verdict
A. IPL warehouse pipeline (CSV → `sqlite.db` → static JSON)	🟡 Grew organically; many one-off scripts; full drop-and-rebuild; hand-applied fixes	Works, but is **not reproducible end-to-end from the repo**
B. Global data engine (Cricsheet archives → `global.db`)	✅ Designed pipeline with hashing, incremental loads, 118 validation checks, tests	The strongest engineering in the repository

---

1. Where match data originates

Source	What	Licence / terms	Used by
Cricsheet (cricsheet.org)	Ball-by-ball JSON and CSV for every match; `people.csv` / `names.csv` registers	**ODC-BY 1.0** — free use **with attribution**	Both warehouses
ESPNcricinfo ball commentary	Length, line, variation, shot, zone, speed text per ball	**Not licensed for reuse** (scraped)	`delivery_commentary`, `ball-commentary.json`
Cricbuzz	Live scores, scorecards	**Not licensed** (scraped with a browser User-Agent)	`/api/live/*`
Google News RSS, ESPNcricinfo RSS	Headlines	RSS terms; link-out only	`/api/ipl/news`
`2026 ipl auction player list*.csv`	Official auction list: capped status, reserve price, specialism	Public document; provenance should be cited	War Room, scouting enrichment
`player-auctions.json`, `squads-by-season.json`, `player-bio.json`, `captains.json`	Auction prices since 2014, squads, bios	The `/data-sources` page states season and auction facts were compiled from Wikipedia (**CC BY-SA 4.0**, share-alike); per-file provenance is not recorded in the repo	War Room, Dugout, player metadata
Official IPL / franchise sites	Player photos (≈100 scraping scripts in `scripts/*.mjs`)	**Copyrighted images**	Player avatars

> **Interview-grade honesty:** only the Cricsheet-derived parts have a clean licence. Commentary text, live scores and photos come from scraping. That is acceptable for a personal learning project but is the first thing a franchise or employer would ask about (Chapter 25 §licensing).

---

2. How raw files are structured (Cricsheet JSON 1.2.0/1.3.0)

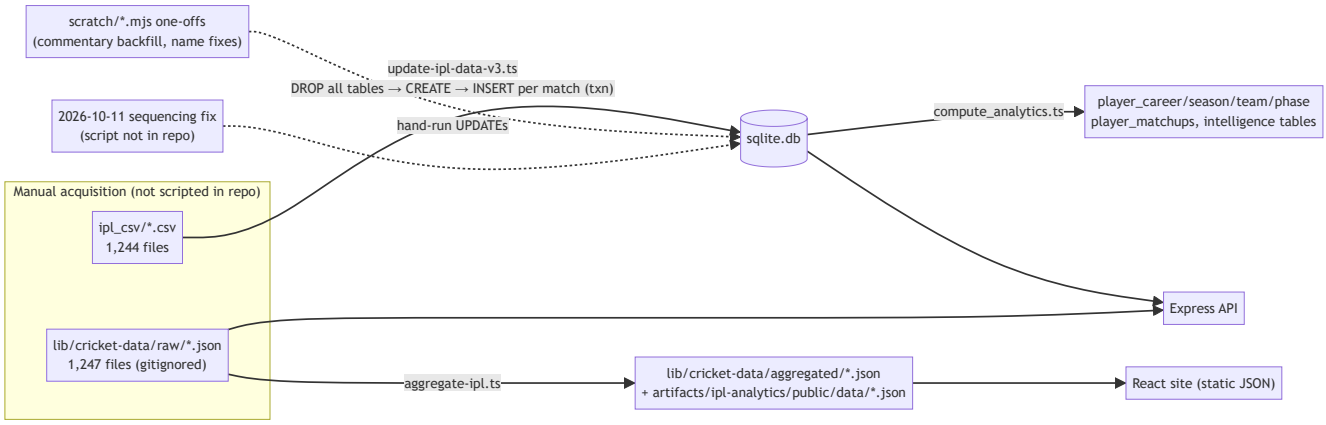
```
{
  "meta": { "data_version": "1.2.0", "revision": 1 },
  "info": {
    "dates": ["2025-05-26"], "season": "2025", "teams": ["Punjab Kings", "Mumbai Indians"],
    "event": { "name": "Indian Premier League", "stage": "Qualifier 1" },   // only place the stage exists
    "registry": { "people": { "RG Sharma": "740742ef", … } },              // name → stable 8-hex id
    "toss": {…}, "outcome": { "winner": "…", "by": { "wickets": 7 } },
    "overs": 20, "balls_per_over": 6, "players": { "<team>": ["…", …] }
  },
  "innings": [
    { "team": "Mumbai Indians",
      "target": { "runs": 204, "overs": 20 },                              // chases (incl. DLS)
      "overs": [
        { "over": 0, "deliveries": [
            { "batter": "RG Sharma", "bowler": "…", "non_striker": "…",
              "runs": { "batter": 0, "extras": 1, "total": 1 },
              "extras": { "wides": 1 } },                                   // wides / noballs / byes / legbyes / penalty
            { …, "wickets": [{ "player_out": "…", "kind": "caught", "fielders": [{ "name": "…" }] }] }
        ]}
      ]}
  ]
}
```

The CSV variant (`ipl_csv/*.csv`, 1,244 files) flattens each ball to one row with columns `match_id, season, start_date, venue, innings, ball ("19.4"), batting_team, bowling_team, striker, non_striker, bowler, runs_off_bat, extras, wides, noballs, byes, legbyes, penalty, wicket_type, player_dismissed, …`.

---

3. Pipeline A — IPL warehouse

3.1 Flow



3.2 Steps with code locations

#	Step	Code	Notes
1	Acquire CSV/JSON	**none in repo**	Files were downloaded by hand. `.gitignore:52` excludes `lib/cricket-data/raw/`, so a fresh clone **has no raw JSON** and many API features (`getMatchData`, stage filters, ball context) silently degrade.
2	Rebuild schema	`scripts/src/update-ipl-data-v3.ts:19-345`	Drops 23 tables then recreates them.
3	Identity	`:378-575`	Builds a canonical player map from each file's registry + `artifacts/api-server/data/players.json`; one row per Cricsheet id; fallback key `name:<string>` when a file has no registry (0 such rows today).
4	Load matches, innings, deliveries, dismissals	`:578-741`	One transaction per file; `ROLLBACK` on any error.
5	Per-innings facts	`:702-733`	`fact_batting_innings`, `fact_bowling_spells` computed in memory.
6	Squads	`:750-785`	`current_squad` = team of last appearance (heuristic).
7	Analytics tables	`scripts/src/compute_analytics.ts`	Career/season/team/phase/matchups.
8	Static JSON for the site	`scripts/src/aggregate-ipl.ts` → writes both `lib/cricket-data/aggregated/` and `public/data/` (`:907-908`)	Overview, top batters/bowlers, seasons, matches, squads…
9	Enrichment	`scratch/build-delivery-commentary.mjs`, `scratch/backfill-delivery-columns.mjs`, `scratch/fix-db-player-names.mjs`	**One-off scripts in `scratch/`**, run by hand.

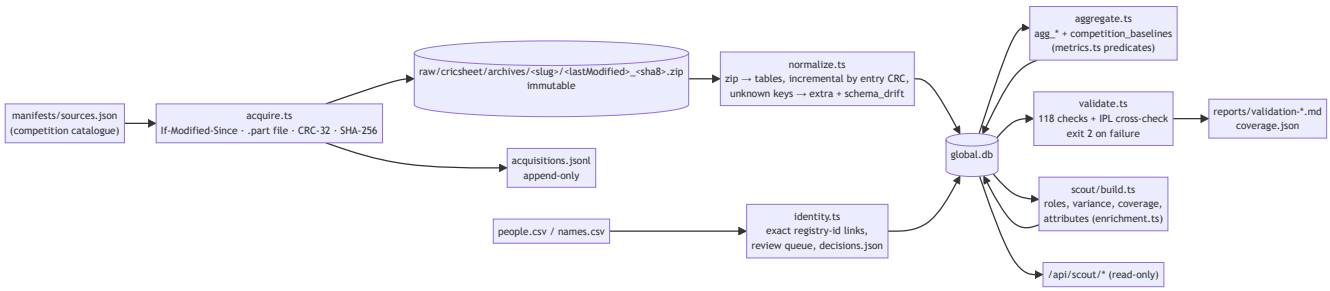
3.3 How each cricket concept is modelled (IPL warehouse)

Concept	Implementation	Problem / fix
Over/ball	`overBall = parseFloat("19.4"); overNum = floor; ballNum = round((overBall − overNum) × 10)` (`:659-661`)	🔴 "19.10" → 19.1 → ball 1. Every over with >9 deliveries wrapped. **Fixed in data on 2026-10-11** (`delivery_seq`), **still present in the script**. Correct approach: split the string on "." and parse each part as an integer, or use the JSON's array position (as `global.db` does with `seq`).
Legal vs illegal delivery	`is_wide`, `is_noball` from CSV columns 9/10	✅
Ball faced	`if (!isWide) balls += 1` (`:705`)	✅ no-balls are faced
Bowler runs conceded (ingest)	`totalBallRuns − byes − legbyes` (`:712`)	✅ correct in `fact_bowling_spells`; but `is_bye/is_legbye` are not stored, so SQL over `deliveries` cannot repeat this → `RUNS_CONCEDED_SQL` workaround
Bowler wicket	excludes `run out`, `retired hurt`, `obstructing the field` (`:714`)	🔴 forgets `retired out` (6 cases)
Dot ball	`runsOffBat === 0 && extras === 0`	⚠️ treats a leg-bye as *not* a dot for the batter; Cricsheet convention (and `global.db`) counts byes/leg-byes as batting dots
Phase	`overNum < 6` PP, `≥ 15` death	✅ consistent with global engine
Super overs	innings 3–6 stored; excluded at query time (`innings_number <= 2`, JSON path `inningIndex >= 2`)	✅
Franchise renames	separate `teams` rows; alias map `TEAM_ALIASES` in `migrate_deliveries.ts`, `getTeamAliases()` in `stats-aggregator.ts:214`	✅ Deccan Chargers ≠ Sunrisers is a deliberate rule (`stats-aggregator.ts:140-150` comment)
Seasons	Cricsheet labels kept (`2007/08`, `2009/10`, `2020/21`)	normalised in code (`normalizeSeason`)
Delivery type / scoring area	`Math.random()` at ingest (`:678-681`)	🔴 latent; data later overwritten from commentary (§6)

---

4. Pipeline B — Global data engine

4.1 Flow



One command runs all of it: `pnpm --filter @workspace/scripts run global:sync` (or `global:build` offline). Full rebuild ≈ 27 s, identical to the incremental result (`docs/global-data-engine/README.md`).

4.2 Engineering properties that are genuinely good

Property	How
Idempotent, incremental	A match is re-ingested only if new or its zip-entry CRC changed (`normalize.ts` header)
Immutable raw layer	Archives stored by `<lastModified>_<sha8>`; never deleted
Lineage	`acquisitions.jsonl` (url, time, Last-Modified, bytes, SHA-256), `source_files`, `ingest_runs`, `meta` timestamps, `in_latest_source` flags
Schema drift detection	Unknown JSON keys stored verbatim + counted (`schema_drift` = 0 rows today)
Single source of metric truth	Every predicate in `scripts/src/global/metrics.ts`, pinned by `tests/metrics.test.ts`
Validation as a gate	`validate.ts` → 118 PASS, 2 WARN, 0 FAIL on 2026-10-10 (`reports/validation-2026-10-10.md`)
No guessing	Attributes Cricsheet lacks (bowling style, hand, capped) are linked only from named sources, with `confidence`; unlinkable rows go to `attribute_link_issues` (285)

4.3 The validation checks (per competition)

From `scripts/src/global/validate.ts:78-189`:

Check	IPL result (2026-10-10)
Match counts agree (catalogue, archive, README, database)	1,243 everywhere
Every match male / club / 5–6-ball overs	0 non-conforming
`runs.total = batter + extras` **and** `extras = wides + noballs + byes + legbyes + penalty`	0 failures / 295,732
Legal balls per over ≤ balls_per_over (except Cricsheet `miscounted_overs`)	4 long overs, all explained
Overs per innings ≤ scheduled	0
Wickets per innings ≤ 10 (≤ 2 in super over), retired hurt excluded	0
Win by runs = 1st total − 2nd total	545 checked, 0 differ
Win by wickets = 10 − wickets lost	650 checked, 0 differ
Winning chase reaches target	650 checked, 0 short
Ties have equal totals	16 checked, 0 differ
Target = 1st total + 1 (non-DLS)	1,203 checked, 0 differ
Every delivery references known players; every squad player in register	0 orphans
Season gaps / thin seasons / schema drift	continuous / none / 0
Cross-warehouse: player-season totals global vs `sqlite.db`	reported in the same file (`:202-306`)

The two WARNs are source facts (e.g., SMAT has no 2016/17 season in Cricsheet).

---

5. Player and team identity

Rule 1 — the canonical key is the Cricsheet registry id (8 hex chars, e.g. `bbd41817` = AD Russell). Every match file maps the written name to this id in `info.registry.people`.

Rule 2 — names never merge records. Verified live: `resolvePlayer("Yuvraj Singh")` returns three candidates (`1c914163 Yuvraj Singh`, `30829ea2 Yuvraj Singh (2)`, `f0a99c08 YY Singh`) and **no** player, forcing disambiguation.

Rule 3 — anything without an exact id goes to a review queue with evidence; a human decision is stored in `manifests/identity-decisions.json` (tracked, survives rebuilds).

Bridge statistic (IPL warehouse → global)	Value
Players in `sqlite.db`	1,141
Linked exactly by registry id	964

Bridge statistic (IPL warehouse → global)	Value
Synthetic `json:N` ids (squad-only, never faced or bowled)	177 → review queue; 59 have exact-name candidates

Player-page identity (Step 3 / 3.5). The website's player pages originally matched names by initial + surname, so "Prince Choudhary" showed Mukul Choudhary's matches. Step 3.5 replaced this with exact registry-identity resolution and an audit (`scripts/src/step3/identity-audit.ts`). Its latest saved report (`docs/global-data-engine/step3_5/identity-audit.md`) shows 778 correct pages, 10 empty (no verified identity), **0 wrong**. **Today's run, however, reports 20 pages as "MISSING (empty page)"** (e.g., `gurjapneet-singh`, `salil-arora`) — the bios have no verified key, so the page shows an empty state although the player has matches. That is a regression against the Step 3 test (Chapter 10).

Team identity. `manifests/team-aliases.json` lists four explicit, sourced renames (Daredevils→Capitals, Kings XI→Punjab Kings, Rising Pune Supergiants→Supergiant, RCB Bangalore→Bengaluru). Nothing is inferred from name similarity.

---

6. Extras, wickets and unusual deliveries

Situation	Correct cricket rule	Global warehouse	IPL warehouse
Wide	Not a ball faced; not a legal ball; all wide runs charged to bowler	✅	✅
No-ball	Ball faced; not legal; 1 run (the no-ball penalty) charged to bowler + bat runs	✅	✅ via `RUNS_CONCEDED_SQL`
No-ball where batters also *run* byes	byes are not the bowler's	✅ (`noballs` + `byes` separate)	🟡 cannot split → 16 of the 17 residual deliveries
Bye / leg-bye	Ball faced; legal; **not** charged to bowler; batting dot	✅	🟡 flags not stored
Penalty runs	Not charged to bowler	✅	🟡 1 residual delivery (wide + 5 penalty)
Boundary that was actually run (`non_boundary`)	Not a four	✅	🔴 counted as a four
Retired hurt	Not a dismissal	✅	✅
Retired out	Batting dismissal, **not** a bowler wicket	✅	🔴 credited to bowler in `fact_bowling_spells`
Run out of the non-striker	Dismissal of the non-striker	✅	✅ (`player_out`)
Super over	Excluded from career stats	✅ `super_over` flag	✅ innings 3–6 excluded at query time
DLS-shortened chase	Target from the file	✅ `target_runs/overs`	rebuilt from raw JSON in `ball-context.ts` (34 DLS targets)
Over with 10+ deliveries	Keep true order	✅ `seq`	✅ after 2026-10-11 fix

---

7. How scorecards and aggregates are computed

- **IPL scorecards** (`/api/ipl/match/:id/scorecard`) are built by `services/match-card.ts` from the raw Cricsheet JSON, not from SQL — so the scorecard is independent of warehouse bugs but requires `lib/cricket-data/raw/`.
- **Advanced Stats** has two engines (`services/stats-aggregator.ts:703-730`): a **SQL path** (Drizzle over `sqlite.db`) and a **JSON replay path** that walks every ball of every raw file in memory (all 1,247 files are pre-loaded at boot — `preLoadAllMatches()`). The JSON path is used when the filter needs per-ball context the DB lacks (batting hand, pressure). Two engines for one feature means two sets of rules to keep consistent — a maintainability risk.
- **Decision modules** use a third representation: `ball-context.ts` builds **typed-array column stores** (~25 MB for 295k balls) once at startup so simulations can scan every ball in milliseconds.
- **Global aggregates** are additive counts per (player, competition, season, phase); every rate is computed *after* summing, never by averaging rates — the correct way to combine samples.

---

8. How data is refreshed

Data	Refresh	Automated?
`global.db`	`global:sync` → incremental	✅ one command; **not scheduled** (manual)
`sqlite.db`	`update-ipl-data-v3` (full rebuild) + `compute-analytics` + hand-run `scratch/` fixes + the undocumented sequencing fix	🔴 not reproducible: re-running the documented steps would **lose** the commentary backfill and **re-introduce** the ball-number wrap and random delivery types
Static JSON (`public/data`)	`aggregate-ipl` and several generators (`scripts/generate-player-recent-matches.mjs`, …)	🟡 manual
Live scores / news	On request with in-memory caching	✅ runtime
Photos	~100 one-off scripts + registry validation (`scripts/validate-photo-registry.mjs`)	🟡 manual
`drizzle-kit push`	`scripts/post-merge.sh` runs `pnpm --filter db push` against `../../sqlite.db` after every Replit merge	🔴 a schema mismatch could alter the production warehouse automatically

---

9. Data lineage summary

Question	Global engine	IPL warehouse
Which file produced this row?	`source_files`, archive SHA-256, match `revision`	match id only
When was it loaded?	`meta.normalized_at`, `ingest_runs`	not recorded
What changed since last load?	`reports/change-*.md`	not recorded
Can I rebuild it from scratch?	Yes, ≈27 s	No (manual steps lost)
Which rule defined this metric?	`metrics.ts` + tests	spread across 3 engines

---

10. Known problems — fixed vs open

Fixed (with evidence): delivery sequencing in data; bowler runs conceded (byes/leg-byes) in all decision/stat code; player-page wrong-player matches (Step 3.5); 33 MB recent-matches file split into shards; random delivery types replaced in data.

Open: see Chapter 4 §7, plus — IPL acquisition not scripted; `sqlite.db` not reproducible; 177 unlinked squad players; bowling style unknown for most non-IPL bowlers; no opposition-strength adjustment; scraped sources without licences; 20 player pages now empty (regression).

---

11. Complete data dictionary (global warehouse core)

Table.column	Type	Meaning
`competitions.code`	TEXT	`ipl`, `smat`, `bbl`, `sa20`, `cpl`, `ilt20`
`matches.match_id`	INTEGER PK	Cricsheet match id
`matches.season_label`	TEXT	as in source (`2025`, `2025/26`)
`matches.balls_per_over`	INTEGER	6 (5 for The Hundred)
`matches.event_group`	TEXT	e.g. "Plate Group" (SMAT lower division)
`matches.stage`	TEXT	Final, Qualifier 1, …
`matches.in_latest_source`	INTEGER	0 if dropped from newest archive
`innings.super_over`	INTEGER	1 for super overs
`innings.target_runs / target_overs`		chase target incl. DLS
`deliveries.over_no`	INTEGER	0-based
`deliveries.seq`	INTEGER	0-based true order within the over
`deliveries.actual_delivery`	TEXT	Cricsheet over.ball label
`deliveries.runs_batter / runs_extras / runs_total`	INTEGER	
`deliveries.wides / noballs / byes / legbyes / penalty`	INTEGER	runs of each extra type
`deliveries.non_boundary`	INTEGER	1 if 4/6 runs were run, not a boundary
`wickets.kind / player_out`		one row per wicket
`wicket_fielders.player_key / substitute`		catches, run-out involvement
`players.cricsheet_id`	TEXT UNIQUE	registry id; `unreg:<comp>:<name>` if absent
`players.unique_name`	TEXT	display name, unique
`agg_batting.(balls_faced, runs, fours, sixes, dots, dismissals)`	INTEGER	per player × competition × season × phase
`agg_bowling.(balls, legal_balls, runs_conceded, wickets, dots, fours, sixes, wides, noballs)`	INTEGER	same grain
`competition_baselines.*`	INTEGER	league totals for the same grain
`scout_roles.(batting_role, bowling_role, usage_profile, inputs)`	TEXT	rules `roles-v1`; `inputs` JSON stores the evidence
`scout_phase_variance.(faced_n, faced_sum, faced_sumsq, legal_n, conceded_sum, conceded_sumsq)`	INTEGER	per-ball variance ingredients
`scout_coverage.status`	TEXT	`partial` if < 60% of median season matches
`player_attributes.(attribute, value, normalized, source, confidence)`	TEXT	bowling style, batting hand, capped status — never guessed
`identity_links.(source, source_key, player_key, method, confidence)`		links from other systems

06 — Scouting and Statistical Engine

> **This is the chapter interviewers will probe hardest.** Every formula below was read from source *and* executed against `global.db` / `sqlite.db` (read-only) on 2026-10-12. Worked examples use real output. Where the code's comments, the UI text, or older docs disagree with what the code does, the code wins and the disagreement is listed.

---

0. Map of the statistical code

Engine	File(s)	Data	Surfaces in
Scout ratings (league adjustment, shrinkage, intervals, percentiles)	`artifacts/api-server/src/services/scout/scout-service.ts` (API copy) and `scripts/src/scout/service.ts` (CLI copy)	`global.db`	`/scout`, `/scout/player/:id`, `/scout/compare`
Role classification	same file `:117-191`; built into `scout_roles` by `scripts/src/scout/roles.ts` + `build.ts`	`global.db`	`/scout/roles`, profile pages
Trends, similar players, strengths/weaknesses	`scout-service.ts:503-740`	`global.db`	Scout profile
Impact model (runs above average per match)	`services/decision/impact-model.ts`	`sqlite.db`	Playing XI, War Room, Insight Feed
Hidden Weakness Detector	`services/decision/weakness-detector.ts`	`sqlite.db`	`/ai-analyst/weakness`
Pressure Genome	`services/decision/pressure-genome.ts`	`sqlite.db`	`/ai-analyst/pressure`
Match engine (outcome tables, DP win probability, Platt calibration, Monte Carlo)	`services/decision/match-model.ts`	`sqlite.db`	Strategy Sim, Alt Universe, Decision Replay, Opposition Map
Player Pool similarity	`services/decision/player-pool.ts`	`sqlite.db`	Player Pool, War Room comparables
Squad DNA	`services/decision/squad-dna.ts`	`sqlite.db`	Squad DNA, War Room "fit"
War Room valuation	`services/decision/auction-war-room.ts`	`sqlite.db` + auction files	`/ai-analyst/auction-war-room`
Playing XI optimiser	`services/decision/playing-xi.ts` (javascript-lp-solver)	`sqlite.db`	Playing XI drawer

None of these is machine learning in the trained-model sense except two small fitted pieces: a 2-parameter Platt calibration (logistic) in the match engine, and a k-nearest-neighbour price estimate in the War Room. Everything else is deterministic statistics (Chapter 9).

---

1. Scout ratings — the core pipeline

1.1 Purpose

Rank T20 players across six leagues on a scale where **100 = league average for the same competition, season and phase**, while (a) not letting tiny samples top the table and (b) showing how uncertain each number is.

1.2 Inputs

For batting, per player × competition × season × phase from `agg_batting`: balls faced `n`, runs `a`, boundaries `b` (4s+6s), dots `c`, dismissals `d`. For bowling from `agg_bowling`: legal balls `n`, runs conceded `a`, wickets `b`, dots `c`, boundaries conceded `d`. (`scout-service.ts:277-283`)

1.3 Step 1 — League expectation (era/league adjustment)

For each season *s* in the window, the league rate is the sum over **all** players:

$$ r_{s} = \frac{\sum_{\text{players}} a_{s}}{\sum_{\text{players}} n_{s}} $$

A player's **expected** count is accumulated season by season using *their* balls:

$$ E[a] = \sum_{s} n_{p,s}, r_{s} \qquad (\texttt{p.ea += r.balls * l.a / l.balls}, \text{ line } 321) $$

Index:

$$ \text{index} = 100 \times \frac{\text{actual}}{\text{expected}} $$

Plain English: "Abhishek Sharma scored 31% more runs than an average IPL 2026 batter would have scored off the same number of balls."

Property (tested): summed over every player, expected = actual exactly, so the league average index is 100 (`scout:build` asserts it).

Limitation: the adjustment is **within a league**. An index of 130 in SMAT and 130 in the IPL are each "30% above their own league" — **not** equal ability. No league-strength multiplier exists (`docs/global-data-engine/scout-service.md`, "Known limitations").

1.4 Step 2 — Shrinkage toward the league average

$$ \text{shrunk} = \frac{n \cdot \text{index} + k \cdot 100}{n + k}, \qquad k = 120 \text{ balls} \quad (\texttt{shrink()}, \text{ line } 96) $$

Plain English: the player's own record counts as *n* balls of evidence; the league average counts as 120 balls of evidence. With 27 balls the league dominates; with 1,500 balls the player dominates.

What kind of statistics is this? It is the **posterior mean of a normal–normal model with a fixed prior weight** — the same algebra as Bayesian credibility weighting. It is "empirical-Bayes-*style*", **not** full empirical Bayes: `k = 120` is a chosen constant, not estimated from the between-player variance of the data. The UI text ("Empirical Bayes shrinkage (K=120)") slightly overstates this. A defensible interview phrasing: *"credibility-weighted toward the league mean with a fixed prior weight of 120 balls; k was chosen, not fitted."*

1.5 Step 3 — Uncertainty intervals (frequentist, not Bayesian)

Metric type	Method	Code
Proportions (boundary %, dot %, wicket rate, dismissal rate)	**Wilson score interval**, 95% (z = 1.959964), on the raw proportion, then divided by the league proportion to express it on the index scale	`wilsonInterval()` `:86-94`; `prop()` `:403-412`
Strike rate, economy (runs per ball)	**Normal approximation**: actual ± z·√(σ²·n), divided by expected; σ² is the **league's** per-ball variance for that competition/season/phase window from `scout_phase_variance` (Σx, Σx², n)	`:291-295`, `:415-421`, `:435-440`
Balls per dismissal	Wilson interval on the dismissal rate, inverted	`:424-430`

Wilson interval, for k successes in n trials:

$$ \tilde p = \frac{\hat p + \frac{z^2}{2n}}{1+\frac{z^2}{n}}, \qquad \text{half} = \frac{z\sqrt{\frac{\hat p(1-\hat p)}{n} + \frac{z^2}{4n^2}}}{1+\frac{z^2}{n}} $$

These are frequentist confidence intervals, not Bayesian credible intervals. Three caveats you must state:

1. The SR/economy interval uses the **league's** per-ball variance because a player's own variance cannot be recovered from aggregates; a six-hitter's true variance is larger, so their interval is too narrow. The code comments call these intervals "approximate".
2. **The interval is centred on the raw index, but the leaderboard headline is the shrunk index.** The two come from different frameworks. Example: V Suryavanshi shows *shrunk 137.4* next to *95% [137.4–157.4]* — the headline sits on the lower edge of its own interval. Display both or neither; do not imply the interval belongs to the shrunk value.
3. For balls-per-dismissal with zero dismissals the upper bound is `Infinity`, which JSON serialises as `null`.

1.6 Step 4 — Percentiles

`percentileRanks()` (`:100-115`): sort the population's **shrunk** values in the "better" direction; a value at sorted positions *i…j* (ties) gets

$$ \text{pct} = 100 \times \frac{(i+j)/2}{N-1} $$

so 100 = best, 0 = worst, ties share the mid-rank. Population = same competition, phase and window, **balls ≥ minBalls**, plus any role filter (`:475-482`). Rows below `minBalls` get `percentile = null`.

Percentiles are computed per competition. In an "All 6 leagues" table, each league's leader is "100th percentile" of *its own* league — the live leaderboard shows six different players at 100th %ile in the top ten.

1.7 Sample tiers

`rankable` ≥ 120 balls · `low` 60–119 · `insufficient` < 60 (`sampleTier()` `:83-84`).

1.8 Worked example 1 — a rankable batter (real output)

Query: IPL, batting, season 2026, phase all (`ratePlayers({discipline:"batting", competitions:["ipl"], seasons:{ipl:["2026"]}})`).

Abhishek Sharma (`f29185a1`), 275 balls, 563 runs, 93 boundaries, 89 dots, 14 dismissals, role `opener`.

Metric	Value	League	Index	Shrunk	95% CI (index)	Percentile (of 57)
Strike rate	204.73	156.34	130.9	**121.5**	116.8–145.1	98.2
Boundary %	33.82	22.47	150.5	135.2	126.8–176.3	96.4
Dot % (lower better)	32.36	34.14	94.8	96.4	79.4–111.6	60.7
Balls per dismissal	19.64	19.05	102.9	102.0	62.8–171.7	33.0

Check the arithmetic yourself:

- Index = 100 × 204.73 / 156.34 = **130.95** ✓
- Shrunk = (275 × 130.95 + 120 × 100) / (275 + 120) = (36,011 + 12,000) / 395 = **121.5** ✓

Interpretation for a recruiter: clearly above league scoring rate with a tight-enough interval to be confident (lower bound 116.8 > 100); boundary-driven; dismissal rate is average, so the value comes from tempo, not survival.

1.9 Worked example 2 — why shrinkage exists

Azmatullah Omarzai, IPL 2026, **27 balls**, 52 runs, SR 192.6 (index 123.2). Shrunk = (27 × 123.2 + 12,000)/147 = **104.3**; CI 78.2–168.2 (contains 100); tier `insufficient`; percentile `null`. A raw-SR table would place him near the top; the engine says "we cannot tell him from average yet." The repository test `shrinkage pulls small samples toward league average` asserts a 190 SR over 35 balls ranks below 155 over 600 balls.

1.10 Worked example 3 — a bowler, and a bug

JJ Bumrah, IPL 2026, **death overs**, 78 legal balls, 106 runs, 1 wicket, role `powerplay_specialist` (role is computed over all phases).

Metric	Value	League	Index	Shrunk	Percentile (of 21)
Economy (lower better)	8.15	10.67	76.4	90.7	95
Balls per wicket	78.0	14.18	18.2 (wkt/ball)	67.8	0
Dot %	32.05	30.87	103.8	101.5	55
Boundary % conceded	16.67	24.07	69.2	87.9	85

The economy percentile is direction-aware and correct (95 = among the best). **But the API's default sort is wrong:** `routes/scout.ts:138-153` sorts bowling by `metrics.economy.shrunk` **descending** (the UI default `sortOrder = "desc"`, `scout-home.tsx:83`). A higher economy index is *worse*. Verified: IPL bowling, latest season — rank 1 is SN Thakur (economy 12.27 vs league 9.77, shrunk 113.4, **percentile 0**). The code comment "shrunk index is already direction-oriented" is false. **This is the single most important scouting defect to fix before showing the page to anyone** (Challenge 6 is built on it).

1.11 Other ratings defects found in this audit

Defect	Evidence	Effect
`minBalls` not applied to the ranking list	`routes/scout.ts:105-125` filters only by search; `competitionRatings` returns all rows. In IPL latest-season bowling, 64 of 125 rows are below 120 balls.	"Showing 25 of 3,435 qualified players" — most are not qualified; a 115-ball player appears in the default "Rankable" view.
Role column always "Unclassified"	API returns `role: {label, inputs}`; `RolePill` expects a string (`components/scout/scout-metric-badge.tsx:99-106`; `scout-home.tsx:533`)	Role filter works server-side, but the role is never displayed.
API typecheck fails	`routes/scout.ts:123` reads `r.unique_name`, absent from `Rating`	Search-by-unique-name branch is untyped; build relies on esbuild skipping type errors.
Bowling label "Strike rate (balls per wicket)" with `better: "higher"`	`BOWLING_METRICS`	Percentile is right (index is wickets per ball) but the label invites the wrong reading.

---

2. Role classification (`roles-v1`)

Roles describe **usage**, not quality. Computed per player × competition × season and stored with their inputs (`scout_roles.inputs` JSON).

Inputs (from every ball, super overs excluded): arrival position, entry point (legal balls already bowled when the batter arrived), share of balls per phase, whether the bowler bowled over 1 or 2.

Batting (first rule that matches, `battingRole() :128-135`):

#	Rule	Role
1	innings < 3	`insufficient`
2	opened (position ≤ 2) in ≥ 50% of innings	`opener`
3	median position ≥ 8	`lower_order`
4	median entry ≥ 72 legal balls (over 13+) **or** (≥ 50% of balls at the death **and** median position ≥ 5)	`finisher`
5	median entry < 36 legal balls	`top_order`
6	otherwise	`middle_order`

Bowling (`bowlingRole() :137-145`): `none` if no balls; `insufficient` < 60 legal balls; `part_time` < 1.5 overs/innings; a **phase specialist** if exactly one phase's share ≥ 1.6 × its natural share (6/20, 9/20, 5/20 overs → thresholds 48% PP, 72% middle, 40% death); else `multi_phase`.

Why 1.6× natural share and not a flat 50%? The death is only 5 of 20 overs, so a flat 50% would make death specialists almost impossible. This is a good design argument to make in an interview.

Usage profile (`usageProfile() :147-155`): `keeper_batter` (has stumpings and bats), `all_rounder`, `bowler`, `batter`, `limited`.

Validation: unit tests pin the rule order and thresholds (`scripts/src/scout/tests/scout.test.ts`: "batting roles follow the documented order", "bowling roles scale with phase length", "usage profiles"). **Not validated:** agreement with how franchises or commentators label players. Roles are a reproducible *description*, not ground truth.

---

3. Trends (`trends() :656-699`)

For each competition: latest season vs all earlier seasons, on the **raw** league-adjusted index (SR for batting, economy for bowling).

$$ \text{SE} = \frac{\text{CI}_{high} - \text{CI}_{low}}{2z}, \qquad \text{SE}_{diff} = \sqrt{SE_L^2 + SE_E^2}, \qquad \text{significant if } |\Delta| > z \cdot SE_{diff} $$

Verdict `improving` / `declining` only if significant; `stable` otherwise; `insufficient sample` if either side has < 60 balls.

Real example (Abhishek Sharma): IPL batting SR index 115 → 131 (±16 at 95%), 1,114 → 275 balls → **improving**; SMAT 107 → 161 (±23) → improving; SMAT bowling economy 87 → 104 (±29) → **stable** (not significant despite a large move); IPL bowling → insufficient sample (29 balls).

Limitation: earlier seasons are pooled (a player who peaked three seasons ago and declined since looks "improving" if the latest season beats the pooled mean); assumes independence between periods.

---

4. Similar players (`similarPlayers()` `:701-740`)

Feature vector per rankable player: the shrunk index of every discipline metric + 100 × powerplay share, death share, middle share. Each feature is **z-scored** across the pool; distance = Euclidean on z-scores; nearest *k* returned. If the target's bowling type is verified (pace/spin), only same-type bowlers are compared; **unknown is never treated as spin**.

Real example: Abhishek Sharma (IPL, last 2 seasons) → FH Allen 0.63, Priyansh Arya 1.13, A Mhatre 1.66, V Suryavanshi 2.16, PD Salt 2.35 — all aggressive openers, which is a plausible sanity check, **not** a validation.

Limitations: equal feature weights; correlated features (SR and boundary %) double-count tempo; no validation set of "known similar" pairs.

---

5. Strengths and weaknesses text (`:503-513`)

A metric at ≥ 75th percentile becomes a strength sentence, ≤ 25th a weakness, generated from the numbers (e.g. "Strike rate 229.0 vs league 155.3 (100th pct of 289)"). No language model is involved.

---

6. Impact model (`impact-model.ts`) — runs above an average player per match

$$ \text{bat} = \sum_s w_s \Big[R - B,\overline{rpb}_s - \big(O - B/\overline{bpo}_s\big) W_s\Big] $$ $$ \text{bowl} = \sum_s w_s \Big[B,\overline{rpb}_s - R_c + \big(Wk - B/\overline{bpw}_s\big) W_s\Big] $$

- W (run value of a wicket) = 0.25 × runs-per-ball × balls-per-out ≈ ¼ of the season's runs per wicket (~6.5 runs).
- Recency weight ×0.55 per season back (`recencyWeight`).
- Per-match rate shrunk toward a prior of −1.5 runs/match with weight 4 matches: `(Σ + 4 × −1.5) / (m + 4)`.
- Confidence: high ≥ 10 weighted matches, medium ≥ 4.

Assumption to defend: a wicket is worth a fixed fraction of runs-per-wicket. *Not validated:* against match outcomes (no win-contribution regression exists).

---

7. Hidden Weakness Detector (`weakness-detector.ts`)

Split a player's balls into cells (bowler type × over window for batters; batter hand × over window for bowlers; plus single dimensions and innings). Per-ball value `v = runs − W·out`. For each cell with ≥ 30 balls:

$$ z = \frac{\bar v_{cell} - \bar v_{rest}}{\sqrt{s^2_{cell}/n_{cell} + s^2_{rest}/n_{rest}}} \qquad p = \Phi(z) $$

Bonferroni correction: confidence = 1 − min(1, p × K) where K = number of cells tested. A weakness needs confidence ≥ 80% **and** a material gap (≥ 12 runs per 100 balls, or getting out 1.5× as often). Otherwise cells go to a "watch list" labelled not significant.

Real saved example (RG Sharma, career, 5,514 balls, 44 cells tested): one significant weakness — spin in overs 7–10, SR 103.8 vs 138 elsewhere, z = −3.51, confidence 99%.

Strength: explicitly controls the multiple-comparisons problem — a common mistake in sports analytics. *Limitation:* Bonferroni is conservative; bowler type depends on the incomplete style data (unknown types are their own row); cells overlap (a ball is in several cells), so tests are not independent.

---

8. Pressure Genome (`pressure-genome.ts`)

Contexts: death overs, knockouts, close finishes (last 5 overs of a live chase, required rate 6–14, ≤ 8 down), steep chase (RRR > 12), collapse rescue (4+ down inside 12 overs), defending (bowling). Each context compared with **the league in the same context**. Value per ball = runs above league − (wicket cost = 0.3 × league runs per wicket) × outs above league; shrunk toward 0 with 40 balls: `v·n/(n+40)`. **Pressure index** = percentile among players with ≥ 120 pressure balls; **lift** = pressure value − calm value.

Interpretation trap: "pressure" is defined by match state, not by anything psychological; "archetype" names ("Ice-cold finisher") are labels on numeric thresholds.

---

9. Match engine (`match-model.ts`)

1. **Ball-outcome tables** (dot/1/2/3/4/6/wicket/wide/no-ball) by era × over band × wickets down × required-rate band; sparse cells shrunk toward parent cells.
2. **Player multipliers** vs league in the same slot, shrunk toward 1, built "as of" a season (no future leakage).

3. **Win probability:** exact dynamic programme over (balls left, runs needed, wickets lost).
4. **Platt calibration:** `p = σ(a + b·logit(p_raw))`, fitted by Newton's method on the start of every over of every 20-over chase (`:414-476`) — the raw DP was under-confident (a raw 15% won ~31%).
5. **Seeded Monte Carlo** for player-specific simulations; the Alt-Universe engine uses common random numbers so the difference between baseline and what-if is the change, not noise.

Validation gap: the calibration is fitted and evaluated on the same historical chases (in-sample). A defensible next step is season-based holdout (fit on ≤ 2023, evaluate on 2024–26) with Brier score and a reliability diagram.

---

10. War Room valuation (`auction-war-room.ts`)

Quantity	Definition	Code
Impact	§6, "as of" the season before the auction	`featuresAsOf()`
Replacement level	20th percentile impact among regulars (≥ 5 matches) that season	`replacementLevel()` `:182-185`
IAR	impact − replacement	
Reliability	sample / (sample + 8)	`:195`
Market M	**k-nearest-neighbour comparable sales**: weighted geometric mean of the 15 nearest past sales (last five auctions) in a feature space (IAR, overseas, log experience, all-rounder, keeper); weight = e^(−d/2) × 0.85^(years ago)	`knnDetail()` `:207-222`
Value V	max(0, IAR × reliability) × median market price per run of reliable IAR (separately overseas/domestic), **capped at the record fee in the window**	`:258-263`, `:345`
Signal	BUY if V ≥ 1.25 M and V − M ≥ ₹1 Cr; AVOID if IAR ≤ 0, or V ≤ 0.75 M with ≥ ₹1 Cr gap, or V < reserve; SCOUT if < 5 IPL matches; else HOLD	`:342-349`
Max bid	V rounded down to the bid increment	`:277-283`
Franchise fit	Squad DNA gaps (importance > 0.05, team percentile < 45) × the player's z-score on the mapped feature → 0–100	`teamNeeds()`, `fitFor()` `:471-518`

> ⚠️ **The file header still says "Ridge regression of ln(price)". The code uses kNN comparable sales.** The header is stale. Quote the code, not the comment.

10.1 Backtest — what the model actually achieved (run 2026-10-12)

`auctionBoard({ view: "backtest" })`: value the real 2026 auction list (369 players) using data up to 2025, then compare with what happened in IPL 2026.

Measure	Result	Honest reading
Training data	372 auction sales 2021–2025	small
Leave-one-out R^2 of ln(price)	0.38	prices are mostly *not* explained by output
Typical LOO miss	2.07×	a ₹5 Cr estimate is often ₹2.4–10 Cr
2026 list coverage	288 of 369 = **SCOUT** (no IPL record)	the model cannot value most of the list
Sold & matched	28	tiny evaluation set
Price rank correlation (Spearman)	0.72	ranks *prices* reasonably
Typical price miss on 2026 sales	1.6×	
Projection: pre-auction IAR vs actual 2026 impact	**ρ = 0.17, n = 9**	essentially **no demonstrated predictive validity** for performance
AVOID group (played ≥ 4 matches)	5 of 5 "Delivered"	the AVOID signal was contradicted
Hits listed by the backtest	0	
Misses listed	5 (e.g., JO Holder flagged AVOID, delivered +8.3 runs/match for ₹7 Cr)	

Live view (CSK needs): V Suryavanshi valued at the cap (₹27 Cr) with **marketBasis 0.1** (almost no comparable sales) — the UI must show that the estimate is extrapolated.

What you can claim: "I built a transparent valuation that separates *market price* from *output value* and backtested it without look-ahead; it ranks prices reasonably (ρ = 0.72 on 28 sales) but did not predict 2026 output (ρ = 0.17, n = 9)." **What you must not claim:** that it identifies undervalued players.

Known sensitivity (Step 3 risk 2): fixing bowler economy changed some batters' market estimates by up to 39% because kNN neighbours re-selected — the price model is unstable near decision boundaries.

---

11. Playing XI optimiser (`playing-xi.ts`)

Mixed-integer LP: maximise Σ $\text{adjustedImpact}_i \cdot x_i$ subject to 11 players, ≤ 4 overseas, ≥ 1 keeper, ≥ 20 overs of bowling capacity, min bowling/batting options, min spinners/pacers, locks/exclusions; solved with `javascript-lp-solver`. Marginal value of each selected player = objective drop when re-solved without them. *Hard constraints use only exact metadata* (overseas from squad files; keeper from parsed stumpings) — never fuzzy names.

12. What has and has not been validated

Component	Validated how	Not validated
Metric definitions (balls, legal balls, conceded, dots, boundaries, dismissals, phases)	Unit tests on fixtures (`metrics.test.ts`, 7 tests) + 118 data checks	—
League adjustment unbiasedness	Build-time self-check (Σexpected = Σactual)	—
Shrinkage behaviour	Unit test (small vs large sample ordering)	choice of k = 120
Percentiles	Unit test (mid-rank, ties, direction)	—
Roles	Unit tests of rules	agreement with expert labels
Intervals	Wilson implementation tested	coverage (does a 95% interval contain the truth 95% of the time?)
Cross-league comparability	—	**not validated**; no league-strength factor
Trends	—	false-positive rate
Similar players	sanity check only	
Weakness detector	method is principled (Bonferroni)	out-of-sample: do flagged weaknesses persist next season?
Match engine	Platt calibration (in-sample)	holdout calibration, Brier score
War Room	backtest (above)	predictive validity is weak
Economy in IPL warehouse	recount vs raw files: exact except 17 deliveries	—

13. How a franchise analyst should read Scout Lab today

1. Read **shrunk index** for ranking, **CI** for confidence, **balls** for sample, **tier** for eligibility — together.
2. Compare players **within a league**. Across leagues, treat a 120 in SMAT and a 120 in the IPL as "both well above their own league," nothing more.
3. Ignore the default bowling order until the sort bug is fixed; sort ascending, or read the percentile.
4. Treat roles as usage descriptions for the selected window.
5. For bowlers outside the IPL, bowling type is usually `unknown`; type-specific comparisons are unavailable.

07 — Frontend and UI Engineering

> **Evidence base:** source reading of `artifacts/ipl-analytics/src`, a static scan of every page (loading/error/responsive/accessibility markers), and live inspection of the site on a test server (desktop 1440×900 and mobile 375×812) on 2026-10-12. Design-quality judgements are in Chapters 20–23; this chapter is about *how the front end is built*.

---

1. Application shell

Concern	Implementation	File
Entry	`createRoot(...).render(<App/>)`; polyfills `MediaQueryList.addListener`; **suppresses** a known "addListener" error globally	`src/main.tsx`
Providers	`QueryClientProvider` → `TooltipProvider` → `TeamProvider` → `WouterRouter`	`src/App.tsx:156-175`
Routing	~75 `<Route>` entries; legacy paths redirect (`/on-field/*`, `/product`, `/merch` …)	`App.tsx:55-150`
Layout	Left icon rail (14 destinations), top bar (live ticker, team switcher, notifications), right "Intelligence Feed" rail, footer, consent banner, "Gully" mascot guide	`components/layout.tsx`, `ai-mascot-guide.tsx`
Theming	Choosing a franchise writes `--team-accent`, `--team-secondary`, `--team-glow`, `--team-surface-tint` on `<html>`; persisted in localStorage	`lib/team-context.tsx:255-298`
Analytics	GA4 page view per client-side navigation (`AnalyticsRouteTracker`), gated by consent	`components/consent-banner.tsx`
Ads	`AdSlot` / `AdVessel` render only if `VITE_ADSENSE_CLIENT` is set	`components/ad-slot.tsx`

Primary navigation (left rail, `layout.tsx:25-38`): Home · My Dugout · Beyond XI · Scout Lab · 2027 Intel · Stats · Pulse · Scores · Intel Hub · Visuals · Auction · Insights · AI Analyst · Shop — 14 top-level items with icon-only labels on desktop.

---

2. Data-access layer (how pages get data)

Client	Fetches	Caching
`lib/aggregates.ts`	**static JSON** `/data/*.json` (overview, seasons, teams, venues, top batters/bowlers, matches, squads, bios, images…) + some API (`team-analytics`, `venue-analytics`, scorecard, commentary, news, insights)	React Query `staleTime` ≈1 h; API-first-then-static fallback helper at `:400-420`
`lib/live.ts`	`/api/live/*`, `/api/auction/*`	polling: scores 30 s, win probability 15 s, **auction state every 1 s**
`lib/scout-api.ts`	`/api/scout/*`	5 min `staleTime` in pages
`lib/discovery-api.ts`, `sim-api.ts`, `ask-api.ts`, `dugout-api.ts`	decision endpoints	per page; saved demo snapshot shown **with a banner** if the API is unreachable
`lib/player-recent-matches.ts`	shard index + one shard per player	1 h
`lib/gemini-client.ts`	Google Gemini directly from the browser	none

---

3. Page-by-page

Legend for **Data**: 🟢 data-driven (warehouse/API) · 🔵 static JSON generated from the warehouse · 🟣 editorial/static content · 🟠 mocked/hardcoded/randomised.

3.1 Homepage — `/home` (`pages/home.tsx`, 653 lines)

- **User problem:** orient a fan: what's on, who is in form, where to go next.
- **Data:** 🟢 live IPL matches, orange cap, trending (`useLiveIplMatches`, `useLiveOrangeCap`, `useLiveTrending`); 🔵 venues, player images; 🟣 hero copy ("IPL is not just about cricket"), feature cards.
- **States:** 4 loading markers; **no error handling** (0 error markers) — if the live scraper fails the sections are simply empty.
- **Responsive:** 5 breakpoint classes only; hero image dominates the fold.
- **Accessibility:** 8 of 32 buttons have no accessible name (measured in-browser).

3.2 Advanced Stats — `/stats` (`pages/stats.tsx`, 4,411 lines)

- **User problem:** "Statsguru-style" slicing of IPL batting/bowling by season, team, opponent, venue, phase, over range, bowler type, delivery type, pressure, innings, match stage, representation, nationality, batter/bowler.
- **Data:** 🟢 `/api/stats/query`, `/drilldown`, `/matchup`, `/scenario`; filters in zustand (`lib/store.ts`).
- **Visualisations:** leaderboards, matchup drill-tabs (zones, delivery types), scenario panels.
- **States:** 7 loading markers, 15 empty-state messages, 1 error marker.
- **Performance:** each filtered query 1.7–2.3 s warm (Chapter 8); the full-screen hero means the first table is below the fold.
- **Hardcoded/dead code:** `FormGuideView` adds `Math.random()` noise to SR/economy and fabricates runs (`:4337-4399`) — **defined but not rendered**; delete it before anyone copies it.

- **Limitations:** 125 instances of 6–9 px text; one 4,400-line component.

3.3 Player profiles — `/players/:slug` (`pages/player.tsx`, 3,241 lines)

- **User problem:** everything about one player: identity, career, seasons, teams, recent matches, matchups, intelligence panels.
- **Data:** 🔵 bios, season stats, top lists, squads, captains, images; 🔵 recent-matches **index + per-player shard** (Step 3); 🟢 `/api/intel/player/:key`, `/api/stats/matchup`.
- **Identity:** URL slug → bio key (`lib/player-key.ts`) → exact registry id or **empty state** (no fuzzy matching).
- **States:** 10 loading markers, "Player not found" page.
- **Responsive:** most breakpoint usage of any page (92); also most tiny text (141).
- **Known gaps:** `/players/rohit-sharma` → "Player not found" (Rohit's page is `/players/rg-sharma`); 20 bios currently show an empty recent-matches state although matches exist (Chapter 10).

3.4 My Dugout — `/dugout`, `/dugout/:team` (`pages/dugout.tsx` + `components/dugout/*`)

- **User problem:** a franchise "headquarters": titles, record, captain, squad by season, stories.
- **Data:** 🟢 `/api/dugout/franchise`, `/api/dugout/squad` (warehouse + squads + captains; nulls shown as unknown, not invented — `dugout-data.ts` header); 🔵 photo registry; 🟣 team articles (`lib/team-articles.ts`), broadcaster links, video registry.
- **Observed:** CSK page renders a large empty band between the tab bar and the stories section at 1440 px (unfilled content area).

3.5 Scout Lab — `/scout`, `/scout/player/:id`, `/scout/compare`, `/scout/roles` (uncommitted)

- **User problem:** compare T20 players across six leagues fairly; shortlist.
- **Data:** 🟢 `/api/scout/*` over `global.db`.
- **Interactions:** league chips, phase, role, min-volume, sort field + order, search, pagination, compare tray (≤ 5), profile, role catalogue.
- **States:** loading and error markers on all four pages.
- **Defects (live-verified):** role column always "Unclassified"; default bowling order shows worst economy first; unqualified (< minBalls) players listed; each league's leader shows "100th %ile" in a mixed-league table; on mobile the table overflows horizontally (inside a scroll container) and the right rail takes ~45% of the width in the audit browser.

3.6 AI Analyst — `/ai-analyst` hub + 11 modules

Route	Module	Data
`/ai-analyst/ask`	Ask the database (+ Gemini tab)	🟢 `/api/ask`; 🟠 Gemini fallback text (Chapter 9)
`/weakness`	Hidden Weakness Detector	🟢; demo snapshot when offline (labelled)
`/pressure`	Pressure Genome	🟢; demo snapshot (labelled)
`/feed`	Insight Feed	🟢; demo snapshot (labelled)
`/strategy-sim`	Strategy Simulator	🟢 Monte Carlo; also calls Gemini client
`/alt-universe`	Alternate Universe	🟢
`/opposition-map`	Opposition Exploit Map	🟢
`/decision-replay`	Captain's Decision Replay	🟢
`/squad-dna`	Squad DNA	🟢
`/auction-war-room`	Auction War Room	🟢 + backtest
`/player-pool`	Player Pool graph	🟢

- Shared shell: `components/decision/module-shell.tsx` (header, player picker, segmented control, `DemoBanner`, `ErrorBanner`, diverging colour tokens).
- These pages have the **best** state handling in the app (5–15 error markers each, explicit empty/demo states) and the clearest methodology text.

3.7 Match Detail — `/match/:id` (`pages/match-detail.tsx`)

- 🟢 scorecard and commentary from raw Cricsheet + ESPN text; win probability (calibrated model for archived matches); intel panel.
- 🟠 `OversView` with `Math.random()` per-over runs and fake wicket dots exists at `:701-718` — **not rendered**.
- 15 loading markers (best on the site).

3.8 Scores / Pulse — `/scores`, `/pulse`

- **Scores:** 🔵 matches, points tables, squads (static archive browser) + filters; 3 empty states.
- **Pulse:** 🟢 live IPL matches (scraped) → 🟠 "emotion" meters (excitement includes `Math.random() * 10`, heat zones `+ Math.random() * 0.15`, `pulse.tsx:137,168`). Presented as live telemetry; it is a heuristic with noise.

3.9 Fan Arena / Beyond XI — `/arena`, `/beyond-xi`, `/fan-arena` (1,307 lines)

- 🟣 presentational landing page ("pixel-accurate rebuild of reference design", commit `e7748b3`); team theme only; 223 hex colours; no data fetching.

3.10 Other routes

Route	Nature
`/intelligence` (1,763 lines)	🟢/🟣 content hub + insight feed; role tabs (data/product/growth/business analyst)
`/feed` ("2027 Intel")	🟣 strategy content from `data/strategy-2027-data.ts` (1,347 lines of authored data)
`/auction`	🟢 multiplayer auction **game** (in-memory server state, random AI bidders)
`/teams/:slug`, `/venues/:slug`	🔵 + intel panels
`/visualizations`	🔵 gallery (sankey, knowledge graph, spatial, telemetry timeline)
`/shop` (1,803 lines)	🟣 merchandise showcase; no cart/checkout backend
`/insights`, `/reports`, `/blogs`	🟣 section pages
`/about`, `/methodology`, `/data-sources`, `/contact`, `/disclaimer`, `/privacy`	🟣 trust pages (methodology and data-sources pages exist — good practice)
`/photo-pipeline`, `/admin/photo-pipeline`	internal tool, **public**

---

4. Cross-cutting engineering assessment

Aspect	Finding	Evidence
Component size	5 pages > 1,300 lines; `stats.tsx` 4,411	`wc -l`
Code splitting	none (`React.lazy` = 0) → 4.11 MB JS (864 KB gzip) loaded on first visit	build output
Type safety at the API boundary	hand-written types drift from API (`role`)	`lib/scout-api.ts:23`
Error boundaries	none; a render error blanks the app	grep
Global error suppression	`main.tsx` swallows a class of runtime errors	`main.tsx:19-45`
Accessibility	`aria-*` attributes absent on most large pages; icon-only nav; `maximum-scale=1` disables pinch-zoom	scan + `index.html:5`
Fonts	body declares **"Outfit", which is never loaded**; Inter and JetBrains Mono are loaded but Inter is not applied to `body`	`index.css:43`, `index.html:14`
Theme tokens	shadcn HSL tokens defined, but Tailwind v4 `@theme` mapping missing → semantic classes like `bg-primary` don't resolve; many hard-coded hexes	`index.css`
Loading / error states	strong in AI Analyst & Scout; weak on Home, Dugout, Fan Arena	scan
Polling	auction 1 s per tab; scores 30 s	`lib/live.ts`
Mobile	at 375 px the right rail and overlays (cookie banner, mascot, tour card) cover a large part of the viewport in the audit browser; Advanced Stats title clipped	screenshots (see note)

> *Note on mobile measurements:* the right-rail width uses a framer-motion animation; in a hidden or throttled tab it can stay wide (documented in project notes). The 375 px screenshots were taken in a visible tab, but confirm on a physical phone before treating the rail width as a production bug.

---

5. How raw data becomes an understandable UI (worked example)

`deliveries` rows → `agg_batting` counts → `ratePlayers()` index/shrunk/CI/percentile → JSON → `scout-home.tsx` row:

```
V Suryavanshi  470f446b • IPL   RANKABLE (449B)   23m • 23i
  Shrunk SR index 137.4  · 100th %ile
  Raw index 147.4 · Raw SR 229.0
  95% [137.4–157.4]
```

The design choice that matters: the page shows **sample size, tier and interval next to the headline number**, which is rarer (and better) than most cricket sites. The weakness: four numbers on similar scales (137.4, 147.4, 229.0, the CI) without units or a legend, and the headline (shrunk) does not correspond to the interval (raw).

---

6. Interview questions

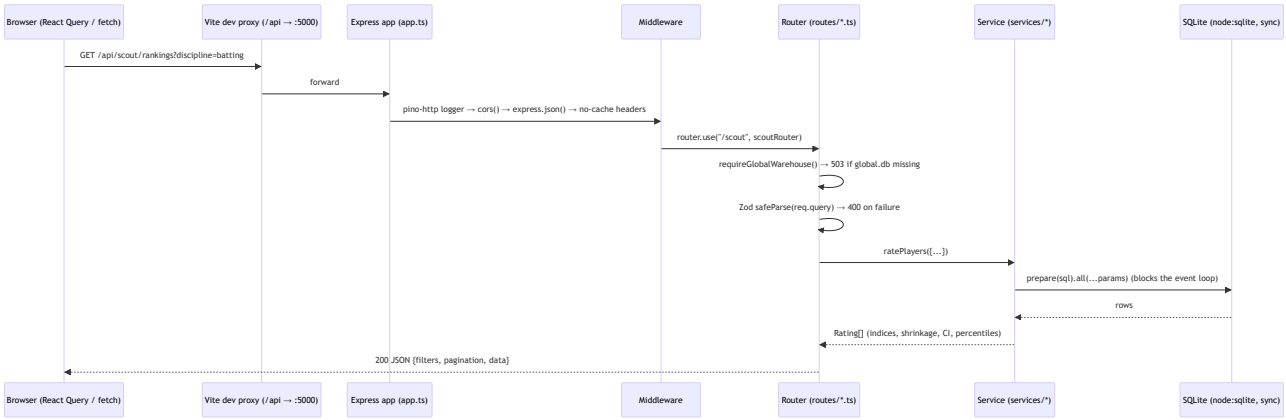
1. *Walk me through what happens between clicking "Bowling Rankings" and seeing the table.* (Chapter 2 §4.)
2. *Why is the role always "Unclassified", and how would you prevent that class of bug?* (Generated types from the OpenAPI spec / Zod schema shared by client and server.)
3. *How would you cut the 4 MB bundle?* (`React.lazy` per route, move three.js and Recharts into lazy chunks, image optimisation, remove dead pages.)
4. *What happens if `/api/live/ipl/matches` fails on the homepage?* (Sections stay empty; no error message — a UX gap.)
5. *How does team theming work without re-rendering every component?* (CSS custom properties on `<html>`.)

08 — API Engineering

> **Server:** Express 5 on Node 24, bundled by esbuild to `artifacts/api-server/dist/index.mjs`. All routes are mounted under `/api` (`src/app.ts:43`). **123 route handlers** in 9 files. **Measured:** latency and payload size were recorded on 2026-10-12 against the built bundle on a test port (`PORT=5055 node dist/index.mjs`), warm cache, same machine. "cold" = first call after start-up; "warm" = median of 3. These are single-user laptop numbers, **not** production benchmarks.

---

1. How a request flows through the server



Middleware stack (`src/app.ts`): `pino-http` (logs method, path without query, status, time) → `cors()` (**all origins**) → `express.json()` → `express.urlencoded()` → `Cache-Control: no-store` on every `/api` response → routers → static SPA fallback if `ipl-analytics/dist/public` exists.

Start-up (`src/index.ts`): listen on `PORT` (default 5000), then **warm eight decision caches synchronously** (measured 11.9 s, during which requests queue).

---

2. Conventions (and where they are broken)

Convention	Followed by	Broken by
Zod validation, `400 { error: "invalid_request", issues }`	`decision.ts`, `decision-intel.ts`, `scout.ts`	**all `ipl.ts` routes** (no validation; any string accepted — `/api/stats/query?season=2099` returns 200 with empty arrays after 2.8 s)
`503 { error: "warehouse_unavailable" }` when a DB is missing	decision, scout	ipl.ts silently falls back to JSON or literals
Stable error codes (`player_not_found`, `unknown_team`, …)	decision, scout	ipl.ts returns `500 { error: err.message }` — leaks internal messages
Read-only data access	everything **except** `/ipl/insights/go/:slug` (click counter) and `/ipl/sync-csv` (reads server files — see §9)	
Response caching	cricbuzz/news (30 s in-memory TTL)	every API response carries `no-store`, so browsers/CDNs can never cache even immutable historical stats

---

3. Endpoint reference

Legend — **Val**: Zod-validated ✅ / none ❌. **Test**: ✅ automated test exists · 🔵 called during this audit and returned the stated status · ❔ not exercised.

3.1 Health — `routes/health.ts`

Method	Path	Purpose	Val	Measured	Test
GET	`/api/healthz`	Liveness `{status:"ok"}` (validated by generated Zod schema)	✅	5 ms	🔵

3.2 Scout Lab — `routes/scout.ts` → `services/scout/scout-service.ts` → `global.db` (read-only)

Method	Path	Params (validation)	Response	Errors	Measured (warm)	Test
GET	`/api/scout/search`	`q` (string, trimmed; empty → `[]`)	`{results: [{player_key,cricsheet_id, name,unique_name}]}` (≤15, `LIKE %q%`)	503	5 ms	🔵
GET	`/api/scout/rankings`	`discipline` batting|bowling; `competitions` csv; `phase` all|powerplay|middle|death; `role`; `minBalls` 0–10000 (def 120); `recentSeasons` 1–20; `page`; `limit` 1–100; `sortBy`; `sortOrder` asc|desc; `search`	`{filters, pagination: {total,page,limit,totalPages}, data: Rating[] + strengths/weaknesses}`	400 `invalid_parameters` (🔵 `minBalls=abc` → 400), 503	35 ms batting; 141 ms bowling death; 36 KB	🔵 (service tested in `scout/tests/api.test.ts`)
GET	`/api/scout/player/:id`	key, Cricsheet id or exact name; ambiguous names return candidates	profile: attributes w/ provenance, season lines, roles, trends, similar players, IPL appearances	404 when no single player resolves (`scout.ts:199`), 503	**552 ms**, 20 KB	🔵
GET	`/api/scout/compare`	`ids` 2–5	side-by-side metrics + deltas	400, 503	85 ms	🔵
GET	`/api/scout/roles`	—	role catalogue, definitions, thresholds, active counts	503	64 ms (268 ms cold)	🔵

Known defects: `minBalls` not applied to the list; default bowling sort puts worst economy first; `unique_name` type error (`:123`). See Chapter 6 §1.10–1.11.

3.3 Decision Intelligence (Phase 1–2) — `routes/decision.ts` → `services/decision/*` → `sqlite.db` (read-only)

Method	Path	Purpose	Validation	Measured	Test
GET	`/api/players/:id/summary`	merged metadata, career, seasons, phases, top matchups	id resolved by `resolvePlayerId`	3 ms (32 cold)	🔵
GET	`/api/optimizer/franchises`	squads available for a season	`season` 4-digit + exists	❔	❔
POST	`/api/optimizer/playing-xi`	MILP Playing XI	`playingXiSchema` (teamId, pitch enum, overseas ≤ 11, locks/excludes, overrides…)	❔	❔
POST	`/api/ask`	natural-language question → `StatQuery` → answer	`question` 2–300 chars	78 ms (UI-observed)	🔵 via UI
POST	`/api/ask/run`	run an edited structured query	strict `statQuerySchema` (perspective, metric enum from `METRICS`, groupBy, filters…)	❔	❔
GET	`/api/history/overview`	warehouse coverage + live health checks	—	2 ms	🔵
GET	`/api/players/search?q=`	player picker	—	2 ms	🔵
GET	`/api/weakness/:id`	Hidden Weakness Detector	`window`, `side`	14 ms	🔵 (404 for unknown id)
GET	`/api/pressure/leaders`	Pressure leaderboards	`window`, `side`	8 ms	🔵
GET	`/api/pressure/:id`	Pressure Genome profile		2 ms	🔵
GET	`/api/insights/daily?date=`	five deterministic discoveries for a date	`date`	2 ms	🔵
GET	`/api/insights/pool/:category`	full candidate pool	category enum	❔	❔

3.4 Decision Intelligence (Phase 3–4, page intel, Dugout) — `routes/decision-intel.ts`

Method	Path	Purpose	Validation highlights	Measured	Test
GET	`/api/decision/catalog`	teams/venues/seasons for pickers	—	❔	❔
GET	`/api/strategy/setup`	batting/bowling squads for a chase	`battingTeam`, `bowlingTeam`, `season` (2008–2100, must exist)	❔	❔
POST	`/api/strategy/simulate`	Monte Carlo chase, ranked bowling plans	target 1–400, runs, balls 0–119, wickets 0–9, attack 2–8, oversUsed ≤ 4, sims 200–5000, seed	❔	❔
GET	`/api/alt-universe/matches`	match picker	`q` ≤ 80, `season`, `limit` ≤ 100	❔	❔
GET	`/api/alt-universe/match/:id`	innings snapshot		❔	❔
POST	`/api/alt-universe/simulate`	what-if (swap bowler / bowling type)	discriminated union on `change.type`	❔	❔
GET	`/api/replay/matches`, `/api/replay/:matchId`	Captain's Decision Replay		❔	❔
GET	`/api/opposition`	pre-match exploit map + full-match sim	`team`, `opponent`, `venue`, `season`, `sims` 200–4000	233 ms (389 cold)	🔵

Method	Path	Purpose	Validation highlights	Measured	Test
GET	`/api/pool/:id`	Player Pool similarity graph	`side`, `window`	❔	❔
GET	`/api/squad-dna/teams` · `/profile` · `/compare`	Squad DNA	`team`, `season`, `a/b`	6 ms (profile)	🔵
GET	`/api/war-room/board`	Auction board	`view` backtest|live, `team`, `role`, `overseas`, `maxPrice` 20–5000, `signal`, `q`, `sort`	3–4 ms, 94–156 KB	🔵
GET	`/api/war-room/player/:key`	Valuation detail + comparables	`view`	❔	❔
GET	`/api/intel/player/:key` · `/team/:key` · `/venue/:key` · `/match/:id`	Intelligence panels on profile pages		59 ms (player)	🔵
GET	`/api/dugout/franchise`	franchise record, titles, H2H, captain	`team` 1–40, `rival`	2 ms	🔵
GET	`/api/dugout/squad`	one season's squad	`team`, **`season` required** (4 digits)	🔵 400 without `season`	🔵

3.5 Legacy IPL analytics — `routes/ipl.ts` → `services/stats-aggregator.ts` + others (no Zod)

Group	Paths	Source	Notes
Overview & lists	`/api/ipl/overview`, `/seasons`, `/toss-analysis`, `/phase-analysis`, `/batsmen`, `/batsmen/impact`, `/batsmen/boundaries`, `/bowlers`, `/bowlers/death-specialists`, `/teams`, `/teams/head-to-head`, `/teams/championship`, `/venues`, `/records`, `/run-distribution`, `/dismissals`, `/top-batters`, `/top-bowlers`, `/matches`, `/current-squads`, `/squads-by-season`, `/player-auctions`, `/captains`, `/points-tables`, `/player-career-stats`, `/player-team-stats`, `/player-season-stats`, `/player-team-season-stats`, `/player-images`, `/player-photo-registry`, `/season-tops`	mostly `readDataJson()` → `ipl-analytics/public/data/*.json`; `/overview` uses SQL	🔴 `/ipl/overview` mixes SQL with **hardcoded literals**: `battingFirstWinPct 43.2` (data: 45.3), `chasingWinPct 56.8`, `avgSecondInningsScore 150`, `dotBallPct`, `totalBatsmen 703`, `totalBowlers 550`; `totalSeasons` = 20 (counts a synthetic "All" row). Its JSON fallback literals swap fours and sixes. `/top-batters` returns **1.2 MB**.
Team / venue analytics	`/api/ipl/team-analytics`, `/venue-analytics`, `/matchup-opponents`	`team-analytics.ts`	
Advanced Stats engine	`/api/stats/query`, `/stats/drilldown`, `/stats/matchup`, `/stats/scenario`, `/stats/player`, `/stats/team`, `/stats/venue` **and duplicates** under `/api/ipl/stats/*`	`aggregateQueryStats` (SQL path or JSON-replay path)	Unfiltered query short-circuits to career JSON (11 ms, 428 KB). **Season-filtered: 2.3 s warm, 492 KB; 10 concurrent: 24 s.** No validation.
Generic resources	`/api/players`, `/teams`, `/matches`, `/matchups`, `/venues`, `/stat-builder`, `/scenario-intelligence`	aggregator	
Match	`/api/ipl/match/:matchId/scorecard` (404 if no raw file), `/commentary`	`match-card.ts` from raw Cricsheet JSON + ESPN commentary	2 ms / 4 ms
Content	`/api/ipl/insights`, `/insights/go/:slug` (**writes** click count, then redirects), `/insights/clicks`, `/content/hub`, `/content/business`, `/news`	`insights.ts`, `content-hub.ts`, `news.ts`	`/news` 878 ms cold (RSS), 3 ms cached
Sync	**POST `/api/ipl/sync-csv`**	`cricsheet-csv-sync.ts`	🔴 **reads any file path supplied in the body** — §9

3.6 Live — `routes/live.ts` → `services/cricbuzz.ts, news.ts`

Path	Purpose	Notes
`/api/live/matches`, `/live/ipl/matches`	live scores (scraped)	30 s cache; **`?refresh=true` bypasses the cache for any caller**; 502 `live_source_unavailable` on failure
`/api/live/news`	ESPN RSS by category	category whitelisted
`/api/live/orange-cap`, `/purple-cap`, `/trending`, `/venue-insights/:venueId`	scraped	
`/api/live/win-probability/:matchId`	archived matches → calibrated chase model; otherwise scraped	replaced an earlier sine-wave fake (comment in `onlineDb.ts`)
`/api/live/match-details/:matchId`	team sheets + commentary from `match-card.ts`	previously returned invented squads (documented in `onlineDb.ts:1-10`)

3.7 Auction game — `routes/auction.ts` → `services/auction.ts`

`GET /state`; `POST /start`, `/select-team`, `/bid`, `/pause`, `/resume`, `/chat`, `/create-room`, `/join-room`, `/set-speed`, `/reset` under `/api/auction`.

🔴 **One global `AuctionEngine` instance** (`services/auction.ts:429`) holds the state for *all* visitors. Two people on the site share one auction; anyone can `POST /reset`. Fine for a single-player demo; unsafe for a public site.

3.8 Clip router — `routes/clip-router.ts`

Path	Purpose
POST `/api/get-clip-route`	maps a free-text query to a highlight clip: keyword matcher, or Gemini 1.5 Flash if `GEMINI_API_KEY` is set on the server
GET `/api/get-clip-catalog`	the clip catalogue

---

4. Example request / response

```
GET /api/scout/rankings?discipline=batting&limit=1 HTTP/1.1
```

```
{
  "filters": { "discipline": "batting", "competitions": ["ipl","smat","bbl","sa20","cpl","ilt20"], "phase": "all", "minBalls": 120, … },
  "pagination": { "total": 3435, "page": 1, "limit": 1, "totalPages": 3435 },
  "data": [{
    "player_key": 456, "cricsheet_id": "470f446b", "name": "V Suryavanshi", "competition": "ipl",
    "seasons": ["2025","2026"], "balls": 449, "tier": "rankable",
    "counts": { "runs": 1028, "boundaries": 177, "dots": 154, "dismissals": 23 },
    "metrics": {
      "strike_rate": { "value": 228.95, "league": 155.31, "index": 147.42,
                       "ci": [137.44, 157.39], "shrunk": 137.42, "percentile": 100, "population": 289 },
      …
    },
    "role": { "label": "opener", "inputs": { "innings": 23, "openShare": 1, "medianPosition": 2, … } },
    "teams": ["Rajasthan Royals"], "strengths": ["Strike rate 229.0 vs league 155.3 (100th pct of 289)", …]
  }]
}
```

```
GET /api/scout/rankings?minBalls=abc  →  400
{ "error": "invalid_parameters", "issues": [ { "code": "invalid_type", "path": ["minBalls"], … } ] }
```

---

5. Contracts between front end and back end

- An **OpenAPI 3.1 spec** exists (`lib/api-spec/openapi.yaml`, 19 paths, Orval config) and generates `lib/api-zod` and `lib/api-client-react`. **The web app imports neither** — it uses hand-written clients (`src/lib/scout-api.ts`, `sim-api.ts`, `discovery-api.ts`, `ask-api.ts`, `dugout-api.ts`, `aggregates.ts`, `live.ts`). Only `/healthz` uses a generated schema.
- Consequence observed in this audit: the Scout API returns `role` as an object while the UI treats it as a string → every row shows "Unclassified". A shared, generated type would have made this a compile error.

---

6. Performance summary (measured)

Class	Examples	Warm latency	Why
Pre-computed / cached	`/players/:id/summary`, `/war-room/board`, `/pressure/*`, `/insights/daily`, `/dugout/*`	2–14 ms	decision caches built at start-up
Light SQL over aggregates	`/scout/rankings`, `/scout/compare`	35–141 ms	`global.db` aggregates
Heavier profile	`/scout/player/:id`	552 ms	repeats `ratePlayers` per season for lines + trends + similar (N+1-style)
Simulation	`/opposition`	233 ms	Monte Carlo
Legacy Advanced Stats	`/stats/query?season=2025`	**2.3 s**; 10 parallel → **24 s**	multi-join SQL + JSON replay, synchronous
Static JSON passthrough	`/ipl/top-batters`	30 ms but **1.2 MB**	no pagination
Concurrency (Scout)	50 parallel `/scout/rankings`	1.9 s wall	event loop serialises synchronous SQLite

---

7. Authentication and authorisation

None. Every endpoint is public, including state-changing ones (`/auction/*`, `/ipl/insights/go/:slug`, `/ipl/sync-csv`) and the photo-pipeline admin page in the front end. CORS allows any origin.

---

8. How to design, debug, test and extend an endpoint (worked recipe)

1. **Design:** write the Zod schema first (input contract). Decide the error codes (400/404/503). Decide the response shape and add it to the OpenAPI spec so a typed client is generated.
2. **Implement:** route handler stays thin: validate → call one service function → map result/null to status codes.
3. **Debug:** `curl -s "http://localhost:5055/api/…" | jq` ; read `pino` logs (`preview_logs` or the terminal); reproduce with the service function directly in a `tsx` script (as done in this audit for `ratePlayers`, `auctionBoard`).
4. **Test:** service-level `node:test` (pattern: `scripts/src/scout/tests/scout.test.ts`); add an HTTP-level test with `supertest` against `app` (not present in the repo today).
5. **Extend:** e.g. adding `GET /api/scout/leaders?metric=boundary_pct` = new Zod schema + call `ratePlayers` + sort by the metric's **direction** (the bug in §3.2 shows why direction must be explicit).

---

9. Security findings specific to the API

#	Finding	Severity	Evidence	Fix
S1	**Arbitrary server-file read**: `POST /api/ipl/sync-csv` takes `filePath` from the request body, checks `fs.existsSync(filePath)` and reads it; a missing file returns an error containing the path (existence oracle). No auth.	**High**	`routes/csv-sync-route.ts:6-36`, `services/cricsheet-csv-sync.ts:38-43`. *Not exploited during the audit.*	Delete the route, or accept only an id that maps to a whitelisted file under `ipl_csv/`.
S2	Shared mutable game state; public `/auction/reset`	Medium	`services/auction.ts:429`	per-session state keyed by a session id
S3	Cache bypass `?refresh=true` lets anyone trigger unlimited outbound scraping	Medium	`routes/live.ts`	remove for public callers; rate-limit
S4	No rate limiting; a few concurrent `/stats/query` calls block the server for tens of seconds	Medium (availability)	measured	rate-limit + move heavy queries to precomputed tables / worker
S5	`500 { error: err.message }` leaks internals	Low	`ipl.ts`, csv-sync	log details, return generic code
S6	Open CORS	Low (no cookies/auth)	`app.ts:28`	restrict to the site origin
S7	Gemini key in browser localStorage / query string (client-side)	Low–Medium	`lib/gemini-client.ts`	proxy through the server; never send keys from the browser

09 — AI Engineering Audit

> **Question answered:** *How much real AI engineering is in CricField?* **Short answer:** very little runtime AI, a lot of deterministic statistics and simulation, and a great deal of AI-*assisted* development. The "AI Analyst" section is almost entirely deterministic. The only runtime LLM calls are optional, ungrounded Gemini requests — one of which falls back to **hardcoded text containing invented statistics**.

---

1. Classification of everything labelled "AI" or "intelligence"

Feature (UI name)	What it actually is	Category	Evidence
Ask the IPL Database ("SQL Database" tab)	Phrase-matching parser → typed `StatQuery` → parameterised SQL → computed answer	**Deterministic NLP (rule-based) + SQL**	`services/decision/nl-parser.ts` header: "Deterministic and explainable … Nothing here touches SQL."
Gemini LLM tab (Ask page)	Browser `fetch` to Google Gemini 1.5 Flash with the user's own key; else hardcoded fallback	**LLM (optional) / mock**	`src/lib/gemini-client.ts`
Strategy Sim "Gemini briefing"	same client, `contextData = {context: "Live Strategy Simulation Dugout Briefing"}` — **no simulation numbers are passed**	LLM (optional) / mock	`pages/ai-analyst-strategy-sim.tsx:173`
Clip search / "Gully AI" clip finder	Server keyword matcher; Gemini JSON routing if `GEMINI_API_KEY` set	LLM (optional) + keyword search	`routes/clip-router.ts`
"Gully AI • Platform Guide" mascot	Scripted product tour and clip search UI	**Not AI**	`components/ai-mascot-guide.tsx`
`AIAnalystPanel` ("AI ANALYST · Online")	Canned answers via `if (k.includes("death"))`, 350 ms fake delay	**Mock**, and **dead code** (not imported)	`components/ai-analyst-panel.tsx:15-45`
AI Insight Feed	Deterministic scans ranking statistical surprises (z-scores, Poisson-style odds)	Statistics	`insight-feed.ts`
Hidden Weakness Detector	Two-sample z-tests with Bonferroni correction	Statistics	`weakness-detector.ts`
Pressure Genome	Context-relative run values, shrinkage, percentiles	Statistics	`pressure-genome.ts`
Strategy Sim / Alt Universe / Decision Replay / Opposition Map	Outcome-table Monte Carlo + exact DP win probability	**Simulation**	`match-model.ts`
Win probability calibration	2-parameter logistic (Platt) fitted by Newton's method	**Fitted statistical model** (tiny ML)	`match-model.ts:414-476`
War Room market price	k-nearest-neighbour regression on past sales (k = 15, distance-weighted)	**Non-parametric ML (kNN)**	`auction-war-room.ts:187-223`
Player Pool / similar players	z-scored feature vectors, Euclidean / $\exp(-d^2)$ similarity	Similarity search (unsupervised, no training)	`player-pool.ts`, `scout-service.ts:701-740`
Squad DNA	z-scores, correlations with win %, cosine similarity	Descriptive statistics	`squad-dna.ts`
Playing XI	Mixed-integer linear programming	**Optimisation (operations research)**	`playing-xi.ts`
Scout Lab ratings	league-relative indices, credibility shrinkage, Wilson CIs	Statistics	`scout-service.ts`

Honest summary for a CV: "deterministic analytics, statistical testing, Monte Carlo simulation, MILP optimisation, kNN pricing and a Platt-calibrated win model; an optional LLM integration." Do **not** describe the AI Analyst as an LLM agent, RAG system or ML platform.

---

2. Runtime LLM features in detail

2.1 Browser Gemini client (`src/lib/gemini-client.ts`)

Aspect	Implementation
Model / provider	`gemini-1.5-flash`, Google Generative Language REST v1beta. (Model availability is **unverified** for 2026; Google has retired older Gemini versions before — check before demoing.)
Key	`VITE_GEMINI_API_KEY` (would be **baked into the public bundle**) or `localStorage["ipl_gemini_api_key"]` entered by the user; sent as a **URL query parameter**
Prompt construction	Single user turn: a fixed `SYSTEM_INSTRUCTION` ("world-class cricket tactical analyst…") + `JSON.stringify(contextData)` + user text
Context / retrieval	**None.** No warehouse data is retrieved or passed. The Strategy Sim call passes only a label string.
Tools / function calling	None
Output validation	None — markdown text rendered as returned
Hallucination controls	None. The system prompt even asks for "calibrated numbers (e.g. over 18-20 economy …)" without supplying any.
Failure handling	Any error or missing key → `generateCalibratedFallback()`

Aspect	Implementation
Cost / latency	Paid per token on the user's key; latency shown (`ms`)
Evaluation	None

The fallback is the critical issue. `generateCalibratedFallback()` (`:118-…`) returns pre-written markdown chosen by keyword (`"death"`, `"kohli"` …) containing specific statistics — e.g. "A dot ball here increases win probability by +18.4%", "league-wide death strike rate against orthodox spin exceeds 188.4" — that are **not computed from any data**, and labels the result `model: "gemini-1.5-flash (Calibrated Quant Engine)"`, `source: "calibrated_model"`. A visitor without a key sees invented numbers presented as model output. This should be removed or replaced with an explicit "No AI key configured" message, or — better — with the deterministic `/api/ask` answer.

2.2 Server clip router (`routes/clip-router.ts`)

Aspect	Implementation
Prompt	Template with the **entire clip catalogue** inlined as JSON + user query (quotes escaped)
Generation config	`temperature: 0.1`, `responseMimeType: "application/json"`
Output validation	strips code fences, `JSON.parse`; returns `parsed.url` **without checking it exists in the catalogue** → a hallucinated URL would be passed to the user
Fallback	deterministic keyword/token matcher `matchClipLocally()` on any failure or missing key
Prompt-injection surface	user text is placed inside the prompt; the model could be told to output an arbitrary URL. Mitigation: accept only `clip_id`, look the URL up server-side.

2.3 Natural-language Ask (deterministic) — the strongest "AI-like" feature

`POST /api/ask` → `parseQuestion()` consumes recognised phrases ("since 2022" → `seasonFrom: 2022`; "death overs" → `phase: "death"`; player names resolved to IDs, ambiguities reported in `notes`) → `StatQuery` (strict Zod) → `runStatQuery()` compiles parameterised SQL, returns additive components, computes ratios in TypeScript, and writes a one-sentence answer. The UI shows the interpretation as removable "READ AS" chips; editing a chip calls `POST /api/ask/run`.

Why this is good design: the language layer can never generate SQL; a future LLM translator would only need to emit a `StatQuery` that passes the same Zod schema (`stats-engine.ts` header). This is the right architecture for **grounded** LLM features — the LLM becomes a *parser*, the warehouse remains the source of truth.

Verified live: "Best economy in the death overs since 2022" → Narine 6.06 (218 runs, 36 overs), 78 ms, matching an independent recount in `global.db`.

---

3. AI-assisted development vs your own work

The repository shows heavy AI coding assistance:

- Replit Agent-style commit messages and `.agents/`, `.local/secondary_skills/` folders (Replit agent skills), `replit.md`.
- `.claude/`, `.kilo/`, `.qodo/`, `.sixth/` tool folders in the outer directory.
- Project-memory notes describing AI sessions building "Decision OS slices" and the global engine.
- Code style: long explanatory header comments, consistent documentation voice across modules.

This handbook makes **no claim** about which lines you wrote. Chapter 26 separates *documented decisions* (scope choices, approvals, review gates visible in `docs/global-data-engine/*` such as "Step 3 presented before applying", "never describe economy as completely resolved") from *implementation*, and lists what you must be able to reproduce yourself.

---

4. What would make CricField's AI claims defensible

Goal	Minimal implementation
Grounded LLM Q&A	Server-side endpoint: LLM converts question → `StatQuery` JSON (function calling), validated by the existing Zod schema, executed by `runStatQuery`; LLM may only paraphrase returned numbers
No key in browser	Server proxy with rate limiting and a spending cap
Hallucination control	Answer must cite only numbers present in the query result; automated check that every number in the text appears in the result
Evaluation	A fixed set of 50–100 questions with expected `StatQuery` and expected answers; report exact-match and execution accuracy
Model output validation	Zod schema on LLM JSON; reject unknown clip ids / metrics

---

5. Interview questions on this chapter

1. *What exactly does the AI Analyst do?* — Deterministic statistics and simulations over the IPL warehouse; the "Ask" box is a rule-based parser into a typed query; there is an optional Gemini tab that is not grounded in the data.
2. *Is the Pressure Genome machine learning?* — No: context-relative averages, shrinkage and percentiles.
3. *What is the only trained component?* — The 2-parameter Platt calibration; the kNN price model "trains" only in the sense of storing past sales.
4. *How would you stop an LLM inventing numbers?* — Function-calling into a validated query schema; numbers only from results; output checks; evaluation set.
5. *Why is the Gemini fallback a problem?* — It presents fabricated statistics as model output; it undermines trust in the genuine analytics.

10 — Testing, Debugging and Quality

> No application code was modified to make anything pass. One test that opens `sqlite.db` without `readOnly` was run against a **copy** of the database to rule out any change to the production file.

---

1. What exists

Type	Exists?	Where	Count
Unit tests (metrics, roles, shrinkage, percentiles, style classification)	✅	`scripts/src/global/tests/metrics.test.ts`, `scripts/src/scout/tests/scout.test.ts`	14
Pipeline / integration tests (normalise → aggregate → validate on in-memory DB)	✅	`scripts/src/global/tests/pipeline.test.ts`	5
Warehouse sanity tests (real `global.db`)	✅	`scripts/src/scout/tests/api.test.ts` (service level, not HTTP)	4
Regression tests for site fixes (identity, economy recount, API snapshot, delivery sequencing)	✅	`scripts/src/step3/tests/*.test.ts`	15 + 3
Data validation as a gate	✅	`scripts/src/global/validate.ts` (118 checks)	—
HTTP/API contract tests (supertest etc.)	❌	—	0
React component tests	❌	—	0
End-to-end browser tests (Playwright/Cypress)	❌	—	0
Performance benchmarks	🟡	`scripts/src/global/bench.ts` (DB compaction/benchmark report); ad-hoc measurements in this audit	—
CI running any of the above	❌	no `.github/workflows`	—
Python tests	🟡	`scratch/test_*.py`, `scripts/test_*.py` are exploratory scripts (image segmentation, query probes), not a test suite	—

Test runner: Node's built-in `node:test` with `node:assert/strict`, executed through `tsx`.

---

2. Results of checks run on 2026-10-12

#	Command (cwd)	Result
1	`npx tsx --test src/global/tests/*.test.ts src/scout/tests/*.test.ts` (`scripts/`)	✅ **23 pass, 0 fail** (2.2 s)
2	delivery-sequence test, path pointed at a copy of `sqlite.db` + WAL	✅ **3 / 3**
3	`npx tsx --test src/step3/tests/identity.test.ts src/step3/tests/step3-regression.test.ts`	🔴 **11 pass, 4 fail** (64 s)
4	`npx tsc -p tsconfig.json --noEmit` (`artifacts/api-server`)	🔴 2 errors — `routes/scout.ts(123,12)` and `(123,29)`: `unique_name` not on `Rating`
5	same (`artifacts/ipl-analytics`)	🔴 11 errors, all in `components/cricket3d/*` (`BatterHandedness` not exported, tuple `.x/.y/.z`, missing `pause`, `toggleMute`, impossible enum comparison, unknown `WAGON_WHEEL` key)
6	same (`scripts`)	✅ 0 errors
7	`pnpm exec tsx src/index.ts` (the `decision-api` launch config)	🔴 crashes: `ReferenceError: __dirname is not defined in ES module scope` (`app.ts:46`)
8	`PORT=5055 node dist/index.mjs` + 37 endpoint probes	✅ all expected statuses (200 / 400 / 404); latency table in Chapter 8
9	Live UI checks (Scout, Ask, War Room, Dugout, Stats; desktop + 375 px)	🟡 works; defects listed in Chapters 6–7
10	`scout-service` and `auction-war-room` functions called directly (read-only)	✅ produced the worked examples in Chapter 6

2.1 The four failing Step-3 tests

Test	Failure	Likely cause (to confirm)
real data: pages that were wrong before Step 3.5 show exactly their own matches	assertion failure	identity index/shards regenerated after the 2026-10-11 data change
real data: links resolve by registry identity, not initial + surname	assertion failure	same
outputs that do not depend on runs conceded are unchanged from the recorded baseline	`FAIL: 25/31 checks` — e.g. "legacy /api/stats/query (2025): batters: 166 batters differ"; bowlers changed in `isIndian`, `team` (unexpected fields)	baseline `docs/global-data-engine/step3/baseline-economy-snapshot.json` predates the 2026-10-11 sequencing fix (`sqlite.db` changed) and later edits to `stats-aggregator.ts`; **not yet shown to be benign**

Test	Failure	Likely cause (to confirm)
player page: every page's recent matches are its own registry id's matches	identity audit reports **20 "MISSING (empty page)"** (e.g. `gurjapneet-singh` 8 matches not shown, `salil-arora` 14, `brijesh-sharma` 13)	bios lacking a verified key after the latest squad/bio regeneration

> **Release-blocker rule:** until each of these is either fixed or explained with a new, reviewed baseline, the site's stats cannot be called "regression-tested". Updating the baseline without explaining the diff would hide real changes.

---

3. Coverage gaps (what is not tested at all)

1. Any HTTP behaviour (status codes, validation messages, pagination, sort direction) — the Scout bowling-sort and `minBalls` bugs would have been caught by one API test each.
2. The legacy Advanced Stats engine (both SQL and JSON paths) — no fixtures, no expected outputs.
3. Decision modules: weakness detector, pressure genome, match model, War Room, Playing XI — no unit tests, only manual/browser checks recorded in docs.
4. Front end: no component, accessibility or visual tests; the `role` type drift is the kind of bug a contract test catches.
5. Security: no tests for path traversal, input limits or rate limits.
6. Statistical validity: no coverage test for intervals, no out-of-sample test for the weakness detector or calibration.

---

4. How previous bugs were diagnosed and fixed (repository evidence)

#	Symptom	Diagnosis	Fix	Evidence
B1	Player page crashed on every load	It loaded a **33.3 MB** JSON of all players' recent matches	Split into a 67 KB index + per-player shards (median 4.2 KB)	`docs/global-data-engine/step3/README.md` (before/after table)
B2	Bowling economy too high for many bowlers	`is_bye/is_legbye` never populated → old expression charged **7,142** bye/leg-bye runs to bowlers	`RUNS_CONCEDED_SQL` using only populated columns; regression vs raw recount	`services/decision/db.ts:77-89`; `economy-regression-results.md` (31/31 at the time)
B3	Residual economy mismatch	17 deliveries: multi-run no-balls and a 5-penalty wide can't be split	Documented, not fixed; migration script prepared	`economy-residuals.md`, `step3_5/migrate-delivery-extras.ts`
B4	Overs with ≥ 10 deliveries out of order	`parseFloat("19.10") = 19.1` → ball 1	`delivery_seq` / renumbered `ball_num` in data (2026-10-11)	backup vs current DB comparison; `delivery-sequence.test.ts`
B5	Wrong player's matches on a page ("Prince Choudhary" showed Mukul Choudhary)	initial + surname fallback	exact registry-identity resolution; audit script	`step3/tests/identity.test.ts`, `step3_5/identity-audit.md`
B6	Fake win-probability curve	`/live/win-probability` was a sine wave	replaced with calibrated chase model	`routes/live.ts:71-82`; project notes
B7	Every match showed the same 22 "squad" players and 2026-final commentary	`onlineDb.ts` returned invented payload	real team sheets + commentary from `match-card.ts`	`services/onlineDb.ts:1-10`
B8	Periodic fake delivery types	`types[(over*6+ball) % 12]`	ESPN commentary parser; nulls kept as nulls	`services/ball-commentary.ts:1-21`
B9	Simulations ~15 runs low; finals looked unwinnable	raw DP under-confident; no season boundary factor; DLS overs mishandled	Platt calibration, season factor, `effectiveOver`	`match-model.ts:414-476`; project notes
B10	A regex silently never matched	Python heredoc wrote a literal backspace (0x08) instead of `\b`	rewrite with an editor; scan for control characters	project memory "Avoid Python heredocs for regex edits"
B11	17 s query	`LEFT JOIN dismissals … AND x.player_out_id = d.batter_id` made SQLite probe the wrong index	join on `delivery_id` only, filter in `CASE` (1 s)	project notes
B12	Fabricated `lib/simulation-models.ts`	invented model outputs	file deleted; modules rebuilt on the warehouse	project notes

These are good **interview stories**: each has a symptom, a measurement, a root cause and a verification. Learn two or three of them so well you can explain them without notes (Challenge 12 asks you to reproduce B4).

---

5. Debugging toolkit used in this project (and how to use it yourself)

Problem	Tool / technique
Wrong number on a page	Reproduce the API call with `curl`; then call the service function directly in a `tsx` script; then run the SQL by hand with `node -e` + `node:sqlite` (`readOnly: true`)
Slow query	`EXPLAIN QUERY PLAN`; check which index is used; time with `performance.now()`
Data mismatch between two sources	Recount from raw files (the Step 3 economy recount); write the diff as a report
Identity errors	Audit every page against ground truth (registry ids from raw files) instead of spot-checking
Front-end data shape bugs	Inspect the JSON in the network panel; compare with the TypeScript interface the component uses

Problem	Tool / technique
"Works in my tab but not in the screenshot"	`document.hidden` and animation state (project note on framer-motion)
Fixes that must not touch production	copy the DB / source into a scratch folder, patch there, diff (`git diff --no-index`), `git apply --check`

---

6. Release blockers (as of this audit)

#	Blocker	Chapter
R1	`/api/ipl/sync-csv` arbitrary file read	8 §9
R2	4 failing Step-3 regression tests (20 empty player pages, unexplained stats diffs)	§2.1
R3	Scout bowling default sort inverted; `minBalls` not applied; role "Unclassified"	6
R4	Gemini fallback presents invented statistics	9
R5	Typecheck failures (API 2, web 11) block `pnpm run build`	§2
R6	`global.db` and raw JSON not part of any deploy artefact	2 §6
R7	Source-run crash (`__dirname`) breaks the documented dev/test server	§2

---

7. A minimal test plan to add (proposal)

1. **API tests** with `supertest` against `app`: one per Scout endpoint (status, schema, sort direction, `minBalls` respected), Ask ("Most runs in IPL 2025" → Sai Sudharsan 759), War Room (shape + backtest fields).
2. **Golden-number tests** for the legacy engine: 10 fixed queries with numbers recounted from raw Cricsheet.
3. **Contract types**: generate the client from Zod/OpenAPI; delete hand-written duplicates.
4. **One Playwright smoke test** per primary journey (Home → Scout → profile; Stats filter; Ask).
5. **CI** (GitHub Actions): `pnpm install`, `typecheck`, `test:global`, `test:step3` (on a fixture DB), build.

11 — Performance, Scalability and Security

> **Framing:** this chapter evaluates CricField *as if* it were preparing for **10,000 daily users**. It does **not** claim that it supports that traffic today. Measurements are from one laptop on 2026-10-12 (Chapter 8 method); they are indicative, not production benchmarks. Proposals are kept separate from current behaviour (§6).

---

1. Back-of-envelope load model (assumption, not data)

10,000 daily users × ~6 API calls per visit ≈ 60,000 requests/day ≈ 0.7 req/s average. Cricket traffic is spiky: during an IPL evening assume 10× the average → **~7 req/s**, with bursts of tens of simultaneous requests after a wicket or a social post.

---

2. Measured performance

2.1 API latency (single user, warm)

Endpoint class	Warm	Capacity implication
Decision caches (`/players/:id/summary`, `/war-room/board`, `/pressure/*`, `/dugout/*`)	2–14 ms	fine
Scout rankings	35–141 ms	~10–25 req/s per process
Scout profile	552 ms	~2 req/s per process
Opposition sim	233 ms	~4 req/s
Advanced Stats filtered (`/stats/query?season=2025`)	**2.3 s**	**< 0.5 req/s per process**

2.2 Concurrency (synchronous SQLite on one event loop)

Test	Wall time
1 × Scout rankings	42 ms
10 × Scout rankings concurrently	351 ms
50 × Scout rankings concurrently	1,883 ms
1 × Advanced Stats (season filter)	2,664 ms
10 × Advanced Stats concurrently	**23,892 ms** — every other request (including `/healthz`) waits behind them

Conclusion: the first bottleneck at the target load is not the database size but the **synchronous query on Node's single thread**. Ten fans filtering Advanced Stats at once would freeze the whole API for ~24 s.

2.3 Start-up

Decision caches warm for **11.9 s** after `listen()`; requests arriving then queue. On autoscaling platforms each new instance pays this cost (cold starts).

2.4 Front-end weight

Asset	Size
JS bundle (single chunk)	4.11 MB (**864 KB gzip**, 669 KB brotli)
CSS	403 KB (45 KB gzip)
Largest images	`TITLE BANNER.png` 2.6 MB, `keshav-maharaj.png` 2.0 MB, `sam-curran.png` 1.8 MB, `hero-players.png` 1.7 MB, `advance stats title image` 1.3 MB (bundled)
`public/` images	255 PNG/JPG = 68.8 MB; 288 WebP = 14.0 MB
`loading="lazy"` usage	6 occurrences in pages/components
Largest JSON payloads	`/api/ipl/top-batters` 1.2 MB; `/api/stats/query` 0.3–0.5 MB; `public/data/player-recent-matches.json` 34 MB (no longer used by the page, still shipped)
Polling	auction state **every 1 s per open tab**; live scores 30 s; win probability 15 s

On a mid-range phone over 4G, ~1 MB of compressed JS plus multi-MB hero images means a slow first paint; the Advanced Stats hero image alone is larger than many whole pages.

2.5 Query-level

`EXPLAIN QUERY PLAN` (Chapter 4 §6): death-overs economy scans the whole bowler index (871 ms); pre-aggregated global queries take 1–6 ms. Duplicate index on `dismissals(player_out_id)`.

---

3. Caching posture

Layer	Today
Browser	React Query (static JSON ~1 h, scout 5 min). API responses carry `Cache-Control: no-store` — the browser and any CDN are **told never to cache**, even for 2008 scorecards that never change.
Server	in-memory: decision caches, raw match JSON (all 1,247 files pre-loaded), scraper cache 30 s (bypassable with `?refresh=true`)
CDN	none configured
Database	SQLite page cache only

---

4. Deployment, cost, logging and monitoring

Topic	Today	Risk at 10k/day
Hosting	Replit autoscale (configured)	each instance needs the data files; local SQLite per instance; in-memory state (auction, click counts) diverges between instances
Data shipping	`global.db` and raw JSON gitignored	Scout Lab and many features fail on a clean deploy
Cost	unknown (no billing data in repo); compute is dominated by Advanced Stats queries and 1 s auction polling	—
Logging	pino JSON to stdout; no retention	no way to investigate incidents after the fact
Error monitoring	none (no Sentry or similar); `main.tsx` suppresses a class of browser errors	silent failures
Uptime / alerts	none	—
Analytics	GA4 (consent-gated)	page views only; no product events

---

5. Security review

5.1 Findings

#	Finding	Severity	Evidence
S1	**Arbitrary file read / existence oracle** via `POST /api/ipl/sync-csv` (`filePath` from request body)	**High**	`routes/csv-sync-route.ts`, `services/cricsheet-csv-sync.ts:38-43`
S2	No authentication or authorisation anywhere; admin "photo pipeline" page public	Medium	`App.tsx` routes, no auth middleware
S3	Global mutable auction state; anyone can reset	Medium	`services/auction.ts:429`
S4	No rate limiting; heavy endpoints enable cheap denial of service	Medium	§2.2
S5	`?refresh=true` lets anyone force outbound scraping	Medium	`routes/live.ts`
S6	Scraping third-party sites with a spoofed browser User-Agent	Legal/ToS	`services/cricbuzz.ts:3-9`
S7	Gemini key: browser-side, stored in localStorage, sent in URL; `VITE_GEMINI_API_KEY` would ship in the bundle	Low–Medium	`lib/gemini-client.ts`
S8	Error messages returned to clients (`err.message`)	Low	`routes/ipl.ts`
S9	Open CORS (`cors()`)	Low (no credentials)	`app.ts:28`
S10	`.env` committed (public ids only)	Low	`git ls-files`
S11	`drizzle-kit push` on every merge against the production DB file	Medium (integrity)	`scripts/post-merge.sh`
S12	`maximum-scale=1` (accessibility, not security)	—	`index.html:5`

5.2 Things done well

- Parameterised SQL in all modern code; enum-guarded interpolation.
- Zod validation on modern routes with bounded numbers and string lengths (e.g. `q` ≤ 80 chars, `sims` ≤ 5000).
- Read-only DB connections for decision and scout layers.
- Consent banner before analytics/ads; `.env.example` explains that `VITE_` values are public.
- pnpm `minimumReleaseAge: 1440` and an allow-list for packages that run install scripts — supply-chain hygiene.
- ODC-BY attribution requirement documented (`docs/global-data-engine/README.md`).

5.3 Data licensing and attribution

Data	Licence position
Cricsheet ball-by-ball	ODC-BY 1.0 — usable with visible attribution. ✅ `/data-sources` credits Cricsheet and links the licence (`pages/data-sources.tsx:65`)
Auction/season facts compiled from Wikipedia	CC BY-SA 4.0 (stated on `/data-sources:80`) — **share-alike**: derived datasets you publish must carry the same licence and attribution
ESPN commentary text, Cricbuzz live data, official player photos, team logos, IPL marks, JioHotstar branding	**not licensed** for redistribution; the site also labels an ad slot "Official Analytics Partner" — remove claims that imply affiliation

---

6. Proposed improvements (not implemented)

Essential

1. Delete or lock down `/api/ipl/sync-csv`.
2. Fix Scout ranking correctness (sort direction, `minBalls`, role display) and the Gemini fallback.
3. Make API responses cacheable: `Cache-Control: public, max-age=…` for historical endpoints; put a CDN in front.
4. Move heavy SQL off the event loop: precompute Advanced Stats aggregates (like `global.db` does) **or** run queries in `worker_threads`; add a timeout.
5. Rate-limit by IP (e.g. `express-rate-limit`) and remove public cache bypass.
6. Ship data deliberately: build artefact = code + `sqlite.db` (checkpointed, no WAL) + `global.db` + raw JSON (or remove runtime dependence on raw JSON).
7. Fix typecheck errors; add CI that blocks merges on typecheck + tests.

Important

8. Route-level code splitting (`React.lazy`) and lazy-load three.js/Recharts; convert PNG heroes to WebP/AVIF with `srcset`; `loading="lazy"` everywhere below the fold.
9. Paginate `/ipl/top-batters` and similar; drop the 34 MB legacy JSON from `public/`.
10. Replace 1 s auction polling with server-sent events or WebSocket, scoped per room.
11. Error monitoring (Sentry) and structured log retention; an uptime check on `/api/healthz`.
12. Remove `drizzle-kit push` from post-merge; use explicit migrations.
13. Remove/qualify affiliation-implying copy; keep only licensed or self-made images.

Optional

14. Move the warehouses to DuckDB (analytics) or PostgreSQL (multi-instance) when concurrency or data size requires it.
15. Server-side LLM proxy with grounding (Chapter 9 §4) and a spend cap.
16. Load tests (k6/autocannon) in CI with latency budgets.

12 — Reverse-Engineering Walkthroughs (12 features)

> Each walkthrough follows the same path: **user action → UI component → API request → backend logic → database query → transformation → response → visualisation**, then *why it exists*, *what breaks without it*, *how to debug*, *how to extend*, *what an interviewer asks*. Code excerpts are short and quoted from the repository.

W1. Scout Lab leaderboard

1. **Action:** open `/scout`, choose *Bowling Rankings*, *Death Overs (16–20)*.
2. **UI:** `pages/scout-home.tsx` state (`discipline`, `phase`, `minBalls`, `sortBy="shrunk"`, `sortOrder="desc"`) → `useQuery({queryKey:["scout-rankings", …]})`.
3. **Request:** `lib/scout-api.ts fetchScoutRankings()` → `GET /api/scout/rankings?discipline=bowling&phase=death&minBalls=120&sortBy=shrunk&sortOrder=desc`.
4. **Backend:** `routes/scout.ts` validates with Zod, calls `ratePlayers()`.
5. **DB:** `SELECT player_key, season_label, … legal_balls balls, runs_conceded a, wickets b, dots c, fours+sixes d FROM agg_bowling WHERE competition_id=? AND phase=? AND season_label IN (…)`.
6. **Transform:** league rate per season → expected → index → shrink → Wilson/normal CI → per-competition percentile.
7. **Response:** paginated `Rating[]` (≈37 KB, 141 ms).
8. **Visual:** rows with tier badge, shrunk index, raw index, CI, percentile.

```
// routes/scout.ts:138-153 (abridged) — the defect
} else if (sortBy === "shrunk") {
  const primaryMetric = discipline === "batting" ? "strike_rate" : "economy";
  valA = a.metrics[primaryMetric]?.shrunk ?? null; …
}
// For economy, lower raw value is better, but shrunk index is already direction-oriented  ← false
return (valA - valB) * order;   // desc ⇒ highest (worst) economy first
```

- **Why:** fair cross-season, phase-aware ranking with uncertainty.
- **Breaks without:** shrinkage → 27-ball cameos top the table; league adjustment → 2026 batters (higher scoring era) dominate all-time lists.
- **Debug:** call `ratePlayers` directly, sort by `metrics.economy.shrunk`, compare with `percentile`; rank 1 having percentile 0 proves the inversion.
- **Extend:** add `better` direction to the sort (`metricDefs(discipline)`), filter `r.balls >= minBalls`.
- **Interview:** *"How did you verify your ranking was correct?"* — answer honestly: it wasn't, for bowling; describe how you found and would test it.

W2. Scout player profile

`/scout/player/:id` → `fetchScoutPlayer(id)` → `GET /api/scout/player/206` → `resolvePlayer()` (key → Cricsheet id → exact name; ambiguous → 404 with candidates) → `playerAttributes()` (style/hand/capped with source + confidence), `seasonLines()` (one `ratePlayers` per competition-season), `trends()`, `similarPlayers()` → 20 KB, **552 ms** (many repeated `ratePlayers` calls) → profile with provenance chips, season table, trend verdicts, similar players.

- **Breaks without exact identity:** two "Yuvraj Singh"s would merge.
- **Extend/optimise:** compute all season lines in one grouped query instead of N calls.
- **Interview:** *"How do you handle duplicate player names?"* → registry id; names never merge; ambiguous names return candidates.

W3. Ask the database

1. **Action:** type "Best economy in the death overs since 2022".
2. **UI:** `pages/ai-analyst-ask.tsx` → `lib/ask-api.ts` → `POST /api/ask {question}`.
3. **Parser** (`nl-parser.ts`): consumes phrases into typed fields.

```
// nl-parser.ts:364 and :398
} else if ((m = t.take(/\b(since|after|from) (20\d\d)\b/))) { … seasonFrom = 2022 }
else if (t.take(/\b(?:at the )?death(?: overs| bowling| phase)?\b|\bslog overs\b/)) f.phase = "death";
```

4. **Engine:** `runStatQuery()` (`stats-engine.ts:1140`) chooses `runBowling`, compiles parameterised SQL returning **additive components** (runs, legal balls, wickets), computes ratios in TS, applies an automatic minimum volume ("min 100 balls bowled").
5. **Response:** answer sentence + table + `query` object.
6. **Visual:** "READ AS" chips (since 2022 · Death overs · min 100 balls); 78 ms.

- **Why additive components?** So renamed franchises and venue variants can be merged *before* dividing (averaging averages is wrong).
- **Debug:** if a phrase isn't understood the API returns `understood:false` + suggestions; inspect the parsed `query` in the response.
- **Extend:** add a phrase → field rule plus a test; or let an LLM emit `StatQuery` JSON validated by `statQuerySchema`.

W4. Advanced Stats filtered leaderboard

`/stats` filter panel (zustand store) → `GET /api/stats/query?season=2025&phase=Death…` → `routes/ipl.ts` (`getStatsQuery` normalises labels; `isDefault` short-circuits to career JSON) → `aggregateQueryStats()` → **SQL path** (Drizzle CTEs over `deliveries` + joins) unless batting-hand or pressure filters require the **JSON replay path** (walk every ball of every raw match file) → batters/bowlers arrays with phase splits, delivery breakdowns, zones → 0.3–0.5 MB → tables and drill tabs.

- **Cost:** 1.7–2.3 s warm; 10 concurrent → 24 s.
- **Breaks:** if `lib/cricket-data/raw` is missing, stage/handedness/pressure filters silently return less.
- **Debug:** logs `stats/query incoming` with the effective filters and result counts (`ipl.ts:470-500`).
- **Extend:** add precomputed aggregate tables per (player, season, phase, opponent…) — the global engine's approach.
- **Interview:** *"Why two engines?"* — historical; the DB lacked per-ball context; consolidation is technical debt.

W5. Player page — recent matches

`/players/virat-kohli` → `lib/player-key.ts` maps slug → bio key → `usePlayerIdentityIndex()` fetches `/data/player-recent-matches/index.json` (67 KB) → if the key maps to a registry id, fetch `/data/player-recent-matches/<id>.json` (Kohli 86 KB) → render list; else **empty state**.

- **Why:** replaced a 33 MB download that crashed the page; exact identity prevents another player's matches appearing.
- **Debug:** `scripts/src/step3/identity-audit.ts` audits all 788 bios against ground truth.
- **Current issue:** 20 bios now resolve to empty although matches exist (Chapter 10).

W6. Match scorecard

`/match/1473469` → `useMatchScorecard(id)` (`lib/aggregates.ts:788`) → `GET /api/ipl/match/1473469/scorecard` → `getMatchScorecard()` (`services/match-card.ts`) reads `lib/cricket-data/raw/1473469.json` (cached) and builds innings, batting/bowling cards, extras, fall of wickets → 6.8 KB, 2 ms → scorecard view.

- **Why from raw JSON:** independent of warehouse bugs; exact Cricsheet record.
- **Breaks:** raw folder absent on a fresh deploy → 404 "no match data held for this id".

W7. Hidden Weakness Detector

`/ai-analyst/weakness?player=RG Sharma` → `fetchWeakness()` → `GET /api/weakness/51` → `scanPlayer()` → uses the in-memory **column store** (`ball-context.ts`) to bucket every ball by bowler type × over window → per-ball value `v = runs − W·out` → z-test per cell:

```
// weakness-detector.ts:72-77
function zScore(cell: Agg, rest: Agg): number {
  const m1 = cell.sumV / cell.n, m2 = rest.sumV / rest.n;
  const v1 = Math.max((cell.sumV2 - cell.n * m1 * m1) / (cell.n - 1), 1e-6);
  const v2 = Math.max((rest.sumV2 - rest.n * m2 * m2) / (rest.n - 1), 1e-6);
  return (m1 - m2) / Math.sqrt(v1 / cell.n + v2 / rest.n);
}
```

→ Bonferroni-adjusted confidence → weaknesses / strengths / watch list + evidence dismissals → heatmap (diverging colours) and evidence lists. 14 ms.

- **Interview:** *"What is the multiple-comparisons problem and how did you handle it?"*

W8. Auction War Room board

`/ai-analyst/auction-war-room?view=live&team=CSK` → `fetchBoard()` → `GET /api/war-room/board?view=live&team=CSK` → `auctionBoard()` → cached `buildBoard("live")` (impact, IAR, kNN market, value, signal) + `teamNeeds("CSK")` (Squad DNA gaps) + `fitFor()` per player → 94 KB, 3 ms (after a 6 s first build) → table with BUY/HOLD/AVOID/SCOUT chips, market vs value, max bid, fit score and reasons; a backtest tab.

- **Honesty requirement:** show `marketBasis` (Suryavanshi 0.1 = extrapolated) and the backtest (projection ρ = 0.17, n = 9).
- **Debug:** `auctionPlayer(key, view)` returns comparables behind a price.

W9. My Dugout franchise HQ

`/dugout/csk` → `resolveTeamCode` → `fetchFranchise({team:"CSK"})` → `GET /api/dugout/franchise?team=CSK` → `dugout-data.ts` builds season-by-season record, titles (from finals on record), head-to-head, latest captain → hero stats; squad via `GET /api/dugout/squad?team=CSK&season=2026` (season is **required**; without it → 400).

- **Design principle in code:** "Where a fact is not in those sources … the field is null and the UI says so rather than filling it in." (`dugout-data.ts:6-9`)

W10. Homepage live scores

`/home` → `useLiveIplMatches()` (`lib/live.ts:67`, refetch 30 s) → `GET /api/live/ipl/matches` → `getIplMatches()` scrapes cricbuzz.com (cached 30 s) → ticker cards. On failure: 502 `live_source_unavailable`; the homepage has **no error UI**, sections stay empty.

- **Risks:** scraping ToS; `?refresh=true` bypass.

W11. Playing XI optimiser

Playing XI drawer → `POST /api/optimizer/playing-xi {teamId, pitch, maxOverseas:4, …}` → Zod `playingXiSchema` → candidate impacts (impact model + pitch adjustment) → MILP:

```
// playing-xi.ts:246-252
if (c.locked) model.constraints[`lock_${c.playerId}`] = { equal: 1 };
model.binaries![v] = 1;
const res = solver.Solve(model);
if (!res.feasible) return null;
```

→ re-solve without each chosen player to get marginal value → XI with reasons. Hard constraints use exact metadata (overseas from squad files, keeper from stumpings) — never fuzzy names.

- **Interview:** *"Why integer programming instead of picking the top 11?"* — constraints (4 overseas, ≥1 keeper, 20 overs of bowling) interact.

W12. Team theming

`TeamSwitcher` → `setTeam("CSK")` in `TeamProvider` → localStorage + `root.style.setProperty("--team-accent", …)` (`team-context.tsx:289-292`) → every component using `var(--team-accent)` re-colours without React re-renders.

- **Breaks:** hard-coded hex colours (hundreds in `fan-arena.tsx`) ignore the theme.

Debugging drill: a wrong number, end to end

1. Note the page, filters and the number.
2. Open the network panel → copy the API URL → `curl` it.
3. Find the route (`grep -n "router.get(\"/stats/query" artifacts/api-server/src/routes/*.ts`).
4. Call the service function in a `tsx` script with the same input.
5. Re-derive the number with a hand-written SQL query on a **copy** of the DB (`readOnly: true`).
6. If the SQL differs from the service → logic bug. If the SQL matches but the raw Cricsheet recount differs → data bug (like B2/B4 in Chapter 10).

20 — UI Design Audit

> **Evidence:** live screenshots at 1440×900 and 375×812 (2026-10-12), computed contrast ratios (WCAG 2.x relative-luminance formula), and code-wide counts of design tokens/classes in `artifacts/ipl-analytics/src`. There are **no Figma files, style guides or design-token documents in the repository**; everything here is reverse-engineered from the implementation.

---

1. Visual identity (as implemented)

Element	What the code does	Evidence
Mood	Dark "broadcast / sci-fi console" look: near-black navy background, glassy cards, neon violet/cyan accents, large uppercase display type, motion	`index.css:5-30`, screenshots
Base palette	`--background hsl(228 60% 5%)`, `--card hsl(226 48% 9%)`, `--primary/--accent hsl(263 83% 58%)` (violet), `--muted-foreground hsl(215 20% 65%)`	`index.css`
Team palette	runtime `--team-accent`, `--team-secondary`, `--team-glow`, `--team-surface-tint` per franchise	`lib/team-context.tsx`, `lib/team-palette.ts`
Radius	`--radius: 16px`; in practice `rounded-xl` (588), `rounded-full` (429), `rounded-2xl` (316), plus arbitrary `rounded-[24px]/[28px]/[20px]`	class counts
Light mode	none (8 `dark:` variants in the whole app; theme is dark-only)	count
Brand names on screen	"IPL Decision Intelligence OS" (title/meta), "IPL 2026 Intelligence OS" (logo lock-up), "Beyond XI" (fan arena), "CricField Scout Lab", "Gully AI" (mascot)	`site-profile.ts`, `index.html`, `scout-home.tsx`

Assessment: the visual direction is distinctive and consistent in *mood*, and team theming is a genuinely good product idea. The identity is fragmented in *naming*: five product names, and "CricField" appears only inside Scout Lab. A reviewer cannot tell what the product is called.

---

2. Typography

Finding	Evidence	Impact
Declared body font **"Outfit" is never loaded** (no `@font-face` or Google Fonts link); the page falls back to the system UI font unless the visitor has Outfit installed	`index.css:43`; `index.html:14` loads Inter + JetBrains Mono only	Typography differs by device; Inter is downloaded but not applied to body text
Type scale is ad hoc: arbitrary pixel sizes dominate — `text-[10px]` ×785, `[9px]` ×475, `[11px]` ×345, `[8px]` ×236, `[7px]` ×62; standard steps (`text-sm`, `text-base`) are rare	class counts	No modular scale; hard to keep consistent
1,558 class usages set text at 7–10 px (785 + 475 + 236 + 62)	counts above; 69 such elements rendered on `/stats` at 375 px	Below comfortable reading size (~12 px minimum for UI labels; 16 px body); fails users with low vision
Heavy weights everywhere: `font-black` ×1,336, `font-bold` ×947	counts	Weight no longer signals hierarchy when almost everything is black/bold
`uppercase` ×1,327 with `tracking-widest` ×450	counts	Uppercase micro-labels are hard to scan; screen readers may spell some out
Display headings very large ("ADVANCED STATS." hero)	screenshot	Strong brand moment on desktop; **clipped at 375 px** ("AD / STA")

---

3. Colour and contrast (computed)

Text opacity on the two main surfaces (WCAG AA needs **4.5:1** for normal text, 3:1 for large text ≥ 18.66 px bold / 24 px):

Text style	Uses in code	On `--background`	On `#0B1020` cards	AA normal text?
`text-white`	—	19.95	18.93	✅
`text-white/80`	102	12.64	12.18	✅
`text-white/60`	212	7.30	7.19	✅
`text-white/50`	180	5.31	5.31	✅
`text-white/40`	**609**	**3.75**	**3.81**	❌
`text-white/30`	**200**	**2.57**	**2.65**	❌
`text-white/25`	22	2.12	2.20	❌
`text-white/20`	84	(lower)	(lower)	❌
`--muted-foreground`	—	7.76	—	✅
violet `#8B5CF6` on background	—	4.71	—	✅ (barely)

Roughly 900 class usages fall below AA, and most are the same tiny uppercase labels from §2 — the worst combination (small + low contrast). The shadcn token `--muted-foreground` passes comfortably; replacing `text-white/30–40` with it would fix most cases at once.

---

4. Spacing, grids and layout

Aspect	Observation
Shell	three-column frame: 56 px icon rail (left), content, collapsible "Intelligence Feed" rail (right)
Grids	Tailwind grids (`grid-cols-1 lg:grid-cols-2`, `sm:grid-cols-10`); breakpoint usage uneven: `player.tsx` 92 breakpoint classes, `dugout.tsx` 1, `home.tsx` 5
Density	Scout and War Room tables are information-dense (8–10 columns) — appropriate for analysts, overwhelming for fans
Hero sections	`/stats`, `/home`, `/dugout` open with full-viewport heroes; on `/stats` the first table is entirely below the fold at 1440×900
Overlays	cookie banner + "Gully AI" mascot + quick-tour card + right rail compete for the bottom-right corner; at 375 px they cover a large share of the screen
Empty regions	`/dugout/csk` shows a large empty band between tabs and content at 1440 px

---

5. Components and consistency

Area	Finding
Primitive library	~50 shadcn/Radix primitives in `components/ui/*` give accessible dialogs, menus, tabs, tooltips
Feature components	Many bespoke cards (`glass-card`, `kpi-card`, `hero-focus-card`, `fixture-ticket-card`, module-shell `Tile`) with overlapping roles
Decision modules	**Most consistent area**: shared `module-shell.tsx` (header, picker, segmented control, Demo/Error banners, diverging colour tokens)
Tokens vs hard-coding	Semantic Tailwind classes (`bg-primary`) don't resolve because the Tailwind v4 `@theme` mapping is missing, so pages hard-code hex values (`fan-arena.tsx` 223 hex literals; `player.tsx` 64; `stats.tsx` 73)
Icons	lucide-react + react-icons; nav uses **icons without visible labels** on desktop

---

6. Accessibility audit (heuristic, not a formal WCAG audit)

Check	Result	Evidence
Pinch-zoom	❌ disabled by `maximum-scale=1`	`index.html:5`
Contrast	❌ ~900 sub-AA class usages	§3
Text size	❌ 1,558 class usages ≤ 10 px	§2
Accessible names	❌ 8 of 32 buttons on `/home` have no name; most large pages have **0** `aria-*` attributes (`stats.tsx`, `player.tsx`, `home.tsx`, `fan-arena.tsx`)	in-browser query + code scan
Images	✅ the 9 images on `/home` all have `alt`	in-browser
Headings	`/home` has one `h1`	in-browser
Keyboard / focus	Radix primitives handle focus in dialogs/menus; custom clickable `div`s and icon buttons not verified	❔ needs a keyboard pass
Reduced motion	`index.css` contains a `prefers-reduced-motion` rule	grep
Colour-only meaning	BUY/HOLD/AVOID chips and diverging heatmaps rely on colour plus text labels (good)	War Room, Weakness pages

Formal status: no axe/Lighthouse report, no screen-reader test and no keyboard-only test exist. Do not claim the site is accessible.

---

7. What is genuinely strong

1. **Uncertainty is visible**: Scout rows show tier, balls, interval and percentile beside the score.
2. **Honest empty and demo states** in Decision modules ("showing a saved scan … start the api-server").
3. **Methodology and Data Sources pages** exist and credit Cricsheet.
4. **Team theming via CSS variables** — cheap, global, delightful.
5. Ask page "READ AS" chips make the system's interpretation inspectable and editable.

---

8. Priority fixes (design debt)

Priority	Fix
P0	Remove `maximum-scale=1`; raise all text to ≥ 12 px; replace `text-white/30–40` with `text-muted-foreground`
P0	Load the font you declare (or declare Inter)
P1	One product name; demote sub-brands to section names
P1	A type scale (e.g., 12/14/16/20/24/32/48) and spacing scale as tokens; add the Tailwind `@theme` mapping so semantic classes work
P1	Move primary tasks above the fold on `/stats`; collapse heroes on return visits
P2	Consolidate card components; label icon-only navigation; mobile layout for wide tables (card view)

21 — UX Engineering

> **No user research, usability testing, analytics funnels or task-completion data exist in the repository.** Everything below is an expert heuristic review (Nielsen's heuristics + task walk-throughs performed by the auditor on 2026-10-12). Treat findings as hypotheses to test with real users.

---

1. Information architecture

1.1 Top-level navigation (left rail, `components/layout.tsx:25-38`)

Home · My Dugout · Beyond XI · Scout Lab · 2027 Intel · Stats · Pulse · Scores · Intel Hub · Visuals · Auction · Insights · AI Analyst · Shop

Issue	Why it matters
14 peer destinations, icon-only on desktop	exceeds what users can scan; icons like `Cpu`, `Activity`, `Radio`, `Zap` don't communicate meaning
Overlapping concepts: *Stats* vs *Intel Hub* vs *Insights* vs *2027 Intel* vs *AI Analyst* vs *Visuals*	users can't predict where an answer lives
Two "auction" experiences: *Auction* (a game) and *AI Analyst → Auction War Room* (valuation tool)	same word, different jobs
Sub-brands (*Beyond XI*, *Gully AI*, *CricField Scout Lab*)	no single mental model of the product
Many legacy redirects (`/on-field/*`, `/product`, `/growth`, `/merch`)	good for links; signals IA churn

1.2 Implied audiences → suggested IA (proposal)

Audience	Primary jobs	Proposed home
Fan	follow a team, check a player, browse matches	**Teams · Players · Matches**
Analyst / fantasy player	filter stats, ask questions	**Stats** (Advanced Stats + Ask)
Scout / franchise analyst	shortlist, compare, value	**Scout** (Scout Lab + War Room + Player Pool)
Curious visitor	stories, visual explainers	**Stories** (Insights, Visuals, 2027 Intel)

Four to six top-level items, with games/shop moved to a secondary menu.

---

2. Key user flows (walked through in the running app)

Flow	Steps	Friction found
Find a player's numbers	Home → (no global search box in the top bar) → Stats or a player link → `/players/:slug`	No site-wide player search; slug conventions (`rg-sharma`, not `rohit-sharma`) cause "Player not found"
Filter Advanced Stats	`/stats` → scroll past full-screen hero → set filters → wait 2–3 s per change	Hero hides the task; long waits without progress context; many filter labels in 7–10 px uppercase
Ask a question	AI Analyst → Ask → type or click a suggestion	✅ Best flow on the site: suggestions, instant answer (78 ms), visible interpretation chips, editable
Scout a death bowler	`/scout` → Bowling → Death	❌ default order shows **most expensive** bowlers first; role column "Unclassified"; percentiles per league in a mixed table
Value an auction target	AI Analyst → Auction War Room → team filter	Rich explanation panels ("How prices and value are set", "Reading the board"); thin-evidence estimates (basis 0.1) not visually flagged enough
Follow my team	Team switcher → My Dugout	✅ re-themes the site; Dugout shows record/titles/captain; empty band in the middle of the page

---

3. Onboarding

- **Consent banner** on first visit (declines non-essential by choice; site works if declined).
- **"Gully AI" quick tour**: 5 scripted cards (Dugout, Advanced Stats, Auction War Room, "12 AI Tactical Decision Engines", Match Pulse) — `components/ai-mascot-guide.tsx`. Useful idea; the "AI" label and "12 AI engines" copy overstate what is AI (Chapter 9).
- No first-run personalisation beyond choosing a team; no explanation of what the indices (100 = league average) mean until you open Methodology.

---

4. Filtering and interaction patterns

Pattern	Where	Assessment
Filter chips + dropdowns	Scout, Stats, War Room	good discoverability; Stats has ~15 filters → cognitive load
URL-synchronised state (`readParams/writeParams`)	Decision modules (weakness, auction, ask)	✅ shareable, back-button friendly

Pattern	Where	Assessment
React-state-only filters	Scout Lab home	❔ filters appear not to be written to the URL → a shortlist view can't be shared/bookmarked
Compare tray (≤ 5 players)	Scout	✅
Editable interpretation chips	Ask	✅ excellent pattern for trustworthy NL interfaces
Seeded simulations	Strategy Sim, Alt Universe	✅ reproducible results

---

5. System status: loading, errors, empty states

Page group	Loading	Error	Empty	Evidence
AI Analyst modules	✅ spinners	✅ `ErrorBanner` + labelled `DemoBanner` fallback	✅	5–15 error markers per page
Scout Lab	✅	✅	🟡	4–5 markers
Advanced Stats	✅	🟡 1 marker	✅ 15 empty messages	
Home, Dugout, Fan Arena, Feed	🟡	❌ none	❌	sections silently empty when live data fails
Player page	✅	🟡	✅ verified-identity empty state	

The best pattern in the codebase is the Decision modules' rule: *"Can't reach the API … showing a saved scan of RG Sharma's career. Start the api-server to scan any player."* — it tells the user what happened, what they are seeing, and what to do.

---

6. Mobile usability (375 × 812, audit browser)

Observation	Severity
Left icon rail (~56 px) + right rail take a large share of a 375 px screen; content column narrow (verify on a real phone — framer-motion measurement caveat in Chapter 7)	High
Cookie banner, mascot, tour card and rail toggle overlap the bottom of the viewport	High
Advanced Stats hero title clipped ("AD / STA")	Medium
Scout table overflows horizontally inside its container	Medium (acceptable with a scroll hint; better as stacked cards)
Pinch-zoom disabled	High (accessibility)
No horizontal page scroll (document width = 375)	✅

---

7. Heuristic scorecard (auditor judgement, 1–5)

Heuristic	Score	Note
Visibility of system status	3	strong in Decision modules, weak on Home/Dugout
Match with the real world (cricket language)	4	phases, economy, strike rate used correctly; indices need explanation
User control & freedom	4	editable chips, URL state in modules
Consistency & standards	2	naming, two auction features, inconsistent page shells
Error prevention	3	Zod on modern APIs; legacy filters accept anything
Recognition over recall	2	icon-only nav, jargon (IAR, shrunk index)
Flexibility & efficiency	3	power filters for analysts; no global search
Aesthetic & minimalist design	2	dense micro-labels, decorative heroes, overlays
Help users recover from errors	3	good banners in modules; "Player not found" dead end
Help & documentation	4	Methodology, Data Sources, in-module "how it works" panels

---

8. Task completion

No task-completion rates, time-on-task or success metrics exist. GA4 is integrated (page views only, consent-gated), so a funnel could be measured once events are added (Chapter 24 §6).

---

9. Highest-impact UX fixes (proposals)

1. Global player/team search in the top bar.
2. Fix Scout bowling sort, show roles, explain "100 = league average" inline.
3. Reduce primary nav to ≤ 6 items; one product name.
4. Collapse decorative heroes; put the first table above the fold.
5. Mobile: hide right rail by default, one overlay at a time, card layout for tables, enable zoom.
6. Add error states to Home/Dugout for live-data failures.
7. Write Scout filters to the URL so shortlists are shareable.

22 — Product Design Case Studies

> **Ground rules.** No user research, interviews, usability tests or A/B tests exist for CricField, and none are invented here. "Intended audience" is inferred from the product itself (copy, features, methodology text). "Evidence of iteration" cites only commits, documents or code comments that are actually in the repository.

Case study 1 — Scout Lab (global talent explorer)

Field	Content
User problem (inferred)	A franchise scout wants to find T20 players outside the IPL (SMAT, BBL, SA20, CPL, ILT20) who perform well *for their context*, without being fooled by tiny samples or high-scoring eras.
Intended audience	Franchise analysts and serious fans; copy says "Global Talent Explorer", "Deterministic cross-competition intelligence".
Key design choices	League-relative index (100 = league average); shrinkage with a visible "shrunk vs raw" pair; sample tier badges; 95% intervals on every row; role taxonomy that describes *usage*; compare tray; role catalogue page that explains every rule.
Implemented interactions	League chips, phase, role, min-volume, sort field and order, search, pagination, compare (≤ 5), profile drill-down, role catalogue.
Limitations	Default bowling order inverted; role column blank; unqualified players listed; per-league percentiles shown in a mixed table; CI shown beside the shrunk number although it belongs to the raw index; bowling type unknown for most non-IPL bowlers; filters not in the URL; uncommitted.
Evidence of iteration	Plan → build → review gates in `docs/global-data-engine/00-audit-and-plan.md`, `01-batch-1-report.md`, `02-scout-service-and-demos.md`; Step 4 "Scout Lab UI" was **on hold** in project notes on 2026-10-10 and now exists as uncommitted code (`pages/scout-*.tsx`). Two attribute-linking methods were tried and **rejected** (initials-form names linked the wrong Kumar; duplicated bio names gave conflicting styles) — `scout-service.md` §"Two link methods were tried and rejected".
Portfolio angle	"Designing trustworthy rankings: showing uncertainty without overwhelming the user." Needs: the bug fixes, a before/after of the row design, and user feedback from at least 3 analysts.

Case study 2 — My Dugout (franchise headquarters)

Field	Content
User problem (inferred)	A fan wants a home for *their* team: identity, history, current squad, stories.
Audience	Team-loyal fans.
Design choices	Team switcher re-themes the entire site via CSS variables; hero with titles, record, captain; squad views (carousel, collapsible grid, table); team stories; broadcaster links. Missing facts are shown as unknown rather than invented (`dugout-data.ts:6-9`).
Interactions	Choose team, season selector, squad view toggle, tabs (Squad, Intelligence, Articles, History, Videos).
Limitations	Large empty band on `/dugout/csk` at 1440 px; no error state if APIs fail; photos scraped from official sites (licensing).
Evidence of iteration	Commits `fe309d0` "redesign Dugout page, map with individual team profiles…", `bae3a9c` "redesign squad placing with compact carousel, collapsible grid, table view…", `9194613` "resolve photo mapping with multi-alias index…" (all 2026-10-09); photo pipeline scripts and validation report.

Case study 3 — Auction War Room

Field	Content
User problem (inferred)	Before an auction, which players are worth more than the market will charge, and how much should we bid?
Audience	Franchise strategists; auction enthusiasts.
Design choices	Separate **market price** (what teams pay) from **output value** (what the player produces); explicit signals (BUY/HOLD/AVOID/SCOUT) with thresholds and a ₹1 Cr materiality rule; max bid rounded to bid increments; "fit" to a chosen franchise's weaknesses with written reasons; a **backtest view** against the real 2026 auction; explanatory panels ("How prices and value are set", "Reading the board").
Interactions	View toggle (next auction / 2026 backtest), team, role, overseas, max price, signal, search, sort (edge/fit/market/impact), player detail with comparables.
Limitations	Backtest shows weak predictive validity (projection ρ = 0.17, n = 9; AVOID group 5/5 delivered); 78% of the 2026 list is SCOUT (no IPL record); thin-evidence prices (basis 0.1) not prominent; price model sensitive to unrelated data changes (up to 39% shifts).
Evidence of iteration	Project notes: a log-linear price fit was "too flat" → replaced by comparable-sales kNN; value capped at the record fee; backtests must use as-of data to avoid leakage. The file header still describes ridge regression (stale). Step 3 docs record the price shifts after the economy fix (risk 2).

Case study 4 — Advanced Stats

Field	Content
User problem (inferred)	Answer "who was best in situation X?" — by season, team, opponent, venue, phase, over range, bowler type, delivery type, innings, match stage, nationality.
Audience	Fantasy players, journalists, analysts ("Statsguru depth" copy).
Design choices	Many dimensions of slicing; batter/bowler leaderboards with phase splits; matchup and scenario drill-downs; commentary-derived delivery types replacing earlier invented ones.
Interactions	~15 filters (zustand store), drill tabs, matchup view, CSV export.

Field	Content
Limitations	2–3 s per query; full-screen hero hides the tool; 4,411-line component; tiny labels; dead `FormGuideView` with random numbers; legacy endpoint unvalidated.
Evidence of iteration	`services/ball-commentary.ts` header documents replacing periodic fake delivery types with commentary-derived data; Step 3 economy fix changed the bowling table (Narine 6.79, Bumrah 7.34, Chahal 8.05 career economies quoted in `step3/README.md`).

Case study 5 — AI Analyst: Ask the database

Field	Content
User problem (inferred)	Ask a cricket question in plain English and get a correct, sourced answer quickly.
Audience	Everyone; especially non-SQL users.
Design choices	Suggestions as starting points; deterministic parser; **interpretation shown as removable chips** ("READ AS"); automatic minimum-volume rule disclosed in the footer ("min 100 balls bowled"); timing shown; demo snapshot when offline, clearly labelled.
Interactions	Type/submit, click suggestion, remove/edit a chip, switch to Warehouse Summary or Gemini tab.
Limitations	Coverage limited to phrases the parser knows; the adjacent "Gemini LLM" tab returns invented text when no key is set; an ad labelled "Official Analytics Partner" sits under the answer.
Evidence of iteration	Header comments in `nl-parser.ts` and `stats-engine.ts` explain the decision to never let language generate SQL, leaving a validated boundary for a future LLM.

Cross-case lessons

1. The strongest designs in CricField share one idea: **show the system's reasoning** (chips, intervals, tiers, methodology panels, backtests).
2. The weakest moments come from **presentation drifting from the data contract** (role object vs string, mislabelled intervals, inverted sort).
3. None of these case studies yet has *user* evidence; adding even five moderated sessions per feature would transform them into portfolio-grade work (Chapter 27).

23 — Data Visualisation Design

> Chart inventory from code (`grep` of Recharts components) plus live inspection. Recharts usage: `LineChart` ×9, `BarChart` ×7, `RadarChart` ×5 (`Radar` ×7), `AreaChart` ×5, `PieChart` ×2, `ReferenceLine` ×1; plus custom SVG (heatmaps, sankey, knowledge graph, spatial/zone maps) and a three.js 3D stadium replay.

---

1. Inventory and fitness for purpose

Visual	Where	Encodes	Verdict
Ranked tables with inline numbers	Scout, Stats, War Room, Ask	the primary analytic output	✅ right choice for lookup tasks; needs units/legend
Diverging heatmap (bowler type × over window)	Weakness Detector	effect per 100 balls, blue↔red with grey midpoint	✅ good: diverging scale for a signed quantity, labelled legend (`DivergingLegend`)
War Room price curve (domestic vs overseas)	War Room "How prices and value are set"	market price vs impact above replacement	✅ shows the model's shape; should show the training points and spread
Radar charts	Scout compare, player archetype	multiple metrics per player	⚠️ radar distorts area, depends on axis order, hard to compare > 2 players; and here the domain is fixed to **[0, 100]** while the series include shrunk indices that can exceed 100 (e.g., 137) — values are clipped or mixed with percentiles (`scout-radar-chart.tsx:64,136`)
Line/area charts	season trends, win probability	time series	✅ appropriate
Pie charts	(2 uses)	shares	⚠️ replace with bars
Sankey, knowledge graph	`/visualizations`	flows/relationships	🟡 decorative unless tied to a question
3D stadium replay	match pages	ball trajectory	🟡 engaging; the adapter tags each ball `VERIFIED_TRACKING`, `SECTOR_RECORDED` or `EVENT_RECONSTRUCTION` (`lib/cricket3d/deliveryAdapter.ts:1-9`) — good provenance design — but the repository contains **no ball-tracking data**, so trajectories are sector-based or reconstructed; keep that label visible
Pulse "emotion" meters	`/pulse`	heuristic + `Math.random()`	❌ looks like measurement, includes noise

---

2. Communicating uncertainty — CricField's distinctive strength and its errors

Strength. Very few cricket sites show uncertainty at all. Scout Lab shows, per row: sample tier, balls, raw and shrunk index, 95% interval, percentile and population. Trends report "improving/declining" only when the interval of the difference excludes zero. The Weakness Detector separates "significant" findings from a "watch list". The War Room publishes its own backtest.

Errors found in the presentation layer:

Issue	Evidence	Fix
The CI tooltip always says "95% Wilson Score Confidence Interval" although SR/economy intervals are normal approximations	`scout-metric-badge.tsx:92`; used for `primaryMetric.ci` on `scout-home.tsx:564`	label by method
On Compare, dot % and boundary % intervals are rendered with a "%" unit, but the API returns them on the **index scale** (e.g., 79.4–111.6)	`scout-compare.tsx:313,327` (default `unit="%"`)	convert back to % or label as index
Headline (shrunk) and interval (raw) come from different estimators	Chapter 6 §1.5	show the raw value with its interval, or compute a posterior interval for the shrunk value
Percentiles per competition shown in a mixed-league table	live leaderboard	add a "league" column header note or compute a pooled percentile only when leagues are calibrated
Thin-evidence War Room prices (basis < 3) not visually distinct	War Room live board	hatch or grey the price; show "based on N comparable sales"

---

3. Analytical storytelling

Practice	Present?
Headline sentence generated from numbers (Ask answer, strengths/weaknesses, insight feed headlines with evidence bullets)	✅
"Method" disclosure next to results (weakness method string; War Room panels; Methodology page)	✅
Reproducibility (seeded simulations, deterministic daily feed)	✅
Comparison to a baseline (league average = 100; "elsewhere" in weakness cells)	✅
Narrative pages with charts tied to a question (Insights, 2027 Intel)	🟡 mostly editorial

---

4. Visual encoding hygiene

Principle	Status
Direction clarity (“lower is better” for economy, dot % batting)	metric definitions carry `better`, but the bowling sort ignores it
Units on every number	❌ indices, raw rates and percentages often unlabeled
Colour accessibility	diverging palette + text labels ✅; low-contrast labels ❌ (Chapter 20)
Small multiples over dual axes	mostly ✅
Truncated axes	radar fixed domain issue
Decoration vs data	hero art and motion frequently dominate data views

---

5. Recommendations

1. Replace radars with **dot plots** (one row per metric, players as coloured dots, shared 0–200 index axis with a line at 100).
2. Add an “index explainer” micro-legend: *100 = league average for the same league, season and phase*.
3. Draw intervals as **horizontal error bars** in the leaderboard instead of bracketed text.
4. Label every estimate’s evidence (balls, comparable sales) and grey out low-evidence values.
5. Remove random noise from Pulse or label it as a playful, non-statistical meter.

24 — Product Management

> **Separation of fact and proposal.** §1–§4 describe what the repository shows. §5–§7 are *proposals* (personas, KPIs, roadmap) and are labelled as such. **No product metrics (users, retention, engagement, revenue) exist in the repository.** GA4 is wired in, but no analytics data is stored or reported here.

---

1. Product vision (as expressed in the product)

Source	Statement
Home hero	"IPL is not just about cricket." "Data-driven insights across sports, business, fans and the IPL economy."
`index.html` meta	"IPL Decision Intelligence OS: Live Strategy Simulator, Alternate Universe Engine, Opposition Exploit Map, Captain's Decision Replay, Auction War Room, and 280,000+ ball-by-ball deliveries."
Scout Lab	"Deterministic cross-competition intelligence … normalized against match, season, and phase baselines."
Project notes (2026-10-10)	Planned first public features: **Global Player Search, Role-Based Comparisons, Franchise Scouting Shortlists**; data reliability before more UI.

Synthesis: an IPL-first analytics product that serves fans (Dugout, Beyond XI, Pulse) and aspires to serve professional decision-makers (Scout Lab, War Room, Decision OS). The tension between these two audiences is the central product question (§4).

---

2. Feature inventory by maturity

Tier	Features
Core & credible	Ask the database; Scout Lab ratings engine (after bug fixes); Weakness Detector; Pressure Genome; scorecards; player pages; global data engine
Promising, needs validation	War Room (weak backtest); Strategy Sim / Alt Universe / Decision Replay (calibration in-sample); Playing XI optimiser; Squad DNA
Engagement / content	Dugout, Beyond XI (fan arena), Pulse, Insights, 2027 Intel, Visualisations, auction game
Presentational / placeholder	Shop (no commerce), AI mascot claims, Gemini fallback, photo-pipeline admin (internal)

Breadth is very high for one builder: ~50 pages, 123 endpoints, 11 decision modules. **Depth and correctness lag breadth** (Chapters 6, 10).

---

3. Documented scope and prioritisation decisions (evidence)

Decision	Evidence	PM reading
Build the global engine as a **separate warehouse** rather than altering the live IPL DB	`00-audit-and-plan.md §3.1`, "Decisions needed" #2	risk isolation over convenience
Download scope phased (P1 six leagues first; P2 later)	"Decisions needed" #1; Batch 2 `enabled: false` in `sources.json`	incremental delivery
Storage options compared (zips, SQLite, Parquet/DuckDB, Postgres) with explicit "not now"	`00-audit-and-plan.md §3.2`	explicit trade-offs
Every site change "presented before applying"; patches + regression reports	`step3/README.md`, `*.patch`	change control
"Never describe the economy problem as completely resolved"	project notes; `economy-residuals.md`	honesty in communication
Scout Lab UI, franchise-fit, Batch 2 **on hold** pending data reliability	project notes 2026-10-10	sequencing data quality before features
Franchise recommendations **not exposed** "until squad eligibility & valuations verified"	`routes/scout.ts` header	responsible scope
Build on existing Express + SQLite rather than a suggested Next.js + FastAPI + DuckDB stack	project notes 2026-10-07	avoid a rewrite
Delete fabricated modules (`simulation-models.ts`), replace fake win probability and fake squads	Chapter 10 B6, B7, B12	quality over appearance

These are real product-management behaviours. They are documented largely in AI-session notes and plan documents; in an interview, present them as *decisions you made and why*, and be ready to explain each trade-off without notes.

---

4. Technical trade-offs that shaped the product

Trade-off	Choice	Cost
Speed of building vs correctness	many features quickly	correctness debt found in this audit
Static JSON vs API	static for fan pages (fast, cacheable)	stale until regenerated; 62 MB in `public/data`
Deterministic analytics vs LLM	deterministic core	the "AI" label over-promises
SQLite simplicity vs concurrency	SQLite	24 s under 10 concurrent heavy queries
Fan delight vs analyst clarity	visual-heavy, dense micro-labels	accessibility and comprehension costs

5. Personas (proposed — not researched)

Persona	Goal	Top tasks	Current fit
Fan "Aarav", 22	follow his team, settle debates	team page, player page, ask a question	Dugout + Ask good; nav confusing
Fantasy player "Neha", 28	pick a matchup edge before a deadline	venue/phase filters, batter vs bowler	Advanced Stats powerful but slow
Franchise analyst "Rahul", 34	build a shortlist of uncapped domestic talent	cross-league ranking, comparisons, evidence	Scout Lab closest; correctness bugs block trust
Journalist "Meera", 40	quick, citable numbers	Ask, records, methodology	Ask + Data Sources good

Validate with 5 interviews per persona before using them in a portfolio.

---

6. Success metrics (proposed KPIs — none measured today)

Area	KPI	How to instrument
Activation	% of new visitors who complete one analytical task (filter, ask, compare) in the first session	GA4 events on submit/compare
Engagement	weekly returning users; questions asked per user	GA4
Quality	% of Ask questions understood; error rate per endpoint; p95 latency	API logs
Trust	% of sessions that open Methodology/Data Sources; correction reports	events + contact form
Scout value (B2B)	shortlists exported; analyst-reported usefulness (1–5)	interviews, export events
Model validity	War Room projection ρ by season; calibration Brier score	backtest scripts

---

7. Roadmap (proposed)

Horizon	Theme	Items
Now (2–4 weeks)	Trust & safety	remove `/sync-csv`; fix Scout sort/minBalls/role; remove Gemini fallback numbers; fix failing tests and typecheck; commit work in logical commits
Next (1–2 months)	Focus	one product name; nav ≤ 6 items; global search; mobile pass; accessibility P0s; API caching; Advanced Stats aggregates
Later (quarter)	Validation	15 user interviews; War Room/weakness out-of-sample studies; league-strength model using players who moved between leagues; Batch 2 leagues
Not now		shop/commerce, real-time multiplayer auction, LLM chat without grounding

---

8. Release strategy (proposed)

1. Private beta of Scout Lab to 5–10 analysts/coaches with a feedback form.
2. Public "fan" release limited to Home, Dugout, Players, Matches, Ask — the most correct, least risky surfaces.
3. Every release gated by: typecheck, tests, the Step-3 regression suite (rebaselined with an explained diff), and a security checklist.

---

9. Product risks

Risk	Likelihood	Impact
Wrong numbers damage credibility (inverted rankings, fabricated fallback text)	High today	High
Licensing/trademark complaints (scraped photos, logos, "official partner" wording)	Medium	High
Live data source blocks scraping	Medium	Medium
Scope sprawl prevents depth	High	Medium
Performance collapse under traffic spikes	Medium	High

25 — Business and Growth Strategy

> **No traction, revenue, customers or partnerships exist in the repository, and none are claimed here.** AdSense and GA4 integrations exist in code (env-gated); whether they are live is **unverified**. Everything below is analysis and hypothesis.

---

1. Positioning

Segment	Offer CricField could make	Existing alternatives
B2C fans	team HQ, player pages, natural-language stats, explainers	ESPNcricinfo Statsguru, Cricbuzz, official IPL app, fantasy apps
B2C fantasy players	matchup/phase/venue filters, weakness scans	fantasy-tool subscriptions, Twitter analysts
B2B franchises / scouts	cross-league talent discovery with uncertainty; auction valuation	in-house analytics teams, CricViz, commercial data vendors
B2B media	embeddable "Ask" answers and insight cards	agency research desks

Differentiator (defensible): transparency — league-relative indices with sample sizes and intervals, published methodology, published backtests, identity rules that never merge players by name. **Not a differentiator:** visuals or "AI" branding.

---

2. Distribution hypotheses

Channel	Hypothesis	Cheapest test
Social (X/Instagram/LinkedIn)	daily Insight Feed cards travel during the IPL	post 20 cards; measure click-through to site
SEO	player/venue pages capture "X stats" searches	requires server-side rendering or prerendering; today the SPA is client-rendered
Newsletter/Substack	weekly scouting notes (a `substack/` folder exists with images)	4 issues, track subscribers
Direct B2B outreach	scouts value cross-league shortlists	10 conversations with coaches/analysts (state associations, T20 franchises)

---

3. Monetisation hypotheses (untested)

Model	Fit	Risk
Display ads (AdSense wired)	B2C volume needed; IPL seasonal	low CPMs; ad labelled "Official Analytics Partner" misleads
Freemium pro tier (export, saved shortlists, API)	analysts/fantasy	needs auth, billing, reliability
B2B scouting reports / licences	franchises, academies	data licensing, validation burden, sales cycle
Sponsored content	media	brand-safety and IPL marks

---

4. Licensing and legal risks (must resolve before any commercial use)

Asset	Status	Risk
Cricsheet data	ODC-BY — commercial use allowed with attribution	low ✅
Wikipedia-compiled auction/season facts	CC BY-SA 4.0 — share-alike	must license derived data the same way
ESPN commentary text	scraped; no licence	high
Cricbuzz live data	scraped with spoofed UA	high
Player photos from official/franchise sites	copyrighted	high
IPL, franchise and JioHotstar marks/logos; "Official Broadcast"/"Official Analytics Partner" wording	trademark / passing-off	high — the site does include a non-affiliation disclaimer in `broadcaster-registry.ts`, but other copy contradicts it
Player likeness for commercial products (Shop)	personality rights	high

---

5. Customer-discovery gaps

- No interviews with fans, fantasy players, scouts or franchise staff.
- No evidence that scouts want cross-league indices in this form, nor what price they would pay.
- No measure of which features fans use.
- No competitive teardown in the repository.

First discovery sprint (proposal): 5 fan interviews, 5 fantasy players, 5 cricket analysts/coaches; one landing page per segment with an email capture; decide segment focus from evidence.

---

6. IPL-first strategy

Strengths: Cricsheet's complete IPL history; seasonal attention spike; franchises' auction cycle creates a recurring decision moment. Weakness: seasonality (~2 months of peak traffic), crowded B2C market, legal exposure around IPL IP. The global engine (six leagues) is the credible bridge from IPL fandom to year-round scouting.

26 — Solo Product Builder Assessment

> **Purpose:** separate what the repository proves about *you* from what it proves about the *code*. You stated that you independently directed the product vision, feature planning, visual identity, UI/UX decisions, data analysis, implementation workflow and deployment, with AI coding assistance. This chapter accepts that as your account and maps it to evidence; it does **not** infer who typed which line.

---

1. The end-to-end responsibilities a product like this involves

Responsibility	Would normally belong to	Evidence it was exercised in CricField
Product vision & positioning	Founder / PM	hero copy, meta description, feature set, planned public features (notes)
Scope & prioritisation	PM	phased plans, "on hold" decisions, approval gates (Chapter 24 §3)
Information architecture & UX	Product designer	50 pages, nav, flows (with clear IA problems)
Visual identity & UI	Visual/UI designer	consistent dark broadcast aesthetic, team theming, redesign commits
Data sourcing & licensing	Data engineer + legal	Cricsheet ODC-BY attribution; but scraped sources without licences
Data modelling & ETL	Data engineer	two warehouses; global engine with validation
Statistics & modelling	Analyst / data scientist	league adjustment, shrinkage, intervals, z-tests, Monte Carlo, calibration, kNN, MILP
Backend/API	Backend engineer	123 endpoints, Zod contracts
Frontend engineering	Frontend engineer	React app, React Query data layer
QA & testing	QA engineer	41 automated tests (global/scout/step3), regression reports; no UI/API tests
Security	Security engineer	parameterised SQL; but a file-read route and no auth
DevOps & deployment	DevOps	Replit autoscale config, esbuild/Vite builds; no CI
Documentation	Tech writer	`docs/global-data-engine/*` — the best artefacts in the repo
Growth & monetisation	Growth / BD	AdSense/GA4 wiring, consent banner; no measured growth

---

2. Your documented decisions and contributions (evidence-based)

Decision / contribution	Evidence
Chose to build on the existing Express + SQLite stack instead of a Next.js + FastAPI + DuckDB rewrite	project notes 2026-10-07
Required site changes to be **presented before applying** (patches, isolated copies, regression results)	`step3/README.md`, `*.patch`, isolated-testing notes
Gated every commit and deploy personally	project notes ("the user gates every commit and deploy")
Insisted the economy issue never be called "completely resolved"	project notes; `economy-residuals.md`
Put Scout Lab UI, franchise-fit and Batch 2 on hold until data reliability improved	project notes 2026-10-10
Selected the competitions and phased data acquisition	`00-audit-and-plan.md` "Decisions needed"; `sources.json` enabled flags
Directed redesigns (Dugout, Beyond XI, squad views, photo registry)	commit messages 2026-10-09
Chose team-themed visual identity and fan/analyst dual audience	product itself
Commissioned photo sourcing and quality review (contact sheets, verification reports)	`public/contact-sheet-rcb-*.html`, `ipl-jersey-verification-report.md`

---

3. Work implemented with AI coding assistance

Signals in the repository: Replit Agent commits and `.agents/` / `.local/secondary_skills/` folders; `.claude`, `.kilo`, `.qodo`, `.sixth` tool folders; AI-session project notes describing modules being built ("Decision OS slices", "global data engine"), and very uniform explanatory code commentary. **Assume an interviewer will treat all implementation as AI-assisted unless you can reproduce it.**

---

4. Skills demonstrated by repository evidence vs. needing practical verification

Skill	Repository shows the *artifact*	You must still demonstrate
Product direction & scoping	✅ decisions and gates are documented	explain trade-offs live; write a one-page PRD unaided
Cricket domain modelling	✅ correct rules in `metrics.ts`	state and apply the rules (legal ball, conceded runs) on paper
SQL analytics	✅ many queries	write Q1–Q9 (Chapter 4) without help
Statistics	✅ shrinkage, Wilson, z-tests, calibration	derive the shrinkage formula; compute a Wilson interval by hand
Data pipeline design	✅ global engine	design an idempotent incremental loader on a whiteboard
React/TypeScript	✅ large app	build a small filterable table with React Query from scratch
API design	✅ Zod contracts	build a validated Express endpoint + test
UI/visual design	✅ distinctive look	produce Figma components, a type scale, an accessible palette

Skill	Repository shows the *artifact*	You must still demonstrate
UX research	❌ no evidence	run and synthesise 5 interviews
Testing discipline	🟡 data tests only	write API and component tests
Security	🟡	spot and fix the file-read route
DevOps	🟡	set up CI and a reproducible deploy

---

5. Work that would ordinarily belong to specialists (and where the gaps show)

Specialist area	Gap visible in this audit
QA	inverted bowling sort and role display shipped to the UI unnoticed
Security	unauthenticated file read; shared auction state
Accessibility	contrast, text size, zoom disabled
Legal/licensing	scraped media and affiliation-implying copy
Data engineering (legacy)	non-reproducible IPL warehouse

This is normal for a solo builder. The honest narrative is: *"I directed a broad product alone with AI assistance; I then audited it, found X, Y, Z, and here is how I fixed/would fix them."* That narrative is stronger than claiming everything works.

---

6. Self-assessment questions to answer in writing (no AI)

1. Which three decisions am I proudest of, and what alternatives did I reject?
2. Which bug in Chapter 10 can I explain from symptom to fix in two minutes?
3. Which module could I rebuild in a weekend with no AI? Which could I not?
4. What would I cut if I had to ship in two weeks?

27 — Design Portfolio Readiness

> A UI/UX or product-design portfolio case study is judged on **process evidence** (problem framing, research, iterations, testing, outcomes), not only on the final screens. The repository contains final screens (code) and some engineering process documents, but **no Figma files, wireframes, research notes, usability results, design-iteration artefacts or accessibility audits**.

---

1. Readiness by candidate case study

Candidate	Story potential	What exists	Missing artefacts	Readiness
Scout Lab — designing trustworthy rankings	High: uncertainty UX is unusual and interesting	live UI; methodology docs; rejected approaches documented	Figma of the row/leaderboard; 2–3 iterations; 5 analyst interviews; before/after of fixes (sort, role, CI labels); accessibility check	🟠 Low–Medium
Ask the database — explainable NL querying	High: "READ AS" chips are a strong pattern	live UI; parser design rationale	journey map; tests with 5 users on 10 questions (success rate); iteration on failure states	🟠 Low–Medium
My Dugout — team-personalised fan home	Medium–High	redesign commits; team theming	wireframes; mood board; fan interviews; mobile design; empty/error states	🟠 Low
Auction War Room — decision support under uncertainty	High for product/analytics roles	panels explaining method; backtest	user goals from franchise people; design of "evidence strength"; iteration on signals	🟠 Low–Medium
Beyond XI landing	Low (visual rebuild of a reference)	commit "pixel-accurate rebuild of … reference design"	original design rationale — and a reference-copy page is **not** original design work	🔴 Not suitable
Advanced Stats	Medium	complex filtering UI	task analysis; simplification iterations; performance UX	🔴 Low

---

2. Minimum artefact set per case study

1. Problem statement and target user (1 page).
2. Research: 5 interviews or a survey with ≥ 30 responses; synthesis board.
3. Information architecture / user flow diagram.
4. Low-fi wireframes (2 alternatives) → mid-fi → high-fi in **Figma**, with components and variants.
5. Usability test: 5 participants, task success, time, quotes.
6. Iteration: what changed and why.
7. Accessibility check: contrast, keyboard, screen reader spot-check.
8. Outcome: before/after metrics (even if only test metrics) and what you would do next.

---

3. Claims you can and cannot make today

Can say	Cannot say (yet)
"I set the visual direction and product scope for CricField and directed its implementation with AI assistance."	"I designed CricField in Figma."
"I designed an interface that shows sample size and confidence intervals next to rankings."	"Users found the rankings trustworthy."
"I ran an expert heuristic audit and an accessibility review of my own product."	"The site is accessible."
"I redesigned the Dugout squad views (carousel, grid, table)."	"The redesign improved engagement."

---

4. Fastest path to one portfolio-grade case study (4–6 weeks)

Pick **Scout Lab**. Week 1: interview 5 people who scout or coach. Week 2: Figma rebuild of the leaderboard + profile; fix sort/role/CI labels. Week 3: usability test 5 users on 5 tasks. Week 4: iterate, accessibility pass, write-up with before/after.

13 — Skill Inventory and Proficiency Audit

> **The rule this chapter enforces:** *code in the repository is not evidence of personal mastery.* Much of CricField was implemented with AI coding assistance. Every skill below is therefore rated twice:
>
> - **Repository evidence** — *Implemented in repository* (code exists and runs), *Partially demonstrated* (some evidence, with gaps), or *Not demonstrated*.
> - **My demonstrated proficiency** — at most *Requires oral/practical verification*: you only earn a stronger rating by explaining or reproducing the work without AI (Chapter 16 challenges, Chapter 15 questions).
>
> The same data is in `MASTER-SKILL-INVENTORY.csv`.

1. Full skill matrix (45 skills)

#	Skill	Evidence in CricField	Required knowledge	Interview difficulty	Repository evidence	My demonstrated proficiency
1	**SQL aggregation (GROUP BY, CASE, HAVING)** (Data, foundational)	Q1–Q9 in Ch.4; services/decision/stats-engine.ts; scout-service.ts	joins, grouping, conditional aggregation, NULL handling	Medium	Implemented in repository	Requires oral/practical verification
2	**Cricket scoring rules as data (legal balls, conceded runs, dismissals)** (Domain, foundational)	scripts/src/global/metrics.ts + tests; RUNS_CONCEDED_SQL (decision/db.ts:88)	wides/no-balls/byes rules, bowler wicket kinds, super overs	Medium	Implemented in repository	Requires oral/practical verification
3	**Relational modelling & keys** (Data, foundational)	sqlite.db 25 tables; global.db composite WITHOUT ROWID keys	PK/FK, grain, normalisation vs denormalisation	Medium	Implemented in repository	Requires oral/practical verification
4	**Indexing & EXPLAIN QUERY PLAN** (Data, advanced)	Ch.4 plans (871 ms scan vs 6 ms aggregate)	B-tree indexes, covering indexes, query planner	Hard	Implemented in repository	Requires oral/practical verification
5	**TypeScript fundamentals** (Engineering, foundational)	all packages; tsc --noEmit	types, unions, generics, narrowing	Medium	Implemented in repository	Requires oral/practical verification
6	**JavaScript async & event loop** (Engineering, foundational)	sync node:sqlite blocking (10 req → 24 s)	promises, single thread, blocking vs non-blocking	Medium	Implemented in repository	Requires oral/practical verification
7	**React components & hooks** (Frontend, foundational)	pages/*.tsx; useMemo/useState	state, effects, memoisation, keys	Medium	Implemented in repository	Requires oral/practical verification
8	**Server-state fetching (React Query)** (Frontend, foundational)	lib/aggregates.ts; scout-home.tsx:89-118	query keys, staleTime, loading/error	Medium	Implemented in repository	Requires oral/practical verification
9	**REST API design with Express** (Backend, foundational)	123 routes; routes/scout.ts	routing, middleware, status codes	Medium	Implemented in repository	Requires oral/practical verification
10	**Input validation (Zod)** (Backend, foundational)	rankingsQuerySchema, statQuerySchema	schemas, coercion, safeParse, error shapes	Easy	Implemented in repository	Requires oral/practical verification
11	**ETL design (idempotent, incremental)** (Data, advanced)	scripts/src/global/{acquire,normalize,sync}.ts	hashing, CRC, upserts, lineage	Hard	Implemented in repository	Requires oral/practical verification
12	**Data validation & quality checks** (Data, advanced)	validate.ts 118 checks; Step 3 recount	invariants, reconciliation, test oracles	Medium	Implemented in repository	Requires oral/practical verification
13	**Identity resolution (entity matching)** (Data, advanced)	global identity.ts; Step 3.5 identity audit	stable ids, review queues, false merges	Hard	Implemented in repository	Requires oral/practical verification
14	**Rates vs counts; correct aggregation** (Analytics, foundational)	stats-engine computes ratios after summing	why averaging averages is wrong	Medium	Implemented in repository	Requires oral/practical verification
15	**League/era adjustment (index = actual/expected)** (Analytics, advanced)	scout-service.ts:285-325	expected values, baselines	Medium	Implemented in repository	Requires oral/practical verification
16	**Shrinkage / credibility weighting** (Statistics, advanced)	shrink() k=120; impact model prior	normal-normal posterior mean, bias-variance	Hard	Implemented in repository	Requires oral/practical verification
17	**Confidence intervals (Wilson, normal approx)** (Statistics, advanced)	wilsonInterval(); SR/economy CI	binomial proportion CI, standard error	Hard	Implemented in repository	Requires oral/practical verification

#	Skill	Evidence in CricField	Required knowledge	Interview difficulty	Repository evidence	My demonstrated proficiency
18	**Percentiles & ranking with ties** (Statistics, foundational)	percentileRanks()	mid-rank, direction	Easy	Implemented in repository	Requires oral/practical verification
19	**Hypothesis testing & multiple comparisons** (Statistics, advanced)	weakness-detector.ts z-test + Bonferroni	z-test, p-values, family-wise error	Hard	Implemented in repository	Requires oral/practical verification
20	**Probability calibration (Platt) & simulation** (Statistics, advanced)	match-model.ts DP + Platt + Monte Carlo	logistic regression, Monte Carlo, seeds	Hard	Implemented in repository	Requires oral/practical verification
21	**Backtesting without look-ahead** (Analytics, advanced)	auction-war-room backtest (asOf)	time-based splits, leakage, Spearman	Hard	Implemented in repository	Requires oral/practical verification
22	**k-nearest-neighbour regression** (ML, advanced)	knnDetail() k=15	distance weighting, LOO evaluation	Medium	Implemented in repository	Requires oral/practical verification
23	**Optimisation (MILP)** (Analytics, advanced)	playing-xi.ts with javascript-lp-solver	objective, constraints, binaries	Hard	Implemented in repository	Requires oral/practical verification
24	**Similarity search (z-scores, distance)** (Analytics, advanced)	similarPlayers(); player-pool.ts	standardisation, Euclidean/cosine	Medium	Implemented in repository	Requires oral/practical verification
25	**Debugging data bugs from symptom to root cause** (Engineering, foundational)	Ch.10 B1–B12	hypothesis, measure, isolate, verify	Medium	Implemented in repository	Requires oral/practical verification
26	**Automated testing (node:test)** (Quality, foundational)	41 tests in scripts/src/**/tests	unit vs integration, fixtures	Medium	Partially demonstrated	Requires oral/practical verification
27	**API/HTTP and UI testing** (Quality, advanced)	none	supertest, Playwright, contract tests	Medium	Not demonstrated	Not demonstrated
28	**Web security basics (injection, path traversal, authZ)** (Security, foundational)	parameterised SQL; /sync-csv flaw	OWASP Top 10	Medium	Partially demonstrated	Requires oral/practical verification
29	**Performance measurement & optimisation** (Engineering, advanced)	Ch.8/11 measurements; Step 3 33 MB → shards	latency, payload, caching, bundling	Medium	Partially demonstrated	Requires oral/practical verification
30	**Git & change management** (Engineering, foundational)	19 commits, patches in docs; much uncommitted	branches, small commits, review	Easy	Partially demonstrated	Requires oral/practical verification
31	**Build tooling (Vite, esbuild, pnpm workspaces)** (DevOps, advanced)	build.mjs, vite.config.ts, pnpm-workspace.yaml	bundling, monorepos	Medium	Implemented in repository	Requires oral/practical verification
32	**Deployment & CI/CD** (DevOps, advanced)	.replit only; no CI	pipelines, environments, artefacts	Medium	Partially demonstrated	Requires oral/practical verification
33	**Tailwind/CSS layout & responsive design** (Frontend, foundational)	index.css; breakpoint classes	flex/grid, breakpoints	Easy	Partially demonstrated	Requires oral/practical verification
34	**Accessibility (WCAG)** (Design, advanced)	audit found contrast/size/zoom issues	contrast, semantics, keyboard, ARIA	Medium	Not demonstrated	Not demonstrated
35	**Visual/UI design & design systems** (Design, advanced)	distinctive theme; no tokens/Figma	type scale, colour, components	Medium	Partially demonstrated	Requires oral/practical verification
36	**UX research & usability testing** (Design, advanced)	none	interviews, synthesis, task tests	Medium	Not demonstrated	Not demonstrated
37	**Information architecture** (Design, foundational)	14-item nav, overlapping sections	card sorting, navigation design	Medium	Partially demonstrated	Requires oral/practical verification
38	**Data visualisation design** (Design, advanced)	heatmaps, CI display, radar misuse	encoding, uncertainty display	Medium	Partially demonstrated	Requires oral/practical verification
39	**Product scoping & prioritisation** (Product, foundational)	phased plans, holds, approval gates	trade-offs, MVP, roadmaps	Medium	Partially demonstrated	Requires oral/practical verification
40	**Product metrics & experimentation** (Product, advanced)	GA4 wiring only	funnels, KPIs, A/B tests	Medium	Not demonstrated	Not demonstrated
41	**Technical writing & documentation** (Product, foundational)	docs/global-data-engine/*	clarity, audience, evidence	Easy	Implemented in repository	Requires oral/practical verification

#	Skill	Evidence in CricField	Required knowledge	Interview difficulty	Repository evidence	My demonstrated proficiency
42	**LLM integration (prompting, grounding, evaluation)** (AI, advanced)	gemini-client.ts (ungrounded), clip router	function calling, validation, evals	Hard	Partially demonstrated	Requires oral/practical verification
43	**Python data tooling (Pandas, OpenCV)** (Data, foundational)	image tooling only	dataframes, image processing	Medium	Partially demonstrated	Requires oral/practical verification
44	**Web scraping & data licensing** (Data, foundational)	cricbuzz.ts, photo scripts; licence gaps	ToS, robots, licences	Easy	Partially demonstrated	Requires oral/practical verification
45	**Monitoring & logging** (DevOps, foundational)	pino logs only	structured logs, alerts, error tracking	Easy	Partially demonstrated	Requires oral/practical verification

2. The 30 most important skills to defend this project (priority order)

Foundational skills come first because every advanced claim rests on them. Learn in this order; each links to evidence you must be able to explain.

1. **SQL aggregation (GROUP BY, CASE, HAVING)** — foundational; evidence: Q1–Q9 in Ch.4; services/decision/stats-engine.ts; scout-service.ts
2. **Cricket scoring rules as data (legal balls, conceded runs, dismissals)** — foundational; evidence: scripts/src/global/metrics.ts + tests; RUNS_CONCEDED_SQL (decision/db.ts:88)
3. **Relational modelling & keys** — foundational; evidence: sqlite.db 25 tables; global.db composite WITHOUT ROWID keys
4. **TypeScript fundamentals** — foundational; evidence: all packages; tsc --noEmit
5. **React components & hooks** — foundational; evidence: pages/*.tsx; useMemo/useState
6. **REST API design with Express** — foundational; evidence: 123 routes; routes/scout.ts
7. **Data validation & quality checks** — advanced; evidence: validate.ts 118 checks; Step 3 recount
8. **ETL design (idempotent, incremental)** — advanced; evidence: scripts/src/global/{acquire,normalize,sync}.ts
9. **Indexing & EXPLAIN QUERY PLAN** — advanced; evidence: Ch.4 plans (871 ms scan vs 6 ms aggregate)
10. **JavaScript async & event loop** — foundational; evidence: sync node:sqlite blocking (10 req → 24 s)
11. **Server-state fetching (React Query)** — foundational; evidence: lib/aggregates.ts; scout-home.tsx:89-118
12. **Input validation (Zod)** — foundational; evidence: rankingsQuerySchema, statQuerySchema
13. **Identity resolution (entity matching)** — advanced; evidence: global identity.ts; Step 3.5 identity audit
14. **Rates vs counts; correct aggregation** — foundational; evidence: stats-engine computes ratios after summing
15. **League/era adjustment (index = actual/expected)** — advanced; evidence: scout-service.ts:285-325
16. **Shrinkage / credibility weighting** — advanced; evidence: shrink() k=120; impact model prior
17. **Confidence intervals (Wilson, normal approx)** — advanced; evidence: wilsonInterval(); SR/economy CI
18. **Hypothesis testing & multiple comparisons** — advanced; evidence: weakness-detector.ts z-test + Bonferroni
19. **Backtesting without look-ahead** — advanced; evidence: auction-war-room backtest (asOf)
20. **Debugging data bugs from symptom to root cause** — foundational; evidence: Ch.10 B1–B12
21. **Automated testing (node:test)** — foundational; evidence: 41 tests in scripts/src/**/tests
22. **Probability calibration (Platt) & simulation** — advanced; evidence: match-model.ts DP + Platt + Monte Carlo
23. **Web security basics (injection, path traversal, authZ)** — foundational; evidence: parameterised SQL; /sync-csv flaw
24. **Performance measurement & optimisation** — advanced; evidence: Ch.8/11 measurements; Step 3 33 MB → shards
25. **Git & change management** — foundational; evidence: 19 commits, patches in docs; much uncommitted
26. **Accessibility (WCAG)** — advanced; evidence: audit found contrast/size/zoom issues
27. **Visual/UI design & design systems** — advanced; evidence: distinctive theme; no tokens/Figma
28. **UX research & usability testing** — advanced; evidence: none
29. **Data visualisation design** — advanced; evidence: heatmaps, CI display, radar misuse
30. **Product scoping & prioritisation** — foundational; evidence: phased plans, holds, approval gates

3. Foundational vs advanced — what "defensible" means for each

Level	You can defend it when you can…
Foundational (SQL, TypeScript, React, Express, Zod, cricket rules, rates vs counts, Git, debugging)	build a small version from a blank file in under an hour and explain every line
Advanced (shrinkage, intervals, multiple testing, calibration, backtesting, ETL design, identity resolution, indexing)	derive or justify the method, state its assumptions and failure modes, and compute one example by hand

4. Skills not demonstrated at all

API/HTTP and UI testing · accessibility · UX research and usability testing · product metrics and experimentation. These are also the cheapest to add (Chapter 17).

14 — Job Role Mapping (the 16 roles you listed)

> **Project relevance is not job readiness.** A role is "relevant" when CricField contains work of that kind; you are "ready" only after you can reproduce and defend that work yourself (Chapters 15–17). Interview readiness below is an estimate from the evidence, pending your own verification. The complete 25-role matrix across all career families is in Chapter 28 and `CAREER-READINESS-MATRIX.csv`.

Summary

Priority	Roles
Apply now (after 2–4 weeks of verification work)	Sports Data Analyst · Cricket Performance Analyst · Data Analyst · Analytics Engineer · Technical Product Analyst · Data Engineer
Apply after one more independent project	Product Analyst · Business Analyst · Backend Developer · Full-Stack Developer · Software Engineer · APM
Longer path	AI Engineer · Applied AI Engineer · Sports Technology Researcher · Machine Learning Engineer

The strongest evidence in the repository is **data modelling, SQL analytics, statistical method and data-quality engineering applied to cricket**. The weakest is ML, LLM engineering, testing, accessibility and user research.

Role-by-role

Data Analyst

Item	Assessment
1. Relevance of CricField	High
2. Verified technical evidence	SQL over 295k/772k-ball warehouses; Ask engine; aggregates; validation reports
3. Skills still missing	BI dashboard (Power BI/Tableau), Excel, stakeholder-facing analysis memo
4. Interview topics	SQL, Excel/Sheets, a BI tool, statistics basics, communication — plus defending the CricField evidence above
5. Additional independent work	Write Q1–Q9 unaided; build one Power BI/Tableau dashboard from an exported CricField table; write a 1-page insight memo
6. Recommended priority	1 — apply now

Analytics Engineer

Item	Assessment
1. Relevance of CricField	High
2. Verified technical evidence	global engine: layered model, additive aggregates, metric definitions with tests, validation gate
3. Skills still missing	dbt or equivalent, warehouse (BigQuery/Snowflake/DuckDB), orchestration
4. Interview topics	SQL modelling, dbt-style layering, testing, documentation — plus defending the CricField evidence above
5. Additional independent work	Rebuild agg_batting in dbt+DuckDB with tests; document lineage
6. Recommended priority	1 — apply now

Product Analyst

Item	Assessment
1. Relevance of CricField	Medium
2. Verified technical evidence	feature breadth; GA4 wiring; KPI thinking (Ch.24)
3. Skills still missing	measured product metrics, experiments
4. Interview topics	SQL, funnels, A/B testing, product sense — plus defending the CricField evidence above
5. Additional independent work	Instrument 3 events in GA4; design an A/B test for Scout default sort; write the analysis plan
6. Recommended priority	2 — apply after 1 project

Business Analyst

Item	Assessment
1. Relevance of CricField	Medium
2. Verified technical evidence	requirements-like plan docs; trade-off tables
3. Skills still missing	stakeholder work, BRDs, process models
4. Interview topics	requirements, process mapping, SQL/Excel, communication — plus defending the CricField evidence above
5. Additional independent work	Write a BRD + user stories for Scout shortlists; a process map of the auction-prep workflow
6. Recommended priority	2

Sports Data Analyst

Item	Assessment
1. Relevance of CricField	Very high
2. Verified technical evidence	cricket metrics, phases, matchups, pressure, weakness scans, cross-league indices
3. Skills still missing	Python/R notebooks, video/tracking data exposure, published analysis
4. Interview topics	cricket knowledge, SQL/Python/R, stats, visual storytelling — plus defending the CricField evidence above
5. Additional independent work	Publish 3 analytical write-ups using CricField data (with caveats); one in Python/pandas
6. Recommended priority	1 — apply now

Cricket Performance Analyst

Item	Assessment
1. Relevance of CricField	High
2. Verified technical evidence	phase/matchup analysis, weakness detector, pressure genome, opposition map
3. Skills still missing	video-tagging workflow (e.g., Hudl/Dartfish), coach-facing reports
4. Interview topics	cricket tactics, video analysis tools, coach communication — plus defending the CricField evidence above
5. Additional independent work	Produce a 2-page opposition report for a real fixture; validate one weakness finding out-of-sample
6. Recommended priority	1 — apply now (state associations, academies, leagues)

Sports Technology Researcher

Item	Assessment
1. Relevance of CricField	Medium
2. Verified technical evidence	uncertainty-aware rankings, calibration, backtesting
3. Skills still missing	a formal study (hypothesis, data, evaluation)
4. Interview topics	research methods, statistics, writing — plus defending the CricField evidence above
5. Additional independent work	Write a short paper: does shrinkage improve next-season prediction across leagues?
6. Recommended priority	3

Data Engineer

Item	Assessment
1. Relevance of CricField	High
2. Verified technical evidence	incremental ETL with hashing, validation, lineage, idempotency
3. Skills still missing	Python pipelines, cloud, orchestration, CI
4. Interview topics	Python/SQL, orchestration (Airflow), cloud warehouses, Spark basics — plus defending the CricField evidence above
5. Additional independent work	Port global:acquire+normalize to Python; schedule with a cron/Airflow DAG; add CI
6. Recommended priority	1–2

Backend Developer

Item	Assessment
1. Relevance of CricField	Medium–High
2. Verified technical evidence	Express API, Zod contracts, services, perf measurements
3. Skills still missing	API tests, auth, caching, async DB, the security fixes
4. Interview topics	Node/Java/Go, REST, DBs, testing, security — plus defending the CricField evidence above
5. Additional independent work	Build a validated, tested, rate-limited API from scratch; fix /sync-csv and scout sort with tests
6. Recommended priority	2

Full-Stack Developer

Item	Assessment
1. Relevance of CricField	Medium–High
2. Verified technical evidence	React + Express + SQLite end to end
3. Skills still missing	component tests, CI/CD, auth, smaller clean codebase
4. Interview topics	frontend + backend + deployment — plus defending the CricField evidence above
5. Additional independent work	Build a mini-CricField (one page, one API, tests, CI) in a fresh repo without AI
6. Recommended priority	2

Software Engineer (generalist)

Item	Assessment
1. Relevance of CricField	Medium
2. Verified technical evidence	large TS codebase, debugging stories

Item	Assessment
3. Skills still missing	DSA practice, code review, tests
4. Interview topics	DSA, system design basics, code quality, testing — plus defending the CricField evidence above
5. Additional independent work	LeetCode-style practice (arrays, hashing, sorting) + 2 system-design write-ups using CricField
6. Recommended priority	2–3

AI Engineer

Item	Assessment
1. Relevance of CricField	Low–Medium
2. Verified technical evidence	deterministic NL→query design ready for an LLM; ungrounded Gemini client
3. Skills still missing	grounded LLM feature with evals
4. Interview topics	LLM APIs, function calling, RAG, evaluation, guardrails — plus defending the CricField evidence above
5. Additional independent work	Build server-side LLM→StatQuery with Zod validation and a 50-question eval set
6. Recommended priority	3

Applied AI Engineer

Item	Assessment
1. Relevance of CricField	Low–Medium
2. Verified technical evidence	same as above
3. Skills still missing	deployed, evaluated LLM feature
4. Interview topics	productionising LLM features, cost/latency control — plus defending the CricField evidence above
5. Additional independent work	Same as AI Engineer + cost/latency dashboard
6. Recommended priority	3

Machine Learning Engineer

Item	Assessment
1. Relevance of CricField	Low
2. Verified technical evidence	kNN pricing, Platt calibration, backtest
3. Skills still missing	a trained, evaluated model in Python; feature pipelines
4. Interview topics	ML theory, Python/sklearn/PyTorch, MLOps — plus defending the CricField evidence above
5. Additional independent work	Train a model predicting next-season impact; compare to shrinkage baseline with time-split CV
6. Recommended priority	4

Associate Product Manager

Item	Assessment
1. Relevance of CricField	Medium–High
2. Verified technical evidence	scope decisions, phased plans, approval gates, honesty constraints
3. Skills still missing	user research, metrics, PRDs, stakeholder evidence
4. Interview topics	product sense, prioritisation, metrics, communication — plus defending the CricField evidence above
5. Additional independent work	Write 2 PRDs (Scout shortlists, Ask v2) with metrics; run 10 user interviews
6. Recommended priority	2

Technical Product Analyst

Item	Assessment
1. Relevance of CricField	High
2. Verified technical evidence	deep technical + analytical artefacts; documentation
3. Skills still missing	product metrics
4. Interview topics	SQL, product analytics, technical communication — plus defending the CricField evidence above
5. Additional independent work	Instrument events; build a KPI dashboard; write a technical spec
6. Recommended priority	1–2

28 — Complete Career Role Matrix

> 25 entry-level paths across software engineering, data & analytics, AI/ML, UI/UX & product design, product management, business & growth, sports technology & scouting, and technical consulting / solutions engineering. **Portfolio readiness** = how much *presentable, verifiable* evidence exists today. **Interview readiness** = estimate, pending your verification through Chapters 15–16. Same data: `CAREER-READINESS-MATRIX.csv`.

Family	Role	Relevance	Evidence	Required skills	Not yet demonstrated	Exercises	Portfolio	Interview	Priority
Data & analytics	**Data Analyst**	High	SQL over 295k/772k-ball warehouses; Ask engine; aggregates; validation reports	SQL, Excel/Sheets, a BI tool, statistics basics, communication	BI dashboard (Power BI/Tableau), Excel, stakeholder-facing analysis memo	Write Q1–Q9 unaided; build one Power BI/Tableau dashboard from an exported CricField table; write a 1-page insight memo	Medium	Low–Medium (until SQL verified)	1 — apply now
Data & analytics	**Analytics Engineer**	High	global engine: layered model, additive aggregates, metric definitions with tests, validation gate	SQL modelling, dbt-style layering, testing, documentation	dbt or equivalent, warehouse (BigQuery/Snowflake/DuckDB), orchestration	Rebuild agg_batting in dbt+DuckDB with tests; document lineage	Medium	Low–Medium	1 — apply now
Data & analytics	**Product Analyst**	Medium	feature breadth; GA4 wiring; KPI thinking (Ch.24)	SQL, funnels, A/B testing, product sense	measured product metrics, experiments	Instrument 3 events in GA4; design an A/B test for Scout default sort; write the analysis plan	Low	Low	2 — apply after 1 project
Data & analytics	**Business Analyst**	Medium	requirements-like plan docs; trade-off tables	requirements, process mapping, SQL/Excel, communication	stakeholder work, BRDs, process models	Write a BRD + user stories for Scout shortlists; a process map of the auction-prep workflow	Low	Low–Medium	2
Sports	**Sports Data Analyst**	Very high	cricket metrics, phases, matchups, pressure, weakness scans, cross-league indices	cricket knowledge, SQL/Python/R, stats, visual storytelling	Python/R notebooks, video/tracking data exposure, published analysis	Publish 3 analytical write-ups using CricField data (with caveats); one in Python/pandas	Medium	Medium (domain strong once verified)	1 — apply now
Sports	**Cricket Performance Analyst**	High	phase/matchup analysis, weakness detector, pressure genome, opposition map	cricket tactics, video analysis tools, coach communication	video-tagging workflow (e.g., Hudl/Dartfish), coach-facing reports	Produce a 2-page opposition report for a real fixture; validate one weakness finding out-of-sample	Low–Medium	Medium	1 — apply now (state associations, academies, leagues)
Sports	**Sports Technology Researcher**	Medium	uncertainty-aware rankings, calibration, backtesting	research methods, statistics, writing	a formal study (hypothesis, data, evaluation)	Write a short paper: does shrinkage improve next-season prediction across leagues?	Low	Low	3
Sports	**Scouting / Recruitment Analyst**	High	Scout Lab, roles, War Room, identity	talent ID, cross-league adjustment, communication	validation of cross-league comparisons; scout interviews	Validate league-strength using players who moved leagues; build a shortlist with evidence	Low–Medium	Medium	1
Software engineering	**Data Engineer**	High	incremental ETL with hashing, validation, lineage, idempotency	Python/SQL, orchestration (Airflow), cloud warehouses, Spark basics	Python pipelines, cloud, orchestration, CI	Port global:acquire+normalize to Python; schedule with a cron/Airflow DAG; add CI	Medium	Low–Medium	1–2
Software engineering	**Backend Developer**	Medium–High	Express API, Zod contracts, services, perf measurements	Node/Java/Go, REST, DBs, testing, security	API tests, auth, caching, async DB, the security fixes	Build a validated, tested, rate-limited API from scratch; fix /sync-csv and scout sort with tests	Low–Medium	Low	2
Software engineering	**Full-Stack Developer**	Medium–High	React + Express + SQLite end to end	frontend + backend + deployment	component tests, CI/CD, auth, smaller clean codebase	Build a mini-CricField (one page, one API, tests, CI) in a fresh repo without AI	Low–Medium	Low	2
Software engineering	**Software Engineer (generalist)**	Medium	large TS codebase, debugging stories	DSA, system design basics, code quality, testing	DSA practice, code review, tests	LeetCode-style practice (arrays, hashing, sorting) + 2 system-design write-ups using CricField	Low	Low	2–3

Family	Role	Relevance	Evidence	Required skills	Not yet demonstrated	Exercises	Portfolio	Interview	Priority
Software engineering	**Frontend Developer**	Medium	React 19 app, React Query, Tailwind, three.js	React, accessibility, performance, testing	accessibility, tests, code splitting, design-system work	Refactor one 4,000-line page into tested components; fix contrast & zoom; lazy routes	Low	Low	3
AI & ML	**AI Engineer**	Low–Medium	deterministic NL→query design ready for an LLM; ungrounded Gemini client	LLM APIs, function calling, RAG, evaluation, guardrails	grounded LLM feature with evals	Build server-side LLM→StatQuery with Zod validation and a 50-question eval set	Low	Low	3
AI & ML	**Applied AI Engineer**	Low–Medium	same as above	productionising LLM features, cost/latency control	deployed, evaluated LLM feature	Same as AI Engineer + cost/latency dashboard	Low	Low	3
AI & ML	**Machine Learning Engineer**	Low	kNN pricing, Platt calibration, backtest	ML theory, Python/sklearn/PyTorch, MLOps	a trained, evaluated model in Python; feature pipelines	Train a model predicting next-season impact; compare to shrinkage baseline with time-split CV	Low	Low	4
AI & ML	**Data Scientist**	Medium	statistical methods, simulation, testing	statistics, Python/R, experimentation, modelling	Python/R modelling evidence, out-of-sample validation	Out-of-sample validation of the weakness detector and calibration (Brier score)	Low–Medium	Low–Medium	2–3
UI/UX & design	**UI Designer**	Medium	distinctive dark theme, team theming, component set	Figma, typography, colour, design systems, accessibility	Figma files, type/spacing scales, accessible palette	Recreate the Scout leaderboard and Dugout in Figma with components and variants; AA palette	Low	Low	3
UI/UX & design	**UX Designer**	Medium	flows and interaction patterns (chips, compare tray)	research, IA, wireframing, usability testing	research and testing artefacts	Run 5 usability tests on Scout; synthesise; iterate	Low	Low	3
UI/UX & design	**Product Designer**	Medium	end-to-end product direction	problem framing, research, interaction + visual design, metrics	full case study with research & outcomes	Complete the 4–6 week Scout Lab case study (Ch.27 §4)	Low	Low	3
Product	**Associate Product Manager**	Medium–High	scope decisions, phased plans, approval gates, honesty constraints	product sense, prioritisation, metrics, communication	user research, metrics, PRDs, stakeholder evidence	Write 2 PRDs (Scout shortlists, Ask v2) with metrics; run 10 user interviews	Low–Medium	Medium	2
Product	**Technical Product Analyst**	High	deep technical + analytical artefacts; documentation	SQL, product analytics, technical communication	product metrics	Instrument events; build a KPI dashboard; write a technical spec	Medium	Medium	1–2
Business & growth	**Growth Analyst / Marketer**	Low–Medium	SEO/social hypotheses; GA4/AdSense wiring	funnels, experimentation, content, SEO	any measured growth	Run a 4-week content experiment (Insight cards) and report CTR honestly	Low	Low	4
Consulting	**Technical / Analytics Consultant**	Medium	audit-style documentation, trade-off analysis	structured problem solving, slides, client communication	client-style deliverables	Turn Ch.11 into a 10-slide client recommendation deck	Low–Medium	Medium	2–3
Consulting	**Solutions Engineer / Sales Engineer**	Medium	can demo a technical product end to end; API docs	demos, APIs, customer problem framing	customer-facing demos, POCs	Record a 5-minute demo of Ask + Scout with honest limitations; write an API quick-start	Low–Medium	Medium	3

Reading the matrix

1. **Highest leverage:** Sports Data Analyst / Scouting Analyst / Data Analyst / Analytics Engineer — CricField is directly on-topic, and the missing pieces (a BI dashboard, a Python notebook, verified SQL) take weeks, not months.
2. **Design roles** need artefacts the repo cannot supply (Figma, research, testing). Don't apply with screenshots alone.
3. **AI roles:** the repository supports a *story* ("I designed the NL query boundary so an LLM can be grounded") but not a claim of AI engineering until you build and evaluate the grounded version.
4. **Consulting / solutions roles** reward the audit itself: your ability to explain trade-offs and limitations honestly is the product.

15 — Interview Defense Handbook (100 questions)

> **How to use this chapter.** Read the question, answer aloud *without* looking, then compare with the model answer. A model answer is only useful if you can produce its substance in your own words and point to the evidence. Exercises marked here have answer notes in Chapter 16's answer key where relevant. Same content: `INTERVIEW-QUESTIONS.csv`.
>
> **Answering principle for every CricField question:** *state what the code does → why → its limitation → what you would do next.* Interviewers reward the limitation more than the feature.

Worked answers to two exercises referenced below

- **B06 economy over** `1, wd(1), 4, 0, lb(1), 0, 2`: legal balls = 6 (the wide doesn't count; the leg-bye does). Runs conceded = 1 + 1 + 4 + 0 + 0 + 0 + 2 = 8 (leg-bye not charged). Economy = 6 × 8 / 6 = **8.00**.
- **I13 shrink** an index of 150 over 40 balls (k = 120): (40×150 + 120×100)/(160) = (6,000 + 12,000)/160 = **112.5**.

Beginner (30)

B01 · Overview — In two sentences, what is CricField?

Model answer. An IPL-first cricket analytics web app: a React front end and an Express API over two SQLite warehouses (IPL 2008–2026 and a six-league T20 warehouse). It offers stats exploration, player/team pages, deterministic decision tools (weakness scans, simulations, auction valuation) and a cross-league scouting view.

- **Evidence:** App.tsx routes; api-server routes; sqlite.db, global.db
- **Follow-ups:** Who is it for? What is the one feature you would demo first?
- **Common misconception:** Calling it an 'AI platform' — almost all of it is deterministic statistics.

B02 · Overview — What are the two databases and why are there two?

Model answer. sqlite.db is the IPL warehouse the site was built on; global.db is a separate, rebuildable Cricsheet warehouse for six leagues. It was kept separate so the new engine could never break the live IPL site, and because its schema follows Cricsheet more faithfully.

- **Evidence:** docs/global-data-engine/00-audit-and-plan.md §3.1
- **Follow-ups:** What is the cost of two copies of IPL data?
- **Common misconception:** 'Two databases is always bad' — it was a deliberate risk-isolation choice.

B03 · Cricket — What is a legal delivery?

Model answer. A delivery that is neither a wide nor a no-ball. Only legal deliveries count toward the six balls of an over and toward a bowler's balls bowled.

- **Evidence:** metrics.md 'Legal ball'; is_wide/is_noball columns
- **Follow-ups:** Is a no-ball a ball faced by the batter?
- **Common misconception:** Treating every row in deliveries as a legal ball.
- **Exercise:** Count legal balls in one over that contains a wide and a no-ball.

B04 · Cricket — Which deliveries count as balls faced by a batter?

Model answer. Every delivery except a wide. No-balls are faced.

- **Evidence:** update-ipl-data-v3.ts:705 `if (!isWide) balls += 1`
- **Follow-ups:** Why are wides excluded?
- **Common misconception:** Excluding no-balls from balls faced.

B05 · Cricket — How do you compute batting strike rate correctly?

Model answer. 100 × runs off the bat ÷ balls faced, where balls faced excludes wides. Extras never count as the batter's runs.

- **Evidence:** Q1 in Ch.4; v_batting view
- **Follow-ups:** What about super-over balls?
- **Common misconception:** Using total_runs (which includes extras) instead of batter_runs.
- **Exercise:** Compute SR from: 4 runs off 3 balls plus a wide.

B06 · Cricket — How do you compute bowling economy correctly?

Model answer. 6 × runs conceded ÷ legal balls. Runs conceded = bat runs + wide runs + no-ball penalty runs; byes, leg-byes and penalties are not charged.

- **Evidence:** RUNS_CONCEDED_SQL (decision/db.ts:88); metrics.md

- **Follow-ups:** Why does CricField's IPL warehouse get 17 deliveries slightly wrong?
- **Common misconception:** Dividing by all deliveries or charging byes to the bowler.
- **Exercise:** Over: 1, wd(1), 4, 0, lb(1), 0, 2 → economy?

B07 · Cricket — What are the three T20 phases in CricField?

Model answer. Powerplay overs 1–6, middle overs 7–15, death overs 16–20 (0-based over numbers 0–5, 6–14, 15–19).

- **Evidence:** metrics.md 'Phases'; phaseFromOver
- **Follow-ups:** How are 5-ball sets in The Hundred handled?
- **Common misconception:** Off-by-one errors from 0-based over numbers.

B08 · Cricket — Why are super overs excluded from career stats?

Model answer. They are tie-breakers, not part of the match proper; official records exclude them. In sqlite.db they are innings 3–6; in global.db they are flagged super_over = 1.

- **Evidence:** innings table counts (16+16+1+1)
- **Follow-ups:** How would including them bias a death-overs economy?

B09 · SQL — What does GROUP BY do in Q1 (career batting)?

Model answer. It collapses all of a player's deliveries into one row so SUM/COUNT aggregate per player.

- **Evidence:** Ch.4 Q1
- **Follow-ups:** Why COUNT(DISTINCT match_id)?
- **Common misconception:** Thinking GROUP BY filters rows (that is WHERE/HAVING).
- **Exercise:** Add a GROUP BY season to Q1.

B10 · SQL — What is the difference between WHERE and HAVING?

Model answer. WHERE filters rows before grouping; HAVING filters groups after aggregation (e.g., HAVING balls >= 200).

- **Evidence:** Ch.4 Q2
- **Follow-ups:** Can HAVING use a column alias in SQLite?

B11 · SQL — What is a primary key? Give one from CricField.

Model answer. A column (or set) that uniquely identifies a row. deliveries in global.db uses (match_id, innings_no, over_no, seq).

- **Evidence:** global schema.sql
- **Follow-ups:** Why is a composite key better than an auto-increment id here?

B12 · SQL — What is an index and why does deliveries have one on batter_id?

Model answer. A sorted lookup structure that lets the database find rows without scanning the whole table; it makes per-player queries fast.

- **Evidence:** idx_deliveries_batter; Q1 plan
- **Follow-ups:** What do indexes cost?
- **Common misconception:** Indexes always make everything faster (they slow writes, use space).

B13 · Web — What is the role of React in CricField?

Model answer. It renders the UI from state; each page is a component that fetches data via hooks and displays tables/charts.

- **Evidence:** pages/*.tsx
- **Follow-ups:** What is a hook?

B14 · Web — What is Express doing?

Model answer. It is the web server: matches URLs to route handlers, runs middleware (logging, CORS, JSON parsing) and returns JSON.

- **Evidence:** app.ts, routes/*
- **Follow-ups:** List the middleware in order.

B15 · Web — What is a REST endpoint? Give a CricField example.

Model answer. A URL + HTTP method that returns a resource representation, e.g. GET /api/scout/rankings returns a JSON page of player ratings.

- **Evidence:** Ch.8
- **Follow-ups:** Which endpoints change state?

B16 · Web — What is JSON and why is it used between front end and back end?

Model answer. A text format for structured data that JavaScript parses natively; both sides speak it.

- **Evidence:** every API response

B17 · Web — What does Vite do?

Model answer. Runs the development server with hot reload and builds the production bundle; it also proxies /api to the backend in development.

- **Evidence:** vite.config.ts
- **Follow-ups:** Why is the production bundle 4.1 MB?

B18 · Web — What is TypeScript and why use it?

Model answer. JavaScript with static types that catch mismatches at compile time.

- **Evidence:** tsconfig.base.json
- **Follow-ups:** Name a bug TypeScript did not catch in CricField.
- **Common misconception:** Types guarantee runtime correctness.

B19 · Data — Where does CricField's match data come from?

Model answer. Cricsheet ball-by-ball files (ODC-BY licence), plus enrichment from ESPN commentary, scraped live scores and compiled squad/auction files.

- **Evidence:** Ch.5 §1; /data-sources page
- **Follow-ups:** Which sources are licensed?

B20 · Data — What is ETL?

Model answer. Extract, Transform, Load: download raw files, convert them into tables/metrics, and load them into the warehouse.

- **Evidence:** scripts/src/global/*
- **Follow-ups:** Which step validates data in CricField?

B21 · Data — What is a Cricsheet registry id?

Model answer. An 8-character hex identifier per person, stable across matches, used to tell players apart even if names collide.

- **Evidence:** identity.md
- **Follow-ups:** Why not use names?

B22 · Stats — What does an index of 130 mean in Scout Lab?

Model answer. 30% above the league average for the same competition, season and phase, given the player's own balls.

- **Evidence:** Ch.6 §1.3
- **Follow-ups:** Is 130 in SMAT the same as 130 in the IPL?
- **Common misconception:** Reading it as a percentage of the best player.

B23 · Stats — What is a percentile?

Model answer. The share of the comparison population the player is better than (here 100 = best, mid-rank for ties).

- **Evidence:** percentileRanks()
- **Follow-ups:** Why compute it on the shrunk value?

B24 · Stats — Why does sample size matter?

Model answer. Small samples are noisy: a 190 strike rate over 27 balls tells you little. CricField shows balls and tiers and shrinks small samples.

- **Evidence:** Omarzai example Ch.6 §1.9

B25 · Product — Who is the target user?

Model answer. Two groups: IPL fans (Dugout, players, Ask) and analysts/scouts (Scout Lab, War Room). The tension between them shapes the UI.

- **Evidence:** Ch.24
- **Follow-ups:** Which would you pick if you had to choose?

B26 · Product — What is the AI Analyst?

Model answer. A set of deterministic analysis modules (stats Q&A, weakness scans, pressure profiles, simulations, auction valuation) plus an optional Gemini tab.

- **Evidence:** Ch.9 §1
- **Follow-ups:** Which parts are actually AI?
- **Common misconception:** Calling it an LLM agent.

B27 · Tools — What is pnpm and what is a workspace?

Model answer. A package manager; a workspace manages several packages (web, API, libs, scripts) in one repository with shared versions.

- **Evidence:** pnpm-workspace.yaml

B28 · Tools — What is Git used for and what is the state of the repo?

Model answer. Version control. The repo has 19 commits and ~274 uncommitted paths, so history doesn't yet show the recent work.

- **Evidence:** git log / status
- **Follow-ups:** How would you split the uncommitted work into commits?

B29 · Debugging — A page shows a wrong number. What is your first step?

Model answer. Reproduce the API call (network panel/curl) to see whether the number is wrong in the data or in the UI.

- **Evidence:** Ch.12 drill

B30 · Ethics — Which parts did you build with AI assistance?

Model answer. Be specific and honest: implementation was largely AI-assisted; I directed scope and design, set review gates, and verified results; here is what I can rebuild myself (name it).

- **Evidence:** Ch.26
- **Follow-ups:** What could you rebuild without Claude Code?
- **Common misconception:** Claiming sole authorship of all code.

Intermediate (40)

I01 · SQL — Why did you choose SQLite?

Model answer. One developer, read-mostly data (~300k and ~770k rows), no DB server to run on Replit, file is easy to copy and rebuild, and the global engine stays fully reproducible. The known ceiling is concurrency: the synchronous driver blocks Node's event loop.

- **Evidence:** 00-audit-and-plan.md §3.2 storage options table
- **Follow-ups:** When would you switch, and to what?
- **Common misconception:** 'SQLite can't handle production' — it can for read-mostly workloads with caching.

I02 · SQL — Why does the death-economy query take 871 ms while a global query takes 6 ms?

Model answer. The first scans the bowler index across all 295k raw deliveries and joins matches for the season; the second reads pre-aggregated rows keyed by competition/season/phase.

- **Evidence:** Ch.4 Q3 vs Q7 plans
- **Follow-ups:** How would you speed up Q3?
- **Common misconception:** Adding random indexes without checking the plan.
- **Exercise:** Run EXPLAIN QUERY PLAN on Q3 on a copy and propose one index.

I03 · SQL — How do you avoid SQL injection in CricField?

Model answer. Prepared statements with ? placeholders (node:sqlite) and Drizzle's sql tag, which binds values as parameters; the one interpolated fragment uses a Zod-validated enum.

- **Evidence:** scout-service.ts inList(); routes/scout.ts phase enum
- **Follow-ups:** Show the unsafe pattern in the ingest script and explain why it isn't exploitable.
- **Common misconception:** Escaping strings manually is enough.

I04 · SQL — Why compute ratios in TypeScript after summing, not in SQL per group?

Model answer. So groups can be merged first (renamed franchises, venue variants) — you add runs and balls, then divide. Averaging ratios gives the wrong answer.

- **Evidence:** stats-engine.ts header
- **Follow-ups:** Give a numeric example where averaging SRs is wrong.
- **Exercise:** Two seasons: 100 runs/50 balls and 10 runs/20 balls → correct SR?

I05 · SQL — What does WITHOUT ROWID do and why use it for deliveries?

Model answer. Stores the table clustered by its primary key instead of a hidden rowid, saving space and making PK range scans fast for composite keys.

- **Evidence:** global schema.sql

I06 · SQL — What is the WAL file next to sqlite.db?

Model answer. Write-ahead log: recent writes not yet checkpointed into the main file. Copying the DB without it can give stale data.

- **Evidence:** sqlite.db-wal 27 MB
- **Follow-ups:** When is it checkpointed?

I07 · Data — How do you handle duplicate player names?

Model answer. Join on the Cricsheet registry id; names never merge records; ambiguous names return all candidates and no player; unlinked records go to a review queue.

- **Evidence:** identity.md; resolvePlayer('Yuvraj Singh') → 3 candidates
- **Follow-ups:** What went wrong with initial+surname matching?
- **Common misconception:** Fuzzy matching is fine for display.

I08 · Data — Explain the ball-number wrap bug.

Model answer. The CSV ball label '19.10' was parsed as a float 19.1, giving ball 1; any over with 10+ deliveries restarted numbering. Fixed in data on 2026-10-11 with delivery_seq; still present in the ingest script.

- **Evidence:** update-ipl-data-v3.ts:659-661; backup vs current DB
- **Follow-ups:** How would you fix the parser?
- **Exercise:** Write a parser for '19.10' that returns over 19, ball 10.

I09 · Data — Why were bowler economies wrong before Step 3?

Model answer. is_bye/is_legbye were never populated, so the conceded-runs expression charged 7,142 bye/leg-bye runs to bowlers. The fix uses bat runs + wide runs + 1 per no-ball.

- **Evidence:** decision/db.ts:77-89
- **Follow-ups:** What residual remains?

I10 · Data — What makes the global ETL idempotent and incremental?

Model answer. Archives stored by content hash; matches re-ingested only if new or their zip entry CRC changed; aggregates recomputed per competition; rerunning gives the same result (27 s rebuild = incremental).

- **Evidence:** normalize.ts header; README
- **Follow-ups:** What happens when Cricsheet corrects a match?

I11 · Data — Name five validation checks the global engine runs.

Model answer. Match counts agree across catalogue/archive/README/DB; runs.total = batter + extras and extras = sum of types; legal balls per over ≤ 6 except recorded miscounts; win margins reconcile with totals; every delivery references known players.

- **Evidence:** validate.ts; validation-2026-10-10.md
- **Follow-ups:** What does a WARN mean?

I12 · Data — What is data lineage and how does CricField record it?

Model answer. Being able to trace a number to its source file and version. Global engine: acquisitions.jsonl (URL, time, SHA-256), source_files, meta timestamps, in_latest_source, change reports. The IPL warehouse lacks this.

- **Evidence:** Ch.5 §9

I13 · Stats — Why is Bayesian shrinkage useful?

Model answer. It balances a player's own evidence against the league average in proportion to sample size, so tiny samples don't top rankings and estimates are less noisy.

- **Evidence:** shrink() k=120
- **Follow-ups:** Is CricField's version fully Bayesian?
- **Common misconception:** Shrinkage 'penalises' small-sample players unfairly — it reduces expected error.
- **Exercise:** Shrink an index of 150 over 40 balls with k=120.

I14 · Stats — How do you distinguish confidence from sample size?

Model answer. Sample size is how much data; confidence (interval width) also depends on the variability of the quantity. Two players with 200 balls can have different interval widths for boundary % vs dismissal rate.

- **Evidence:** Abhishek: SR CI 116.8–145.1 vs balls/dismissal CI 62.8–171.7 on the same 275 balls
- **Common misconception:** Equating 'rankable' with 'certain'.

I15 · Stats — What kind of interval does Scout Lab report?

Model answer. Frequentist 95% intervals: Wilson score for proportions; a normal approximation using league per-ball variance for strike rate and economy. Not Bayesian credible intervals.

- **Evidence:** scout-service.ts:86-94, 415-440
- **Follow-ups:** Why use league variance?
- **Common misconception:** Calling them credible intervals.

I16 · Stats — Why the Wilson interval instead of p ± 1.96·SE?

Model answer. The Wald interval misbehaves for small n and proportions near 0 or 1 (it can go negative); Wilson stays in [0,1] and has better coverage.

- **Evidence:** wilsonInterval()
- **Exercise:** Compute the Wilson interval for 3 boundaries in 20 balls.

I17 · Stats — How do you verify cross-league comparisons?

Model answer. Today you can't claim equivalence: indices are within-league. Verification would use players who moved between leagues (paired comparison) to estimate league-strength factors, then test them out of sample.

- **Evidence:** scout-service.md 'Known limitations'
- **Follow-ups:** What bias could movers introduce?
- **Common misconception:** Treating a 120 in SMAT as a 120 in the IPL.

I18 · Stats — What is the multiple-comparisons problem in the Weakness Detector?

Model answer. Testing 44 cells at 95% would produce ~2 false 'weaknesses' by chance. The detector multiplies p-values by the number of cells (Bonferroni) and requires a material effect size.

- **Evidence:** weakness-detector.ts
- **Follow-ups:** Why is Bonferroni conservative?

I19 · Stats — What does the impact model measure?

Model answer. Runs per match above an average IPL player: batting runs above league rate minus a wicket cost, bowling the mirror image; recency-weighted and shrunk toward a slightly negative prior.

- **Evidence:** impact-model.ts
- **Follow-ups:** Why is W ≈ ¼ of runs per wicket?

I20 · Stats — What is Platt scaling and why was it needed?

Model answer. A logistic recalibration p = σ(a + b·logit(raw)). The raw dynamic-programming win probability was under-confident (raw 15% chases won ~31%).

- **Evidence:** match-model.ts:414-476
- **Follow-ups:** How would you validate it properly?

I21 · Stats — How did the War Room perform in its backtest?

Model answer. On the 2026 auction: price rank correlation 0.72 on 28 matched sales, typical miss 1.6×; projection of performance ρ = 0.17 on only 9 players; all 5 AVOIDs who played delivered. It ranks prices but has not shown predictive value.

- **Evidence:** auctionBoard({view:'backtest'}) run 2026-10-12
- **Follow-ups:** What would you change?
- **Common misconception:** Claiming it finds undervalued players.

I22 · API — How does a React component retrieve data in CricField?

Model answer. A React Query hook (useQuery) with a key containing all inputs calls a fetch helper (e.g. fetchScoutRankings), caches the JSON, and exposes isLoading/isError/data to the component.

- **Evidence:** scout-home.tsx:89-118
- **Follow-ups:** What is staleTime?

I23 · API — What happens when an API fails?

Model answer. Modern routes return structured errors (400 invalid_request, 404 player_not_found, 503 warehouse_unavailable); React Query sets isError; decision pages show an ErrorBanner or a labelled demo snapshot; some legacy pages just render empty.

- **Evidence:** routes/decision.ts; module-shell.tsx
- **Follow-ups:** Which pages have no error UI?

I24 · API — Why is Zod used on the server?

Model answer. To validate and coerce untrusted input at the boundary (query strings are strings) and reject bad requests with clear issues; types are inferred from schemas.

- **Evidence:** rankingsQuerySchema
- **Follow-ups:** Which routes don't use it?

I25 · API — Walk through GET /api/scout/rankings.

Model answer. Middleware → warehouse guard → Zod parse → ratePlayers per competition → search → sort → paginate → strengths/weaknesses → JSON.

- **Evidence:** Ch.2 §4
- **Follow-ups:** Where are the two bugs?

I26 · API — Why is /api/stats/query slow?

Model answer. It runs multi-join SQL over raw deliveries (or replays every raw JSON file) for each filter combination, synchronously, with no caching; 2.3 s warm.

- **Evidence:** Ch.8 §6
- **Follow-ups:** Propose two fixes.

I27 · API — What does Cache-Control: no-store on every API response cost you?

Model answer. Browsers and CDNs can never reuse responses, even for immutable historical data, so every visit recomputes.

- **Evidence:** app.ts:32-38

I28 · Frontend — Why is every Scout role 'Unclassified'?

Model answer. The API returns role as {label, inputs} but the client type and RolePill expect a string, so the pill receives an object and falls back.

- **Evidence:** scout-api.ts:23; scout-metric-badge.tsx:99
- **Follow-ups:** How do you prevent this class of bug?
- **Exercise:** Write the one-line fix.

I29 · Frontend — How does team theming work?

Model answer. TeamProvider writes CSS custom properties (--team-accent etc.) on the root element and stores the choice in localStorage; components use var(--team-accent).

- **Evidence:** team-context.tsx:255-298

I30 · Frontend — Why is the JavaScript bundle 4.1 MB?

Model answer. All ~50 pages, three.js, Recharts and Radix are imported statically into one chunk; no React.lazy.

- **Evidence:** build output
- **Follow-ups:** How would you split it?

I31 · Frontend — How did the player page go from crashing to fast?

Model answer. It used to download a 33 MB file of every player's matches; now it loads a 67 KB identity index and one per-player shard (median 4 KB).

- **Evidence:** step3/README.md

I32 · Debugging — How would you debug the inverted bowling ranking?

Model answer. Call ratePlayers directly, sort by economy.shrunk desc, and see rank 1 has percentile 0; inspect the sort comparator; fix by applying each metric's 'better' direction; add an API test.

- **Evidence:** Ch.6 §1.10

I33 · Debugging — A filter returns nothing. How do you know if it's a data or code problem?

Model answer. Reproduce with curl; check the parsed filters in logs; run the equivalent SQL on a copy; if SQL returns rows, it's the code; if not, check the data (e.g., pressure_state is always 'Standard').

- **Evidence:** ipl.ts logs; Ch.4 §5.2

I34 · Architecture — What happens when 1,000 users query Scout Lab simultaneously?

Model answer. Each request runs synchronous SQL on one event loop; measured 50 concurrent rankings → 1.9 s wall time, so 1,000 would queue for tens of seconds and block every other endpoint. Fix: cache responses (they change only when the warehouse is rebuilt), precompute default views, add a CDN, run multiple instances, rate-limit.

- **Evidence:** Ch.11 §2.2
- **Follow-ups:** Which single change helps most?

I35 · Architecture — Why one monolithic server?

Model answer. Simplest to build and deploy for one developer with a read-mostly workload; services are still modular internally.

- **Evidence:** Ch.2 §2
- **Follow-ups:** When would you split it?

I36 · Security — Find a security vulnerability in CricField.

Model answer. POST /api/ipl/sync-csv reads any filePath from the request body: an unauthenticated file-existence oracle and read. Remove it or whitelist file ids.

- **Evidence:** csv-sync-route.ts; cricsheet-csv-sync.ts:38-43
- **Follow-ups:** What other risks exist?

I37 · Security — Is it safe to put a Gemini API key in VITE_GEMINI_API_KEY?

Model answer. No. VITE_ variables are compiled into the public JS bundle; anyone can read them. Keys belong on the server behind a proxy.

- **Evidence:** .env.example warning; gemini-client.ts

I38 · Product — What exactly does the AI Analyst do?

Model answer. Ask parses English into a typed query and runs SQL; Weakness/Pressure/Feed are statistical scans; Strategy/Alt Universe/Replay/Opposition are simulations; War Room is valuation; Squad DNA is descriptive stats; an optional Gemini tab is ungrounded.

- **Evidence:** Ch.9

I39 · Product — What would you cut if you had two weeks to launch?

Model answer. Keep Home, Players, Matches, Ask, Dugout; fix Scout and launch it as beta; cut Shop, auction game, Pulse emotions, Gemini tab.

- **Evidence:** Ch.24 §7

I40 · Testing — What tests exist and what would you add first?

Model answer. 41 node:test tests for metrics, pipeline, scouting rules and Step 3 regressions. First additions: HTTP tests for Scout (sort direction, minBalls) and golden-number tests for Advanced Stats.

- **Evidence:** Ch.10

Advanced (30)

A01 · Stats — Derive the shrinkage formula used in Scout Lab and state its assumptions.

Model answer. Normal–normal model: the player's true index θ has prior N(100, τ^2); the observed index has variance σ^2/n per ball. The posterior mean is $(n/\sigma^2 \cdot x + 1/\tau^2 \cdot 100)/(n/\sigma^2 + 1/\tau^2) = (n{\cdot}x + k{\cdot}100)/(n + k)$ with $k = \sigma^2/\tau^2$. CricField fixes k = 120 instead of estimating σ^2 and τ^2 from data, so it is credibility weighting with a chosen prior strength, not fitted empirical Bayes.

- **Evidence:** scout-service.ts:96
- **Follow-ups:** How would you estimate k from the data?
- **Common misconception:** Calling k=120 'empirical'.
- **Exercise:** Estimate τ^2 by method of moments from rankable players' index variance minus mean sampling variance.

A02 · Stats — Your CI is centred on the raw index but you rank by the shrunk index. Is that a problem?

Model answer. Yes — the interval and the headline come from different estimators; the shrunk value can sit at or outside the raw interval (Suryavanshi 137.4 vs [137.4, 157.4]). Options: show raw with CI; or compute the posterior interval for the shrunk value: mean $(n{\cdot}x+k{\cdot}100)/(n+k)$, variance $\approx \sigma^2/(n+k)$.

- **Evidence:** Ch.6 §1.5
- **Follow-ups:** Which would a scout understand better?

A03 · Stats — Why might the strike-rate interval be too narrow for power hitters?

Model answer. It uses the league's per-ball variance; boundary hitters have higher per-ball variance than the league, so their true standard error is larger.

- **Evidence:** scout-service.ts:291-295
- **Follow-ups:** How could you get player-level variance?

A04 · Stats — How would you prove the accuracy of your scouting model?

Model answer. Define the prediction target (next-season index in the same league, or performance after moving leagues), use only data available before the season, compare against baselines (raw rate, league average), report error and rank correlation with confidence intervals, and check calibration of the intervals (do 95% intervals cover ~95%?). None of this has been done yet.

- **Evidence:** Ch.6 §12
- **Follow-ups:** What would count as success?
- **Common misconception:** Using in-sample fit as proof.

A05 · Stats — The trend test pools all earlier seasons. What can go wrong?

Model answer. A player who peaked years ago and declined since can look 'improving' if the latest season beats the pooled mean; seasons aren't independent; multiple metrics × competitions are tested without correction.

- **Evidence:** trends() :656-699
- **Follow-ups:** Propose a better trend model.

A06 · Stats — Evaluate the Bonferroni approach in the Weakness Detector.

Model answer. It controls family-wise error but is conservative when tests are correlated (cells overlap, e.g. 'Spin' and 'Spin|Overs 7–10'), reducing power. Alternatives: Holm–Bonferroni, Benjamini–Hochberg (false discovery rate), or hierarchical/partial pooling of cell effects.

- **Evidence:** weakness-detector.ts
- **Follow-ups:** Which would you choose for a scout-facing tool?

A07 · Stats — How would you validate the match engine's calibration properly?

Model answer. Fit Platt parameters on seasons ≤ 2023; evaluate on 2024–26 with Brier score, log loss and a reliability diagram; compare with a simple baseline (required-rate logistic).

- **Evidence:** match-model.ts
- **Common misconception:** In-sample calibration curves.

A08 · Stats — Is the War Room's kNN price model overfitting or underfitting?

Model answer. Leave-one-out R^2 0.38 and a 2.07× typical miss suggest price is weakly determined by the features; kNN with k=15 and exponential distance weights smooths heavily; the 2026 backtest (ρ 0.72 for price, n=28) suggests it captures market ranking but not value. Sensitivity (up to 39% shifts after an unrelated data fix) indicates instability near neighbour boundaries.

- **Evidence:** auction-war-room.ts; step3 risk 2
- **Follow-ups:** How would you stabilise it?

A09 · Stats — Why is 'AVOID: 5 of 5 delivered' a red flag, and what's the caveat?

Model answer. The signal was contradicted every time it could be checked; but n=5 is too small to conclude much — it is a reason to validate before using signals, not proof the model is wrong.

- **Evidence:** backtest groups

A10 · Data — Design a fix for the IPL warehouse so it is reproducible.

Model answer. One scripted pipeline: acquire Cricsheet (hashing like the global engine) → parse with integer over/ball parsing → store per-type extras → commentary enrichment as a separate idempotent step keyed by delivery → analytics tables → validation (reuse validate.ts checks) → artefact with a version hash. Remove random column generation; remove hand fixes.

- **Evidence:** Ch.5 §8
- **Follow-ups:** Or would you retire sqlite.db and serve IPL from global.db?

A11 · Data — Should the IPL site read from global.db instead of sqlite.db?

Model answer. Arguments for: one source of truth, exact extras, validated, reproducible. Against: hundreds of queries and pages depend on sqlite.db's schema and enrichment (commentary, squads). A staged migration behind the existing service interfaces, verified with golden numbers, is the safe path.

- **Evidence:** Ch.4 §2

A12 · Data — How would you add bowling style for non-IPL bowlers without guessing?

Model answer. Only from named, licensable sources linked by exact id or exact name plus corroboration (team/season), with confidence labels; conflicts recorded; unknown stays unknown and is never treated as spin.

- **Evidence:** enrichment.ts; scout-service.md
- **Follow-ups:** Which sources?

A13 · Architecture — Redesign CricField for 10,000 daily users.

Model answer. CDN + cacheable responses for historical data; precomputed aggregates for Advanced Stats; heavy queries in worker threads or a separate analytics service; read replicas = copies of immutable DB files per instance; rate limiting; route-level code splitting and image optimisation; monitoring and CI. Keep SQLite until concurrency of writes or data size demands Postgres/DuckDB.

- **Evidence:** Ch.11 §6
- **Follow-ups:** What do you do first?

A14 · Architecture — node:sqlite is synchronous. What are your options?

Model answer. Worker threads with a pool of read-only connections; precompute to avoid heavy queries; move to an async driver/database; cache results; set query timeouts.

- **Evidence:** Ch.11 §2.2

A15 · Architecture — How would you deploy CricField reproducibly?

Model answer. Build artefact containing code + checkpointed sqlite.db + global.db + raw JSON (or remove that dependency); container image; CI builds, runs tests and typecheck, then deploys; health check and rollback.

- **Evidence:** Ch.2 §6
- **Follow-ups:** Where should the 120 MB database live?

A16 · Architecture — Your autoscaled host runs several instances. What breaks?

Model answer. In-memory state diverges: the auction game, click counts and caches; each instance warms caches for 12 s; any write goes to one instance's local DB copy.

- **Evidence:** services/auction.ts:429

A17 · Security — Threat-model the public API.

Model answer. Assets: server files, availability, third-party relationships, users' Gemini keys. Threats: file read via sync-csv; DoS via heavy queries and cache bypass; state tampering via auction reset; key leakage in browser; scraping ToS. Controls: remove route, rate limit, cache, auth for admin/state endpoints, server-side LLM proxy.

- **Evidence:** Ch.8 §9, Ch.11 §5

A18 · Security — How would you fix /api/ipl/sync-csv safely if the feature is needed?

Model answer. Accept a dataset id, map it to a whitelisted absolute path under ipl_csv, resolve with path.resolve and verify it stays inside that directory, require admin auth, cap file size, return generic errors.

- **Evidence:** csv-sync-route.ts

A19 · AI — How would you build a grounded LLM analyst on top of the existing code?

Model answer. Server endpoint; LLM receives the question and the StatQuery JSON schema and must call a function returning a StatQuery; validate with statQuerySchema; execute runStatQuery; give the LLM only the result rows to phrase an answer; verify every number in the text exists in the result; evaluate on a labelled question set.

- **Evidence:** decision.ts statQuerySchema; stats-engine.ts header
- **Follow-ups:** How do you measure success?
- **Common misconception:** RAG over match JSON as the first step.

A20 · AI — What is wrong with the Gemini fallback, technically and ethically?

Model answer. It returns pre-written text with specific statistics not computed from data, labelled as model output ('Calibrated Quant Engine'). Technically: ungrounded and untestable. Ethically: it misleads users and undermines the genuine analytics.

- **Evidence:** gemini-client.ts:105-140

A21 · Frontend — The client types drifted from the API. How do you make that impossible?

Model answer. Single source of truth: define response schemas in Zod (or OpenAPI) in a shared package; generate client types and use them in both server handlers and client fetchers; add a contract test in CI.

- **Evidence:** lib/api-spec, lib/api-zod exist but unused

A22 · Frontend — How would you refactor stats.tsx (4,411 lines)?

Model answer. Extract data hooks (query building, fetching), presentational components per panel, and filter state into a typed reducer; delete dead code (FormGuideView); add tests per component; lazy-load heavy panels.

- **Evidence:** pages/stats.tsx

A23 · Performance — Estimate the cost of 1-second auction polling at scale.

Model answer. Each open tab makes 3,600 requests/hour; 500 concurrent viewers → 1.8 M requests/hour for one shared state object. Server-sent events or WebSockets push only changes.

- **Evidence:** lib/live.ts:203

A24 · Product — How do you prove the scouting product is valuable, not just correct?

Model answer. Define a job (build a shortlist of uncapped domestic players), recruit 5–10 analysts, measure time-to-shortlist and quality judged by experts vs their current method; track repeat usage; gather willingness to pay.

- **Evidence:** Ch.24 §6

A25 · Product — Which parts did you build with AI assistance, and what could you rebuild without Claude Code?

Model answer. Implementation was largely AI-assisted. I could rebuild (state honestly after practising): the SQL metrics, a Zod-validated endpoint, a React Query table, the shrinkage/Wilson calculations and the ETL design. I could not yet rebuild the match engine or War Room unaided.

- **Evidence:** Ch.26
- **Follow-ups:** Prove it — here's a laptop.
- **Common misconception:** Over-claiming; or under-claiming the real decisions you made.

A26 · Debugging — Four Step-3 regression tests now fail. How do you proceed?

Model answer. Reproduce each; classify diffs as expected (caused by the 2026-10-11 sequencing fix) or unexpected (e.g., isIndian/team fields changing); fix unexpected ones; regenerate the baseline only with a written explanation; re-run the identity audit and fix the 20 empty pages.

- **Evidence:** Ch.10 §2.1
- **Common misconception:** Just updating the baseline.

A27 · Debugging — A 17-second query became 1 second. What happened?

Model answer. A LEFT JOIN with an extra predicate on player_out_id caused SQLite to probe the per-player index for every delivery; joining on delivery_id only and filtering in CASE let it use the delivery index.

- **Evidence:** project notes (Ch.10 B11)
- **Follow-ups:** How would you have found it?

A28 · Data — How would you model league strength from players who moved between leagues?

Model answer. For players with balls in two leagues in adjacent seasons, compare their indices (or rates) to estimate pairwise league offsets; fit a mixed model with player and league effects; account for selection bias (only good players move) and age; validate on held-out movers.

- **Evidence:** Ch.6 §1.3 limitation

A29 · Testing — Design the minimum CI pipeline for this repo.

Model answer. On every PR: pnpm install (frozen), typecheck all packages, test:global, test:step3 on a fixture DB, supertest API tests, build both apps; block merge on failure; nightly global:validate.

- **Evidence:** Ch.10 §7

A30 · Ethics — A franchise asks to use War Room signals in a real auction. What do you say?

Model answer. Share the backtest honestly (ρ 0.17 for performance on n=9; AVOID contradicted), present it as a market-price lens and an explanation tool, not a recommendation engine, and propose a validation study first.

- **Evidence:** Ch.6 §10.1

29 — Interview and Practical Design Challenges

> Complements Chapter 16 (engineering challenges) with design, product, analytics and mixed take-home tasks of the kind used in UI/UX, product, analyst and engineering interviews. **Do them without AI generation.** Each has a time box, deliverable and rubric. Choosing a design direction is not the same as executing it in Figma — several tasks exist precisely to build that missing evidence.

#	Type	Challenge	Time	Deliverable	Rubric (score 1–5 each)
D1	UI critique	Critique the Scout Lab leaderboard (desktop + 375 px) in 10 minutes aloud, then in writing	30 min	1-page critique with 5 issues ranked by severity and a fix for each	issue quality · prioritisation · evidence (screens, contrast numbers) · actionable fixes
D2	Figma component design	Build a "player rating row" component in Figma with variants: tier (rankable/low/insufficient), discipline (bat/bowl), states (default/hover/selected), mobile/desktop	3 h	Figma file with auto-layout, variants, tokens	component structure · variant logic · tokens used · accessibility (contrast ≥ 4.5:1, ≥ 12 px)
D3	Responsive redesign	Redesign `/stats` for 375 px: filters in a sheet, first table above the fold, card layout for rows	3 h	wireframes (low-fi) + one high-fi screen	task-first layout · progressive disclosure · touch targets ≥ 44 px · realistic content
D4	UX case-study presentation	Present the Scout Lab case study in 8 minutes (Ch.22/27 structure)	2 h prep	8–10 slides	problem clarity · honesty about evidence · iteration story · outcome & next steps
D5	User-flow mapping	Map "scout finds an uncapped death bowler under ₹2 Cr" across Scout → profile → compare → War Room, marking pain points	1 h	flow diagram + pain-point list	completeness · pain points tied to evidence · proposed changes
D6	Product prioritisation	You have 4 weeks. Prioritise 12 items from Chapters 10–11, 21 and 24 with RICE or MoSCoW; justify the top 3	1 h	table + 1-page rationale	framework applied correctly · trade-offs explicit · risk awareness
D7	Metric design	Define activation, engagement and trust KPIs for Ask; specify GA4 events and a dashboard	1 h	event spec + mock dashboard	metric definitions · instrumentability · guardrails
D8	Experiment design	Design an A/B test: default Scout sort by percentile vs by shrunk index	45 min	hypothesis, primary metric, sample-size reasoning, risks	hypothesis quality · metric choice · stats reasoning · ethics
D9	SQL (timed)	Write C1, C2 and a head-to-head query in 30 minutes	30 min	SQL file + results	correctness · edge cases (wides, super overs, renames) · readability
D10	API debugging	Given `curl /api/scout/rankings?discipline=bowling` output, explain why rank 1 has percentile 0 and fix it	30 min	explanation + diff + test	root cause · minimal fix · test
D11	Statistics	Explain to a coach why a 192 SR in 27 balls ranks below 205 in 275 balls; then compute both shrunk indices	20 min	1-paragraph explanation + numbers	plain language · correct maths · uncertainty framing
D12	Engineering design	Whiteboard: make Scout Lab serve 1,000 concurrent users	30 min	diagram + bottleneck list	correct bottleneck (sync SQLite) · caching/CDN · precompute · measurement plan
D13	Accessibility audit	Run axe/Lighthouse on `/ai-analyst/ask`; fix the top 3 issues in a branch	2 h	report + diff	issues found · fixes correct · before/after scores
D14	Data storytelling	One-page brief: "Is death bowling getting more expensive?" using `competition_baselines`	1.5 h	1 chart + 150 words	chart choice · correct numbers · caveats
D15	Ethics & honesty	Rewrite the Gemini fallback and the "Official Analytics Partner" label so nothing misleads	30 min	copy + UI sketch	clarity · honesty · user trust

Model notes (read after attempting)

- **D10:** default `sortBy=shrunk, sortOrder=desc` sorts by `metrics.economy.shrunk` descending; higher economy index = more runs conceded; percentile is direction-aware, so the worst bowler (percentile 0) is first. Fix: use each metric's `better` direction (`metricDefs`) or sort by percentile; add a test asserting rank 1 has the highest economy percentile.
- **D11:** (27 × 123.2 + 12,000)/147 = 104.3 vs (275 × 130.95 + 12,000)/395 = 121.5. Plain language: "With 27 balls we've barely seen him; most of the estimate is the league average. With 275 balls, his own record dominates."
- **D12:** the API is CPU-bound on a single thread with synchronous SQLite; rankings change only when the warehouse is rebuilt → cache the JSON (in memory + CDN) keyed by the query string, precompute default views at build time, add instances behind a load balancer, rate-limit, and measure with a load test before and after.
- **D14 (data):** IPL death-overs run rate in `competition_baselines`: 2024 = 11.21 runs per over; extend to all seasons and compare with the powerplay/middle trend before concluding.

16 — Independent Engineering Challenges

> **Rules:** no AI-generated implementation. You may read documentation (MDN, SQLite docs, React docs) and this handbook's explanations. Work in a **new folder or a scratch copy** — never on the production `sqlite.db` (copy it with its `-wal` file, or open it with `{ readOnly: true }`). Time yourself. The answer key is at the end — attempt first.

Setup once:

```
node -e "const {DatabaseSync}=require('node:sqlite');const db=new DatabaseSync('sqlite.db',{readOnly:true});console.log(db.prepare('select count(*) n from deliveries').get())"
```

C1 — Season batting strike rate (SQL)

- **Prerequisites:** SELECT, JOIN, GROUP BY, HAVING.
- **Instructions:** For IPL season 2025, list the top 10 batters by strike rate with at least 200 balls faced. Exclude super overs. Columns: name, runs, balls, SR (1 dp).
- **Expected result:** Pooran 196.3 (524/267) first; Abhishek Sharma 193.4 second.
- **Rubric:** correct ball definition (wides excluded) 30% · super-over exclusion 20% · correct season filter 20% · readable SQL 15% · explains HAVING vs WHERE 15%.
- **Files:** `04-DATABASE-ENGINEERING.md` Q2; `sqlite.db` schema.

C2 — Bowling economy including extras correctly

- **Prerequisites:** C1, cricket extras rules.
- **Instructions:** Lowest death-overs (overs 16–20) economy since 2022, min 100 legal balls. Charge wides and no-ball penalties to the bowler; not byes/leg-byes.
- **Expected result:** SP Narine 6.06 (218 runs, 216 legal balls).
- **Rubric:** legal-ball definition 25% · conceded-runs expression 35% · phase filter (0-based overs) 20% · explains why `is_bye` can't be used 20%.
- **Files:** `services/decision/db.ts:77-89`.

C3 — A small REST API

- **Prerequisites:** Node, Express basics.
- **Instructions:** In a fresh folder, build `GET /api/players/:name/career` returning runs, balls, SR from a read-only connection to a copy of `sqlite.db`. 404 if the name is unknown; 400 if the name is shorter than 2 characters.
- **Expected result:** `curl /api/players/V%20Kohli/career` → runs 9336, balls 6926, SR 134.8.
- **Rubric:** parameterised SQL 30% · status codes 25% · validation 20% · code structure (route vs service) 25%.
- **Files:** `routes/decision.ts` (pattern), `services/decision/db.ts`.

C4 — A React filter component

- **Prerequisites:** React state, props, `fetch`.
- **Instructions:** Build a table of C1's results with a season dropdown and a minimum-balls slider; refetch on change; show loading and error states.
- **Expected result:** changing season updates the table; an API error shows a message, not a blank page.
- **Rubric:** controlled inputs 25% · loading/error/empty states 30% · no fetch loops 20% · accessible labels 25%.
- **Files:** `pages/scout-home.tsx:88-118`, `lib/aggregates.ts`.

C5 — Add an index and measure it

- **Prerequisites:** C2, EXPLAIN QUERY PLAN.
- **Instructions:** On a **copy** of `sqlite.db`, time C2's query, read its plan, add one index you justify, re-time, re-read the plan.
- **Expected result:** a before/after table with plan lines and milliseconds (baseline ≈ 0.9 s on the audit laptop).
- **Rubric:** measurement method 30% · justified index 30% · correct reading of plans 25% · notes write-cost trade-off 15%.

C6 — Debug a deliberately broken player lookup

- **Prerequisites:** JavaScript, string handling.
- **Instructions:** This function is used to find a player's matches. Find every way it can return the wrong player, write failing tests, then fix it.

```
function findPlayer(bioName, players) {
  const surname = bioName.split(" ").pop().toLowerCase();
  const initial = bioName[0].toLowerCase();
  return players.find(p => p.name.toLowerCase().endsWith(surname) &&
                           p.name[0].toLowerCase() === initial);
}
// players: [{id:"aaa", name:"MD Choudhary"}, {id:"bbb", name:"Mukesh Choudhary"}, {id:"ccc", name:"Prince Choudhary"}]
```

- **Expected result:** tests showing "Prince Choudhary" can match the wrong person in other datasets; a fixed version that requires an exact registry id or exact full-name match and returns *no player* when ambiguous.
- **Rubric:** finds ≥ 3 failure modes 40% · tests 30% · fix returns null on ambiguity 30%.
- **Files:** `scripts/src/step3/tests/identity.test.ts`, `05-DATA-PIPELINES.md §5`.

C7 — Cricket CSV parser (Python or TypeScript)

- **Prerequisites:** file I/O, CSV.
- **Instructions:** Parse one `ipl_csv/<match>.csv` into innings totals and per-batter lines. **Parse the `ball` column as two integers** ("19.10" → over 19, ball 10).
- **Expected result:** innings totals that equal the match result; a test proving "19.10" ≠ "19.1".
- **Rubric:** correct over/ball parsing 30% · extras handled by type 30% · tests 20% · code clarity 20%.
- **Files:** `update-ipl-data-v3.ts:659-661` (the bug to avoid).

C8 — Player comparison visualisation

- **Prerequisites:** a chart library or SVG.
- **Instructions:** For two batters, draw a **dot plot**: one row per metric (SR index, boundary % index, dot % index, balls/dismissal index), a vertical line at 100, error bars for the 95% interval.
- **Expected result:** a readable chart that shows overlap of intervals.
- **Rubric:** correct encoding 30% · reference line & units 25% · uncertainty shown 25% · accessibility (labels, contrast) 20%.
- **Files:** `23-DATA-VISUALIZATION-DESIGN.md`.

C9 — Unit test for a scouting metric

- **Prerequisites:** `node:test`.
- **Instructions:** Write tests for (a) `shrink()` — small samples pull toward 100, large samples stay near their index, n=0 returns 100; (b) `wilsonInterval()` — bounds within [0,100], k=0 has low bound 0.
- **Rubric:** meaningful assertions 40% · edge cases 30% · test names 15% · runs green 15%.
- **Files:** `scout-service.ts:86-98`; `scripts/src/scout/tests/scout.test.ts` (pattern).

C10 — Reproduce the Bayesian adjustment by hand

- **Instructions:** Abhishek Sharma, IPL 2026: 563 runs, 275 balls; league SR 156.34. Compute index, shrunk index (k=120), and the share of the estimate that comes from his own data. Then do the same for 27 balls at SR 192.6.
- **Expected result:** 130.9 → 121.5 (70% own data); 123.2 → 104.3 (18% own data).
- **Rubric:** arithmetic 40% · explains weights 30% · states assumptions 30%.

C11 — Optimise a slow query

- **Instructions:** The legacy `/api/stats/query?season=2025` takes ~2.3 s. Propose and prototype (on a copy) a pre-aggregated table that answers the default batting leaderboard for any season in < 50 ms.
- **Rubric:** schema of the aggregate 30% · correctness check vs original 30% · measured speed-up 25% · refresh strategy 15%.
- **Files:** `global.db agg_batting` (pattern), `stats-aggregator.ts`.

C12 — Trace a browser request through the backend

- **Instructions:** With the app running on test ports, open Scout Lab, choose Bowling. Record: the URL requested, the route handler line, the service functions called, the SQL statements, and the response size and time. Explain why rank 1 has percentile 0.
- **Rubric:** complete trace 40% · correct explanation of the sort bug 40% · proposed fix 20%.
- **Files:** `02-SYSTEM-ARCHITECTURE.md §4`, `routes/scout.ts:128-154`.

C13 — Identify a security vulnerability

- **Instructions:** Review `artifacts/api-server/src/routes/*.ts`. Find the route that lets an unauthenticated caller interact with the server's file system. **Do not exploit it.** Write: impact, likelihood, a fix and a test.
- **Rubric:** finds it 30% · correct impact analysis 30% · safe fix (whitelist + path containment + auth) 30% · test 10%.

C14 — Safe API input validator

- **Instructions:** Write a Zod schema for `/api/scout/rankings` that also enforces: `sortBy` ∈ {shrunk, balls, matches, strike_rate, boundary_pct, dot_pct, economy, wicket_rate}, `competitions` ⊆ known codes, `search` ≤ 40 chars. Return 400 with field-level messages.
- **Rubric:** schema completeness 40% · coercion of numeric strings 20% · error shape 20% · tests 20%.
- **Files:** `routes/scout.ts:64-76`.

C15 — Design an IPL franchise scouting use case

- **Instructions:** One page: a franchise needs an uncapped Indian death bowler for ≤ ₹2 Cr. Specify the data, filters, metrics (with directions), sample thresholds, uncertainty handling, the shortlist format, and what CricField cannot tell them (e.g., bowling type unknown, no opposition-strength adjustment, no fitness data).
- **Rubric:** clear user/job 20% · correct metrics and directions 25% · uncertainty/limitations 30% · actionable output 25%.

ANSWER KEY — attempt the challenges first

C1

```
SELECT p.name, SUM(d.batter_runs) runs, SUM(d.is_wide = 0) balls,
       ROUND(100.0 * SUM(d.batter_runs) / SUM(d.is_wide = 0), 1) sr
FROM deliveries d
JOIN matches m ON m.match_id = d.match_id
JOIN innings i ON i.innings_id = d.innings_id
JOIN players p ON p.player_id = d.batter_id
WHERE m.season_id = '2025' AND i.innings_number <= 2
GROUP BY d.batter_id HAVING balls >= 200
ORDER BY sr DESC LIMIT 10;
```

WHERE filters deliveries before grouping; HAVING filters players after the sums exist.

C2 — see Chapter 4 Q3. Conceded expression: `d.batter_runs + CASE WHEN d.is_wide=1 THEN d.extra_runs ELSE 0 END + CASE WHEN d.is_noball=1 THEN 1 ELSE 0 END`; legal balls: `d.is_wide=0 AND d.is_noball=0`; death: `d.over_num >= 15`. `is_bye/is_legbye` are 0 on every row, so subtracting them changes nothing and would charge every bye to the bowler.

C3 — shape:

```
app.get("/api/players/:name/career", (req, res) => {
  const name = String(req.params.name ?? "");
  if (name.trim().length < 2) return res.status(400).json({ error: "invalid_request" });
  const row = db.prepare(`SELECT SUM(d.batter_runs) runs, SUM(d.is_wide=0) balls
    FROM deliveries d JOIN innings i ON i.innings_id=d.innings_id
    JOIN players p ON p.player_id=d.batter_id
    WHERE p.name = ? AND i.innings_number <= 2`).get(name);
  if (!row || row.balls == null) return res.status(404).json({ error: "player_not_found" });
  res.json({ ...row, sr: +(100 * row.runs / row.balls).toFixed(1) });
});
```

C4 — key points: `useEffect` (or React Query) keyed on `[season, minBalls]`; `AbortController` to cancel stale requests; render `Loading…`, `Error: …`, `No players match` explicitly; `<label htmlFor>` on inputs.

C5 — a reasonable candidate is `CREATE INDEX ix_del_over_bowler ON deliveries(over_num, bowler_id);` so the planner can start from `over_num >= 15` (~25% of rows). Measure; if the season join dominates, an index won't help much and a pre-aggregate (C11) is the better answer. Mention: every index slows inserts and grows the file.

C6 — failure modes: two players sharing initial + surname (MD/Mukesh/Mukul Choudhary all start with M); bios using full names vs scorecard initials ("Prince Choudhary" vs "P Choudhary"); name variants ("Wanindu Hasaranga" vs "PWH de Silva"); first match wins arbitrarily. Fix:

```
function findPlayer(bio, playersById, nameIndex) {
  if (bio.registryId && playersById.has(bio.registryId)) return playersById.get(bio.registryId);
  const hits = nameIndex.get(bio.fullName) ?? [];
  return hits.length === 1 ? hits[0] : null;   // ambiguous or unknown → empty state
}
```

C7 — `const [o, b] = ball.split(".").map(Number);` Never `parseFloat`. Bowler conceded = bat runs + wides + noballs; batter balls = rows where wides == 0.

C8 — x-axis 0–200 index with a line at 100; one row per metric; dot = index; whiskers = CI; label direction ("lower is better" for dot %).

C9

```
test("shrink pulls small samples toward 100", () => {
  assert.ok(Math.abs(shrink(150, 10)! - 100) < Math.abs(150 - 100));
  assert.ok(Math.abs(shrink(150, 5000)! - 150) < 2);
  assert.equal(shrink(150, 0), 100);
});
test("wilson bounds", () => {
  const w = wilsonInterval(0, 20)!; assert.equal(w.low, 0); assert.ok(w.high > 0 && w.high < 100);
});
```

C10 — index = 100 × 204.73 / 156.34 = 130.95. Shrunk = (275×130.95 + 120×100)/395 = 121.5. Own-data weight = 275/395 = 69.6%. Small sample: (27×123.2 + 12,000)/147 = 104.3; weight 27/147 = 18.4%. Assumption: k=120 balls is the prior strength; it was chosen, not estimated.

C11 — table `bat_season(player_id, season, phase, runs, balls, fours, sixes, outs)`; build with one GROUP BY over deliveries; leaderboard = `SELECT … FROM bat_season WHERE season=? AND phase='all' AND balls>=?`; verify equality with the original query for 3 seasons; refresh after each ingest.

C12 — URL `/api/scout/rankings?discipline=bowling&…&sortBy=shrunk&sortOrder=desc`; handler `routes/scout.ts:78`; `ratePlayers → competitionRatings` (aggregates, variance, coverage, roles, names, teams SQL); sort at `:128-154` uses `metrics.economy.shrunk` descending — higher economy index means *more* runs conceded, so the worst bowlers rank first; percentile (direction-aware) is 0 for them. Fix: sort by `better` direction or by percentile.

C13 — `POST /api/ipl/sync-csv` (`routes/csv-sync-route.ts`) reads `filePath` from the body and calls `fs.existsSync`/`fs.readFileSync` on it (`services/cricsheet-csv-sync.ts:38-43`). Impact: file existence disclosure, partial content disclosure, resource exhaustion. Fix: remove; or accept an id → whitelist map; `path.resolve` + `startsWith(allowedDir)`; admin auth; size cap; generic errors. Test: request with `../../package.json` returns 400.

C14

```
const COMPS = ["ipl","smat","bbl","sa20","cpl","ilt20"] as const;
const schema = z.object({
  discipline: z.enum(["batting","bowling"]).default("batting"),
  competitions: z.string().optional().transform(s => s?.split(",").map(x => x.trim().toLowerCase()))
    .refine(a => !a || a.every(c => (COMPS as readonly string[]).includes(c)), "unknown competition"),
  sortBy: z.enum(["shrunk","balls","matches","strike_rate","boundary_pct","dot_pct","economy","wicket_rate"]).default("shrunk"),
  search: z.string().max(40).optional(),
  minBalls: z.coerce.number().int().min(0).max(10000).default(120),
});
```

C15 — filters: SMAT + IPL, phase death, recent 2–3 seasons, `minBalls ≥ 120`, capped status = uncapped (only where `player_attributes` has corroborated status); metrics: economy (lower), boundary % conceded (lower), dot % (higher), wicket rate (higher) — rank by shrunk index *with direction*; show CI and balls; flag `bowling type unknown`; output: 8–10 names with evidence lines and limitations (no opposition-strength adjustment; SMAT Plate Group share; no fitness/age data).

17 — Personalised Learning Roadmap

> **Goal:** move from *understanding* CricField to *independently maintaining and extending* it, prioritising the skills that unlock the most entry-level roles (data/sports analyst, analytics/data engineer, technical product analyst). Effort estimates assume ~12–15 focused hours a week. Nothing here assumes you already know a topic because the repository uses it.

Stage 1 — Foundations (weeks 1–4)

Skill	What to learn	Why it matters	Where CricField uses it	Effort	Hands-on assignment	Completion criteria
SQL	SELECT/JOIN/GROUP BY/HAVING/CASE, subqueries, CTEs, window functions	core of every analytics role	Ch.4 Q1–Q9; `stats-engine.ts`	15 h	C1, C2; rewrite Q1–Q9 from memory	all 9 queries reproduce the handbook numbers, unaided, in < 90 min
Cricket data rules	legal ball, faced ball, conceded runs, dismissals, phases	correctness of every metric	`metrics.ts`, Ch.5 §6	4 h	hand-score 3 overs with extras	100% on a 10-question quiz you write from `metrics.md`
Statistics I	mean/variance, rates vs counts, binomial proportion, standard error, percentiles	interpreting any metric	Ch.6 §1	12 h	compute SR, its SE, a Wilson CI by hand for 3 players	matches `ratePlayers` output to 1 dp
JavaScript/TypeScript	types, unions, async/await, modules, the event loop	reading and changing the codebase	everywhere	15 h	small CLI that reads a match JSON and prints a scorecard	runs, typed, no `any`
Git	branches, small commits, diffs, `git apply`	your evidence trail	274 uncommitted paths	4 h	commit the current work in 6–10 logical commits with messages	a history a reviewer can follow
Debugging	reproduce → isolate → hypothesise → verify	interview stories	Ch.10 B1–B12	6 h	reproduce bug B4 on a DB copy	you can explain B2 and B4 in 2 minutes each

Stage 2 — Data and backend (weeks 5–9)

Skill	What to learn	Why	Where	Effort	Assignment	Done when
Database design	grain, keys, normalisation, indexes, EXPLAIN	warehouse work	Ch.4	10 h	C5	before/after timing table with plan lines
ETL design	idempotency, incremental loads, hashing, validation, lineage	data engineering interviews	`scripts/src/global/*`	15 h	C7 + a mini incremental loader for 20 matches	rerun gives identical output; a changed file is reloaded
Python + pandas	dataframes, groupby, merges, plotting	most analyst roles expect it	(not in CricField's analytics)	15 h	redo C1/C2 in pandas from the CSVs; one notebook	notebook reproduces SQL numbers
REST APIs	Express routing, middleware, status codes	backend roles	Ch.8	10 h	C3, C14	endpoint with validation and tests
Testing	unit vs integration, fixtures, HTTP tests	missing from the repo	Ch.10	10 h	C9 + supertest tests for C3	tests run in CI-like script
Security basics	OWASP Top 10, injection, path traversal, auth	release blockers	Ch.11 §5	6 h	C13	written fix + test

Stage 3 — Frontend and product (weeks 10–13)

Skill	What to learn	Why	Where	Effort	Assignment	Done when
React + React Query	components, state, effects, query keys	frontend & full-stack	Ch.7	15 h	C4	filter table with loading/error/empty states
Data visualisation	encodings, uncertainty, colour, accessibility	analyst & design roles	Ch.23	8 h	C8	dot plot with CIs reviewed by a peer
Accessibility	WCAG contrast, semantics, keyboard, zoom	design & frontend	Ch.20 §6	6 h	fix contrast/zoom in a branch; run Lighthouse/axe	report with 0 critical issues on one page
BI tool	Power BI or Tableau	data analyst job descriptions	—	10 h	dashboard from exported Scout rankings	published dashboard with 3 views
Product thinking	personas, jobs-to-be-done, PRDs, KPIs	APM / product analyst	Ch.24	8 h	PRD for Scout shortlists	PRD reviewed by someone outside the project
User research	interview guides, synthesis	design/product roles	Ch.27	8 h	5 interviews	synthesis board + 3 insights

Stage 4 — Advanced engineering (weeks 14–18)

Skill	What to learn	Why	Where	Effort	Assignment	Done when
Shrinkage & hierarchical models	normal–normal model, empirical Bayes, estimating k	defend Scout Lab	Ch.6 §1.4	10 h	estimate k from data (A01 exercise)	written derivation + comparison of rankings

Skill	What to learn	Why	Where	Effort	Assignment	Done when
Hypothesis testing	z-tests, multiple comparisons, FDR	weakness detector	Ch.6 §7	8 h	rerun a weakness scan with Benjamini–Hochberg	comparison table
Model validation	time-based splits, leakage, calibration, Brier	any modelling claim	Ch.6 §9–10	12 h	holdout calibration of the win model	reliability diagram + Brier score
Performance & scalability	caching, workers, precomputation	system design interviews	Ch.11	8 h	C11	measured speed-up
Deployment & CI	GitHub Actions, artefacts, environments	full-stack/data eng	Ch.2 §6	8 h	CI running typecheck + tests	green pipeline on a PR
Grounded LLM features (optional)	function calling, validation, evals	AI roles	Ch.9 §4	12 h	LLM → StatQuery with 50-question eval	accuracy reported honestly

Stage 5 — Interview readiness (weeks 19–22)

Activity	Done when
Project defence	you can give a 3-minute and a 10-minute CricField talk covering architecture, one bug story, one limitation
Question bank	80% of Chapter 15 answered aloud without notes
SQL interviews	30 timed SQL problems (joins, windows, aggregation)
Coding	40 easy/medium problems (arrays, hashing, sorting, two pointers) for SWE tracks
Architecture	whiteboard Ch.2 diagram + the 10k-user redesign in 15 min
Independent implementation	C1–C15 completed with scores ≥ 70% on each rubric

---

Weekly rhythm

- 60% building (challenges), 25% reading/learning, 15% writing (explanations in your own words — the best test of understanding).
- After each stage: record a 3-minute video explaining one thing you built *without* notes.

18 — Resume and Career Evidence

> **Principle:** every bullet must survive the follow-up "show me" and "how do you know?". No users, revenue, model accuracy or performance benchmarks are claimed beyond what this audit measured. Where the honest phrasing is "directed / designed with AI-assisted implementation", say so — interviewers respect it and will test it anyway.

---

1. Evidence table

#	Claim	Supporting code	Supporting tests / measurements	Defensible?	Likely interview questions	Demonstrate first
1	Built a cricket analytics web app (React, TypeScript, Express, SQLite) over 1,243 IPL matches / 295k deliveries	`artifacts/*`, `sqlite.db`	row counts (Ch.4)	✅ with "AI-assisted implementation"	architecture walk-through; why SQLite	Ch.2 diagram from memory
2	Designed a six-league T20 data warehouse (3,309 matches, 772k deliveries) with incremental, hash-verified ingestion and 118 automated validation checks	`scripts/src/global/*`	validation report 118 pass / 0 fail; 23 tests pass	✅	idempotency, lineage, what a check caught	C7; explain `validate.ts` checks
3	Implemented league-adjusted player ratings with credibility shrinkage and 95% Wilson/normal confidence intervals	`scout-service.ts`	unit tests; worked example reproduced	✅ (say "k fixed at 120, not fitted")	derive shrinkage; interval types; cross-league validity	C10, I13–I16
4	Identified and fixed data-quality defects: bowler economy overcharged by 7,142 runs; ball-sequence wrap on 10+ ball overs; 33 MB page payload cut to a 67 KB index + per-player shards	`decision/db.ts:77-89`; `delivery_seq`; Step 3 docs	regression reports; delivery-sequence tests	✅	how found, how verified, what remains (17 deliveries)	tell B2/B4 stories
5	Built deterministic natural-language querying that compiles English questions into a validated, typed query (no generated SQL)	`nl-parser.ts`, `stats-engine.ts`	live: 78 ms answer matching an independent recount	✅	why not an LLM; how to add one safely	demo Ask; explain StatQuery
6	Developed statistical decision tools: two-sample z-tests with Bonferroni correction, Monte Carlo match simulation with Platt-calibrated win probability, MILP Playing-XI optimiser	`weakness-detector.ts`, `match-model.ts`, `playing-xi.ts`	no automated tests; calibration in-sample	🟡 claim the methods, not accuracy	multiple comparisons; calibration validation	A06, A07
7	Built an auction valuation model (kNN comparable sales) with a leakage-free backtest	`auction-war-room.ts`	backtest: price ρ 0.72 (n 28); performance ρ 0.17 (n 9)	🟡 only with the numbers	"Did it work?" — answer with the backtest	I21, A08
8	Measured API performance and identified the synchronous-SQLite bottleneck (10 concurrent queries → 24 s)	—	this audit's measurements	✅ if you re-run them	caching/worker plan	re-run the measurements yourself (method in Ch.8 header: built API on a test port, cold + median-of-3 warm timings, 10/50 concurrent requests)
9	Conducted a full technical, security and accessibility audit of my own product	`docs/architecture-handbook/*`	findings with evidence	✅ (state AI-assisted audit)	top 3 findings and fixes	AUDIT-SUMMARY in your words
10	"AI-powered cricket analyst"	Gemini client	none	❌ do **not** use	—	build grounded LLM first
11	"Machine learning scouting model"	—	—	❌ (it's statistics + kNN)	—	—
12	"Supports thousands of users / production-grade"	—	measurements show the opposite	❌	—	—
13	"Improved engagement / users loved it"	—	no data	❌	—	measure first
14	"Designed the UI in Figma"	—	no Figma	❌	—	Ch.27 plan
15	"Identifies undervalued auction players"	—	backtest contradicts	❌	—	—

2. Claim categories

- **Safe, evidence-backed:** 1, 2, 3, 4, 5, 8, 9.
- **Need further validation / careful wording:** 6 (methods yes, accuracy no), 7 (report the backtest).
- **Must not appear:** 10–15.

---

3. Three project descriptions

A. Data Analyst / Analytics Engineer

CricField — T20 cricket analytics warehouse & scouting engine *(personal project; AI-assisted implementation)*

- Designed a six-league T20 warehouse from Cricsheet ball-by-ball data (3,309 matches, 772k deliveries) with incremental, hash-verified loads and 118 automated validation checks (0 failures).
- Defined cricket metrics as tested SQL/TypeScript rules (legal balls, runs conceded, phases) and built league-adjusted player ratings with shrinkage and 95% confidence intervals.
- Found and fixed data-quality defects including 7,142 bye/leg-bye runs wrongly charged to bowlers and a ball-numbering error in 10+-delivery overs, verified against a recount of raw files.

B. Full-Stack / Backend Engineer

CricField — cricket analytics web app *(React 19, TypeScript, Express 5, Zod, SQLite; AI-assisted implementation)*

- Built a 123-endpoint Express API (Zod-validated contracts on the decision and scouting routes) and a React/React Query front end serving player, team, match and scouting views.
- Reduced the player page's data payload from 33 MB to a 67 KB index plus per-player shards (median 4 KB), fixing a page crash.
- Profiled the API (synchronous SQLite blocking: 10 concurrent heavy queries → 24 s) and documented a caching/precomputation plan; audited and documented security issues for remediation.

C. Sports Data / Product Analyst

CricField — IPL decision-support product *(product direction, data analysis; AI-assisted implementation)*

- Directed a cricket decision-support product for fans and franchise analysts: natural-language stats queries, opposition weakness scans with multiple-testing control, match simulations and auction valuation.
- Backtested the auction valuation against the real 2026 auction without look-ahead (price rank correlation 0.72 on 28 sales) and reported its limits (performance correlation 0.17 on 9 players).
- Scoped and gated releases: separate warehouse for new data, patch-review-before-apply, and explicit documentation of unresolved data issues.

> Adjust numbers only after re-verifying them yourself. Replace "AI-assisted implementation" with your own accurate phrasing — but keep the substance.

19 — Technical Glossary

Terms are defined as they are used **in CricField**, with the file where you meet them.

Cricket and cricket data

Term	Meaning
Delivery / ball	one bowling action; one row in `deliveries`
Legal ball	a delivery that is neither a wide nor a no-ball; counts toward the over
Ball faced	every delivery except a wide (no-balls are faced)
Wide / no-ball	illegal deliveries; wide runs and the no-ball penalty are charged to the bowler
Bye / leg-bye	runs taken when the ball misses the bat (bye) or deflects off the batter's body (leg-bye); **not** charged to the bowler; counts as a batting dot
Penalty runs	awarded by umpires; not charged to the bowler
`non_boundary`	Cricsheet flag: 4 or 6 runs were *run*, not a boundary
Runs conceded	bat runs + wide runs + no-ball penalty (`RUNS_CONCEDED_SQL`)
Strike rate (batting)	100 × runs ÷ balls faced
Economy	6 × runs conceded ÷ legal balls
Bowling strike rate	legal balls ÷ wickets
Dot ball	batting: faced ball with 0 bat runs; bowling: legal ball with 0 runs conceded
Phases	powerplay overs 1–6, middle 7–15, death 16–20
Super over	one-over tie-breaker; excluded from career stats
DLS	rain-rule method that revises targets; targets read from raw files
Retired out / retired hurt	retired out is a dismissal (not a bowler's wicket); retired hurt is not a dismissal
Cricsheet	open ball-by-ball cricket data source (ODC-BY)
Registry id	Cricsheet's stable 8-hex person id (`players.cricsheet_id`)
SMAT, BBL, SA20, CPL, ILT20	Syed Mushtaq Ali Trophy (India), Big Bash (Australia), SA20 (South Africa), Caribbean Premier League, International League T20 (UAE)
Capped / uncapped	whether a player has played international cricket (IPL auction category)

Statistics and modelling

Term	Meaning in CricField
League rate / baseline	league total ÷ league balls for a competition, season and phase
Expected count	player's balls × league rate, summed over seasons
Index	100 × actual ÷ expected; 100 = league average
Shrinkage	pulling an estimate toward the league mean in proportion to sample size: (n·x + k·100)/(n + k), k = 120
Credibility weight	n/(n + k): the share of the estimate from the player's own data
Empirical Bayes	shrinkage where the prior is estimated from data — CricField fixes k instead
Confidence interval (95%)	frequentist range that would contain the true value in 95% of repeated samples
Credible interval	Bayesian posterior interval — **not** what CricField computes
Wilson score interval	CI for a proportion that behaves well for small n
Normal approximation	CI = estimate ± 1.96 × SE
Standard error	standard deviation of an estimate's sampling distribution
Percentile (mid-rank)	position in the sorted population, ties share the average rank, 100 = best
Sample tier	rankable ≥ 120 balls, low 60–119, insufficient < 60
z-test	(difference of means) ÷ standard error
Bonferroni correction	multiply p-values by the number of tests to control false positives
False discovery rate	alternative error control (Benjamini–Hochberg)
Monte Carlo	estimating outcomes by simulating many random innings
Seed	fixed random start so simulations are reproducible
Dynamic programming (win probability)	exact recursion over (balls left, runs needed, wickets lost)
Calibration	predicted probabilities match observed frequencies
Platt scaling	logistic re-mapping of raw probabilities
Brier score	mean squared error of probability forecasts
Impact	runs per match above an average player (impact model)
IAR	impact above replacement (20th-percentile regular)
kNN regression	prediction = weighted average of the k most similar past examples
Leave-one-out (LOO)	evaluate each example with a model trained on all the others
Backtest	evaluate a method on a past period using only data available then

Term	Meaning in CricField
Look-ahead / leakage	accidentally using future data in a backtest
Spearman correlation	correlation of ranks
z-score (feature)	(value − mean)/SD, used before computing similarity
MILP	mixed-integer linear programme (Playing XI)

Engineering

Term	Meaning
Monorepo / workspace	one repository containing several packages (pnpm)
SPA	single-page application (React app routed in the browser)
React Query / query key	server-state cache; the key identifies the cached response
`staleTime`	how long cached data is considered fresh
Middleware	functions that run on every request before route handlers (Express)
Zod / schema	runtime validation + inferred TypeScript types
Prepared statement	SQL with `?` placeholders; values bound separately (prevents injection)
WAL	SQLite write-ahead log (`sqlite.db-wal`)
`WITHOUT ROWID`	SQLite table clustered by its primary key
Index / EXPLAIN QUERY PLAN	lookup structure / SQLite's description of how it will run a query
ETL	extract, transform, load
Idempotent	running twice gives the same result
Incremental load	only new or changed inputs are processed
Lineage	traceability from a number back to its source
Schema drift	source format changes over time
Event loop	Node's single thread that runs JavaScript; blocking it delays all requests
Bundle / code splitting	the JS file(s) the browser downloads / splitting them per route
esbuild / Vite	bundlers for the API and the web app
CORS	browser rule controlling cross-origin requests
Path traversal	reading files outside an intended folder via crafted paths
Rate limiting	capping requests per client per time window
CDN	content delivery network caching responses near users
CI/CD	automated build/test (and deploy) on every change
Contract test	test that client and server agree on request/response shapes

Product and design

Term	Meaning
Persona	evidence-based archetype of a user (proposed only in this handbook)
Job-to-be-done	the progress a user is trying to make
IA	information architecture: how content is organised and navigated
Heuristic evaluation	expert review against usability principles
WCAG AA	accessibility standard; e.g. 4.5:1 text contrast
Design token	named design value (colour, size, spacing) shared across components
KPI	key performance indicator
PRD	product requirements document
A/B test	randomised comparison of two variants